

Apigee X vs. Apigee Hybrid

An Architect-Level Guide to Control Plane, Runtime Plane, and Kubernetes Architecture on Google Cloud

Prepared for Sai Rathish — Prodapt-AID Practice

August 2026

Contents

How to Use This Handbook	7
A note on sourcing and intellectual honesty	7
Section 1 — Introduction to API Management	8
1.1 What Is API Management?	8
1.2 Why Apigee Exists	8
1.3 Apigee Product Evolution	9
1.3.1 Apigee Edge	9
1.3.2 Apigee Hybrid	9
1.3.3 Apigee X	9
1.3.4 The single question that drives the rest of this handbook	10
Section 2 — Apigee X Architecture	11
2.1 Apigee X in One Picture	11
2.2 The Resource Hierarchy	11
2.2.1 Organization	12
2.2.2 Instance	12
2.2.3 Environment	12
2.2.4 Environment Group	12
2.2.5 API Proxy, Policy, API Product, Developer App	13
2.3 The Apigee X Runtime, in Detail	13
2.4 Networking Architecture	13
2.4.1 VPC Peering / Private Services Access (PSA)	13
2.4.2 Private Service Connect (PSC)	14
2.4.3 Northbound vs. Southbound — the Official Security Pattern	14
2.4.4 Scaling Past One Peering Hop: Hub-and-Spoke and Shared VPC	14
2.5 Component Responsibility and Request-Flow Summary	15
Section 3 — Apigee Hybrid Architecture	16
3.1 The Split: Management Plane and Runtime Plane	16
3.2 Runtime Plane Components, in Detail	17
3.2.1 MART (Management API for Runtime [Data])	17
3.2.2 Synchronizer	17
3.2.3 Cassandra	18
3.2.4 Message Processor (apigee-runtime)	18
3.2.5 Ingress Gateway	19
3.2.6 Telemetry (Prometheus, OpenTelemetry, logging)	19
3.2.7 UDCA (Universal Data Collection Agent)	19
3.2.8 Watcher (apigee-watcher)	19
3.2.9 Apigee Connect Agent	20
3.3 Where Kubernetes Actually Fits	20
3.4 Supported Kubernetes Platforms	20
Section 4 — Kubernetes Deep Dive for Apigee Hybrid	21
4.1 Cluster and Node	21
4.2 Namespace	21
4.3 Pod	21
4.4 Deployment	21
4.5 StatefulSet	22
4.6 Service	22
4.7 Ingress and the Istio Ingress Gateway	22
4.8 ConfigMap	22
4.9 Secret	22

4.10 PersistentVolume	23
4.11 HorizontalPodAutoscaler (HPA)	23
4.12 Putting It Together: the GKE Cluster Layout	23
4.13 Illustrative Configuration Example	23
Section 5 — Control Plane vs. Runtime Plane	25
5.1 Apigee X: Both Planes, One Owner	25
5.2 Apigee Hybrid: Two Planes, Two Owners	25
5.3 Responsibility Matrix	25
5.4 Why Is Kubernetes Hidden in Apigee X?	27
Section 6 — End-to-End Traffic Flow	28
6.1 Apigee X Request Flow	28
6.2 Apigee Hybrid Request Flow	29
6.3 The One Structural Difference That Matters Most	29
Section 7 — Configuration Flow and Synchronization	30
7.1 Proxy Deployment, End to End	30
7.2 Shared Flows	30
7.3 Key Value Maps (KVMs)	31
7.4 Environment Configuration and Routing	31
7.5 The Synchronization Process, Summarized	31
Section 8 — Production Enterprise Architecture	32
8.1 An Honest Starting Point	32
8.2 Single-Region Apigee X	33
8.3 Multi-Region Apigee X	34
8.4 Single-Region Apigee Hybrid	35
8.5 Multi-Region Apigee Hybrid	36
8.6 Disaster Recovery Architecture	37
8.7 High Availability Architecture — Summary Checklist	37
Section 9 — Troubleshooting Architecture	38
9.1 Runtime Pod (apigee-runtime / Message Processor) Failure	38
9.2 Synchronizer Failure	38
9.3 Cassandra Failure	38
9.4 MART Failure	39
9.5 Ingress Failure	39
9.6 Failure-Impact Comparison Table	39
9.7 The Broader Resilience Narrative	39
Section 10 — Apigee X vs. Apigee Hybrid	40
10.1 Comparison Matrix	40
10.2 Architect Recommendations	41
Section 11 — Interview Preparation	43
11.1 Top 50 Apigee X Architecture Questions	43
Q1. What is an Apigee organization, and why does it matter that it is bound 1:1 to a Google Cloud project?	43
Q2. What is an Apigee Environment?	43
Q3. What is an Environment Group, and why does it exist as a separate layer above Environments?	43
Q4. What is an Instance, and what is the hard limit around instances per region?	44
Q5. Can an environment belong to multiple environment groups, and what does that enable architecturally?	44
Q6. Walk through the full Apigee X resource hierarchy from organization down to API proxy.	44
Q7. What is the difference between an API Product and an API Proxy?	44
Q8. What is a Developer App, and how does it relate to API Products?	44
Q9. What is a Revision, and can two revisions of the same proxy be live in an environment at once?	44

Q10. What are the environment and environment-group limits per organization, and when would you actually hit them?	45
Q11. What is the Message Processor in Apigee X, and is it still current, official terminology? . . .	45
Q12. Where does the Message Processor physically run in Apigee X?	45
Q13. Does Google disclose whether Apigee X’s runtime is GKE specifically?	45
Q14. What is a “tenant project” in Apigee X, and why does the customer never see it directly? . .	45
Q15. How does Apigee X scale within a single region — at the instance level or the pod level? . .	46
Q16. Why can’t you inspect or SSH into the Kubernetes cluster backing Apigee X?	46
Q17. What is Private Services Access (PSA), and how does Apigee X use it?	46
Q18. Why is pure VPC Peering a scaling problem when Apigee X needs to reach many backend VPCs?	46
Q19. What is the Shared VPC hub pattern, and why does it get around the peering-transitivity limit? .	46
Q20. What is Private Service Connect, and how does it differ from VPC Peering for Apigee X southbound traffic?	46
Q21. Describe the PSC pattern for reaching a backend like Cloud Run from Apigee X.	47
Q22. What is the distinction between northbound and southbound traffic in Apigee X, and what handles each?	47
Q23. What is the recommended pattern for edge security in front of Apigee X, and why not rely purely on Apigee’s own policies?	47
Q24. How does Apigee X handle egress to the public internet by default?	47
Q25. When would you use a “wheel” topology instead of hub-and-spoke for Apigee X’s VPC connectivity?	47
Q26. How do you reach an on-premises backend from Apigee X?	48
Q27. What are the three policy families the Cloud Architecture Center groups Apigee’s most common policies into?	48
Q28. What’s the architectural difference between OAuthV2 and VerifyAPIKey as security policies? .	48
Q29. When would you use SpikeArrest versus Quota?	48
Q30. Why does Google recommend Cloud Armor over relying solely on Apigee’s native Access-Control policy for IP- and geo-based blocking?	48
Q31. What do JSONThreatProtection and XMLThreatProtection guard against?	48
Q32. How would you use a Shared Flow plus a Flow Hook to enforce a mandatory security policy across an entire environment?	49
Q33. How does Apigee X’s analytics pipeline work at a high level?	49
Q34. What data does Apigee’s analytics pipeline capture per transaction?	49
Q35. Is there a component called “UAP” or “Unified Analytics Platform” in Apigee X?	49
Q36. How would you expose a custom business metric through Apigee analytics?	49
Q37. Walk through what happens end-to-end when you deploy a proxy revision in Apigee X. . . .	50
Q38. What tools can trigger a proxy deployment in Apigee X?	50
Q39. What happens if you try to deploy two revisions that share the same base path to the same environment?	50
Q40. How would you implement a blue/green deployment using Apigee X’s resource model? . . .	50
Q41. What is a Shared Flow, and how does it differ from an API proxy?	50
Q42. What is a KVM, and what are its three scopes?	50
Q43. What is the deployed-proxies-per-environment limit in Apigee X, and why does it matter architecturally?	51
Q44. How many key stores are allowed per environment, and why does that matter for TLS-heavy, multi-tenant designs?	51
Q45. What is the max number of regions/instances per organization, and how does it tie into the underlying Cassandra pod constraint mentioned in Apigee’s limits documentation?	51
Q46. What is the API base-path limit per organization, and what design pattern helps avoid hitting it? .	51
Q47. What is the exact Apigee X SLA tiering?	51
Q48. Does Apigee X publish RTO/RPO figures, and what philosophy does Google offer instead? .	52
Q49. If an entire GCP region goes offline, does Apigee X automatically fail your traffic over to another region?	52
Q50. When would you recommend a customer accept a single-region 99.9% Apigee X deployment rather than paying for multi-region 99.99%?	52
11.2 Top 50 Apigee Hybrid Architecture Questions	52
Q1. What is the fundamental architectural split in Apigee Hybrid?	52

Q2. What does Google operate in Apigee Hybrid, contractually?	52
Q3. What does the customer operate in Apigee Hybrid, contractually?	53
Q4. Why is it significant that Google’s shared-responsibility page says it “delivers packages” rather than “operates the runtime”?	53
Q5. What does it mean in practice that “the runtime plane can function independently from the management plane”?	53
Q6. What’s the single diagnostic question that determines whether a client can realistically absorb Hybrid’s split responsibility model?	53
Q7. What is MART, and what does it do?	53
Q8. What happens to MART when Cassandra becomes unreachable?	53
Q9. What is the Synchronizer, and why is it described as a “pull model”?	54
Q10. What is the actual traffic impact when the Synchronizer loses connectivity to the control plane?	54
Q11. What data does Cassandra store in Apigee Hybrid, and where does that documentation come from?	54
Q12. What replicaCount does Google recommend for Cassandra, and why is a single-replica Cassandra dangerous?	54
Q13. Is Cassandra being deprecated in favor of Redis in Apigee Hybrid?	54
Q14. What is the Message Processor’s role in Apigee Hybrid, and how does it relate to configuration delivered by the Synchronizer?	55
Q15. What is the runtime.release.strategy setting, and what does it control?	55
Q16. What kind of ingress does Apigee Hybrid use, and is it a plain Kubernetes Ingress object?	55
Q17. What is the documented failure mode for Hybrid’s ingress layer, and how severe is it?	55
Q18. What telemetry stack does Apigee Hybrid use, and what custom resource governs it?	55
Q19. Does losing telemetry components take down traffic in Apigee Hybrid?	55
Q20. What is UDCA, and what does it forward?	56
Q21. What happens to analytics data during an extended control-plane connectivity loss?	56
Q22. What does apigee-watcher do, and what evidence proves it (not a separate cert-manager) reconciles ingress TLS state?	56
Q23. What Kubernetes deployment type does apigee-watcher use, and at what scope?	56
Q24. What does the Apigee Connect Agent do, and why does it matter for a customer’s firewall posture?	56
Q25. What Kubernetes deployment type backs the Apigee Connect Agent, and what’s unusual about it compared to the other components?	56
Q26. Which Apigee Hybrid components are Deployments, and which is the one StatefulSet — and why?	57
Q27. What custom resources does the Apigee Operator manage in Hybrid?	57
Q28. What is the default Kubernetes namespace for an Apigee Hybrid install, and can it be changed?	57
Q29. How are TLS certificates and GCP service-account credentials stored and secured inside the Hybrid cluster?	57
Q30. What Kubernetes object gives Cassandra pods stable, addressable network identity within the cluster?	57
Q31. Walk through the full failure cascade when Cassandra goes down in Apigee Hybrid.	57
Q32. Walk through the failure impact when the Synchronizer loses control-plane connectivity, and contrast it with the Cassandra scenario.	58
Q33. Is there a documented “MART alone is down” failure scenario independent of Cassandra?	58
Q34. Walk through the failure impact when ingress TLS secrets are missing or misconfigured.	58
Q35. What happens when the Message Processor itself fails its readiness probe, independent of Cassandra or Synchronizer issues?	58
Q36. Rank the five documented Hybrid failure scenarios by severity and explain the ranking.	58
Q37. What is the “resilience, not isolation” narrative, and why does it matter in a client-facing design conversation?	59
Q38. Give three concrete Google responsibilities and three concrete customer responsibilities from the shared-responsibility model.	59
Q39. Who is responsible for patching the Kubernetes nodes underneath an Apigee Hybrid cluster?	59
Q40. Who owns TLS certificate lifecycle for northbound API traffic in Apigee Hybrid?	59
Q41. Who is responsible for multi-region deployment and disaster-recovery design in Apigee Hybrid?	59
Q42. What is Google’s documented production baseline for how many Kubernetes clusters a Hybrid deployment should run, and how should they be sized relative to each other?	60

Q43. How does Cassandra scale across multiple regions/clusters in a multi-region Hybrid deployment?	60
Q44. Does Apigee Hybrid have a published SLA?	60
Q45. Given that Google publishes no SLA or RTO/RPO for Hybrid, how would you architect DR for it from first principles?	60
Q46. What Kubernetes platforms are confirmed supported for Apigee Hybrid, and what's the EKS-specific gotcha?	60
Q47. Is Apigee Hybrid supported on Anthos on-premises or bare-metal deployments?	60
Q48. Walk through the full deployment flow in Apigee Hybrid, from trigger to traffic serving.	61
Q49. What is apigeectl, and how does the newer Helm-based install path differ from it?	61
Q50. Why would an enterprise choose Apigee Hybrid over Apigee X purely for regulatory reasons, and what does that choice cost them operationally?	61
11.3 Top 30 Kubernetes Questions Related to Apigee Hybrid	61
Q1. What is a Kubernetes Cluster and Node, and what are Google's documented minimum sizing recommendations for Apigee Hybrid?	61
Q2. What is a Pod, and which Apigee Hybrid components run as pods?	62
Q3. What is a Deployment, and which Hybrid components use it?	62
Q4. What is a StatefulSet, and why is Cassandra the only Hybrid component that uses one?	62
Q5. What is a Service, and how do Hybrid components address each other internally?	62
Q6. What is a Kubernetes Ingress, and why does Apigee Hybrid use an Istio-based gateway instead of a bare Ingress object?	62
Q7. What is a ConfigMap, and what does Apigee Hybrid store in one?	62
Q8. What is a Kubernetes Secret, and what two categories of sensitive material does Hybrid store this way?	63
Q9. What is a PersistentVolume, and why does Cassandra specifically need one?	63
Q10. What is a HorizontalPodAutoscaler, and which Hybrid component explicitly cannot use one the way the Message Processor does?	63
Q11. What is a Kubernetes Namespace, and what is Apigee Hybrid's default?	63
Q12. What role does kubectl play in day-to-day Apigee Hybrid operations?	63
Q13. What does the ApigeeOrganization custom resource govern?	63
Q14. What does the ApigeeEnvironment custom resource govern?	64
Q15. What does the ApigeeDatastore custom resource govern?	64
Q16. What does the ApigeeTelemetry custom resource govern?	64
Q17. What does the ApigeeRedis custom resource govern, and is it a replacement for Cassandra?	64
Q18. What does apigeectl actually do under the hood?	64
Q19. What changed with the introduction of Helm support for Apigee Hybrid installs?	64
Q20. What is the ApigeeDeployment custom resource, and how does the Apigee Operator use it?	65
Q21. What are Google's documented minimum production sizing numbers for Cassandra nodes and runtime pods?	65
Q22. Why must Cassandra be scaled in multiples of three rather than one node at a time?	65
Q23. Why does Google say there's no single optimal cluster size for Apigee Hybrid?	65
Q24. How would you validate a sizing estimate for an Apigee Hybrid cluster before going to production?	65
Q25. How would you diagnose a CrashLoopBackOff on apigee-runtime pods?	65
Q26. How would you diagnose a "No active runtime pods" warning in the Apigee UI?	66
Q27. How would you diagnose SSL handshake failures against a specific environment group's host-name?	66
Q28. How would you diagnose Synchronizer connectivity failures on a private GKE cluster specifically?	66
Q29. How would you diagnose Cassandra pods stuck in a Pending state on EKS specifically?	66
Q30. What's the general troubleshooting mental model an architect should apply across all of these Apigee Hybrid failure scenarios?	66
Section 12 — Study Notes, Memory Maps, and Cheat Sheets	67
12.1 Apigee X Cheat Sheet	67
12.2 Apigee Hybrid Cheat Sheet	68
12.3 Kubernetes-for-Apigee-Hybrid Cheat Sheet	70
12.4 Memory Maps	71
12.5 The One-Page Apigee X vs Hybrid Decision Cheat Sheet	72

Glossary	73
Bibliography	78
Primary sources — directly fetched and quoted, cloud.google.com / docs.cloud.google.com	78
Official Google Cloud Blog (Google-authored)	78
Further reading — official URLs confirmed to exist (titles verified); consult directly for current, version-specific detail	78
Kubernetes upstream documentation (generic concept definitions)	79
Secondary / non-official sources (used only for corroboration or historical context, always flagged inline where used)	80

How to Use This Handbook

This handbook is written for a senior software engineer who already knows how to build and operate distributed systems, and who now needs an **architect-level** mental model of Google Cloud Apigee — specifically, the structural differences between **Apigee X** (Google’s fully-managed SaaS offering) and **Apigee Hybrid** (Google’s split-responsibility, customer-operated-runtime offering).

The handbook is organized as twelve sections. Sections 1–3 build the conceptual foundation (what API management is, and the architecture of each product). Sections 4–7 go deep on the mechanics that differentiate Hybrid from X: Kubernetes, the control-plane/runtime-plane split, request flow, and configuration synchronization. Sections 8–9 move into production concerns: reference architectures, and failure/troubleshooting. Section 10 is a head-to-head comparison with architect recommendations. Sections 11–12 are exam- and interview-oriented: question banks and revision cheat sheets. A glossary and bibliography close the book.

A note on sourcing and intellectual honesty

Every claim of fact in this handbook is sourced. Because this is a professional reference, precision about *what is officially documented* versus *what is well-established architectural practice* matters — an architect who cites a fabricated SLA number or a nonexistent Kubernetes object type will eventually be embarrassed in front of a customer. This handbook therefore uses three tiers of attribution, and marks each substantive claim accordingly:

1. **Directly cited** — a specific claim, often a verbatim quotation, attributed to a named Google Cloud documentation page, the Apigee SLA, the Apigee product blog, or the Kubernetes upstream documentation, with a full URL in the Bibliography at the end of this handbook.
2. **Vendor-adjacent / community-corroborated** — a claim confirmed via a Google-run community forum, a Google engineer’s public post, or Apigee’s own legacy documentation domain (`docs.apigee.com`), used only where `cloud.google.com` documentation does not state the fact as plainly, and always labeled as such.
3. **Architectural synthesis** — a claim that is standard, well-established Apigee/Kubernetes architecture knowledge, necessary to connect verified facts into a coherent mental model, but not itself a verbatim quotation from a single official page. These are written in plain descriptive language (“in practice...”, “architecturally...”) rather than presented as a Google quotation.

Where official Google documentation does not publish a specific number (an exact Synchronizer polling interval, an exact minimum supported Kubernetes version, an internal autoscaling threshold), this handbook says so explicitly rather than inventing one. Apigee’s own published reference-architecture coverage on the Cloud Architecture Center is, honestly, thin compared to products like GKE or BigQuery — this handbook says that plainly in Section 8 rather than overstating Google’s published guidance.

All diagrams are either (a) redrawn as Mermaid diagrams from a described official source, with the source titled directly beneath the diagram, or (b) original synthesis diagrams built from verified component facts, labeled as such. No diagram in this handbook is a pixel-for-pixel reproduction of a copyrighted Google image; where an official diagram exists, this handbook links to it and redraws its structure for teaching purposes.

Section 1 — Introduction to API Management

1.1 What Is API Management?

An API management platform sits in front of backend services and takes over everything that backend teams would otherwise have to re-implement for every service they expose: authentication, authorization, rate limiting, quota enforcement, request/response transformation, analytics, developer onboarding, and versioning. Google’s own definition of Apigee — which doubles as a clean, general definition of what an API management platform is — states it plainly:

“Apigee is a platform for developing and managing APIs. By fronting services with a proxy layer, Apigee provides an abstraction or facade for your backend service APIs and provides security, rate limiting, quotas, analytics, and more.” (Source: [What is Apigee?](#))

Two ideas in that sentence matter more than the rest:

- **Facade / abstraction layer.** The platform decouples the *public contract* your API consumers depend on from the *private implementation* your backend teams keep changing. A backend can be re-platformed, re-versioned, or split into microservices behind the scenes without breaking a single external client, because the facade absorbs the change.
- **Cross-cutting concerns, centralized.** Security, rate limiting, quotas, and analytics are implemented once, in policy configuration, rather than N times inside N backend services in N different languages and frameworks.

An **API Proxy** is the concrete unit that implements this facade — “a proxy that acts as a facade for your existing API... This facade decouples your public interface from your backend API, shielding developers from backend changes, while enabling you to innovate at the edge without impacting your internal development teams.” (Source: [Understanding commonly used Apigee terms](#)) A **Policy** is the atomic, reusable unit of logic a proxy executes — “a processing step that executes as an atomic, reusable unit of logic within an API flow,” covering routing, transformation, access control, threat detection, caching, and more (same source).

1.2 Why Apigee Exists

Every organization that exposes more than a handful of APIs eventually re-discovers the same problem: without a management layer, security policy, rate limiting, and observability get re-implemented inconsistently inside every service, by every team, in every language. Apigee exists to make that layer a *platform capability* rather than a *per-team engineering task* — and to make it usable by API product managers and architects, not just backend engineers, through a UI-driven proxy/policy authoring model, a developer portal for onboarding external consumers, and a first-class concept of **API Products** — “a collection of API resources (URIs) combined with a service plan and presented to developers as a bundle” (same source) — that lets an organization monetize or govern access to its APIs independently of how the underlying services are built.

This is the same value proposition whether the platform is deployed as Apigee Edge, Apigee X, or Apigee Hybrid — what has changed across those three generations is *where the platform itself runs*, and *who operates it*. That evolution is the subject of the rest of this section, and it is the foundation for the entire rest of this handbook: Sections 2–10 are, at their core, an extended answer to the question “who runs which part of Apigee, and how does that shape the architecture?”

1.3 Apigee Product Evolution

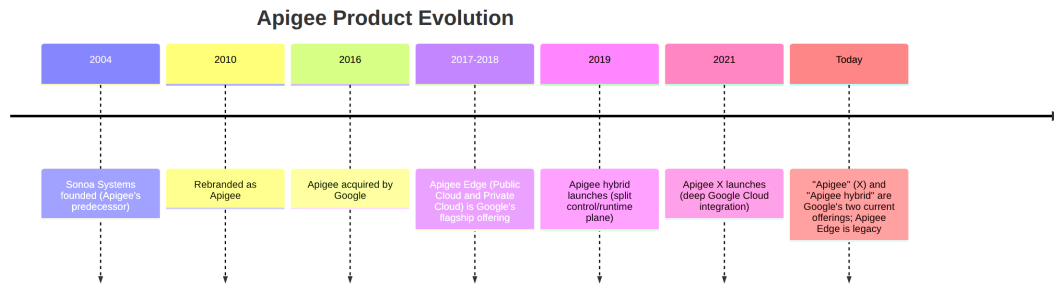


Diagram: original synthesis by the author based on verified component facts and publicly reported history in this handbook; the exact Apigee X GA date (commonly reported as February 4, 2021) is sourced from press coverage, not a directly-verified `cloud.google.com` announcement — treat that specific date as approximate. (Source for Edge/Hybrid/X definitions: [What is Apigee?](#); date context: [TechCrunch, Feb 2021](#), a non-official source used only for the approximate date.)

1.3.1 Apigee Edge

Apigee Edge was the pre-Google-Cloud-native generation of the product, offered in two flavors: **Edge Public Cloud** (a SaaS multi-tenant offering run by Apigee/Google) and **Edge for Private Cloud** (a fully self-hosted distribution that the customer installed on their own virtual machines). Edge for Private Cloud's architecture is worth understanding because nearly every named component in Apigee Hybrid today is a direct descendant of an Edge for Private Cloud component:

- **Message Processor (MP)** — the component that actually executes proxy flows and policies and talks to backend targets. (Source: [Apigee Edge → Apigee X comparison](#), legacy Apigee documentation domain, first-party Google content, not `cloud.google.com`.)
- **Router** — the component that terminated inbound traffic and load-balanced it to Message Processors.
- **Cassandra** — the distributed datastore used for KVMs, quotas, and caches.
- **ZooKeeper** — a coordination service used for cluster membership and configuration propagation among Edge nodes.
- **Management Server / Postgres** — the administrative API and its metadata store.

Every one of these was a process the customer had to install, patch, and scale on raw virtual machines — there was no container runtime, let alone Kubernetes, underneath Edge for Private Cloud. This operational burden is the direct motivation for both Apigee X and Apigee Hybrid.

1.3.2 Apigee Hybrid

Apigee Hybrid, per Google's own definition, "lets you manage APIs on-premises, on Google Cloud Platform (GCP), or a mix of both," with the explicit design goal that "all API traffic is processed within the boundaries of your network and under your control, while management services such as the UI and API analytics run in the cloud and are maintained by Google." (Source: [What is Apigee hybrid?](#)) Hybrid is the direct architectural evolution of Edge for Private Cloud: the same conceptual components (Message Processor, Cassandra, a Router-equivalent) still exist, but they are now **containerized and orchestrated by Kubernetes** instead of running as bare processes on VMs, and the *administrative half* of the product (UI, Management APIs, analytics processing) has been carved out and moved into a Google-managed control plane, so the customer only operates the traffic-processing half.

1.3.3 Apigee X

Apigee X is the fully-managed, Google Cloud-native evolution: both halves — the control plane *and* the runtime/data plane — are operated by Google. Google's own definition of the base "Apigee" runtime confirms this directly: "Apigee runtime: A set of containerized runtime services in a Kubernetes cluster that Google maintains," describing the SaaS product as "a cloud version hosted by Apigee in which Apigee maintains the environment, allowing

you to concentrate on building your services.” (Source: [What is Apigee?](#)) Apigee X deploys into the customer’s own Google Cloud organization (an Apigee organization is bound 1:1 to a Google Cloud project — see Section 2), connects to the customer’s VPC via Private Service Access/VPC Peering or Private Service Connect, and exposes a Google Cloud–idiomatic experience: Cloud IAM for access control, Cloud Load Balancing at the edge, Cloud KMS for encryption keys, and Cloud Monitoring/Logging for observability — none of which existed as first-class integrations in Edge.

Naming note: Google’s current documentation uses the plain product name “Apigee” for the SaaS offering rather than “Apigee X” (the page title is “What is Apigee?”, not “What is Apigee X?”). “Apigee X” remains the term everyone — including Google’s own blog posts and this handbook — uses informally to distinguish the SaaS product from Apigee Hybrid, and this handbook uses “Apigee X” throughout for clarity, consistent with the terminology in the objective you set for this handbook.

1.3.4 The single question that drives the rest of this handbook

Every one of the three generations above answers the same underlying architectural question differently: **who owns and operates the compute that actually processes API traffic?**

Generation	Who operates the control/management layer?	Who operates the traffic-processing runtime?
Apigee Edge (Private Cloud)	Customer (bare VMs)	Customer (bare VMs)
Apigee Edge (Public Cloud SaaS)	Apigee/Google	Apigee/Google
Apigee Hybrid	Google (cloud-hosted control plane)	Customer (customer’s own Kubernetes cluster)
Apigee X	Google	Google (Google-managed Kubernetes-based runtime, internals undisclosed)

Sections 2 and 3 build out each side of this table in full architectural detail; Section 5 returns to this exact table and turns it into a full responsibility matrix covering upgrades, scaling, certificates, and Kubernetes ownership.

Section 2 — Apigee X Architecture

2.1 Apigee X in One Picture

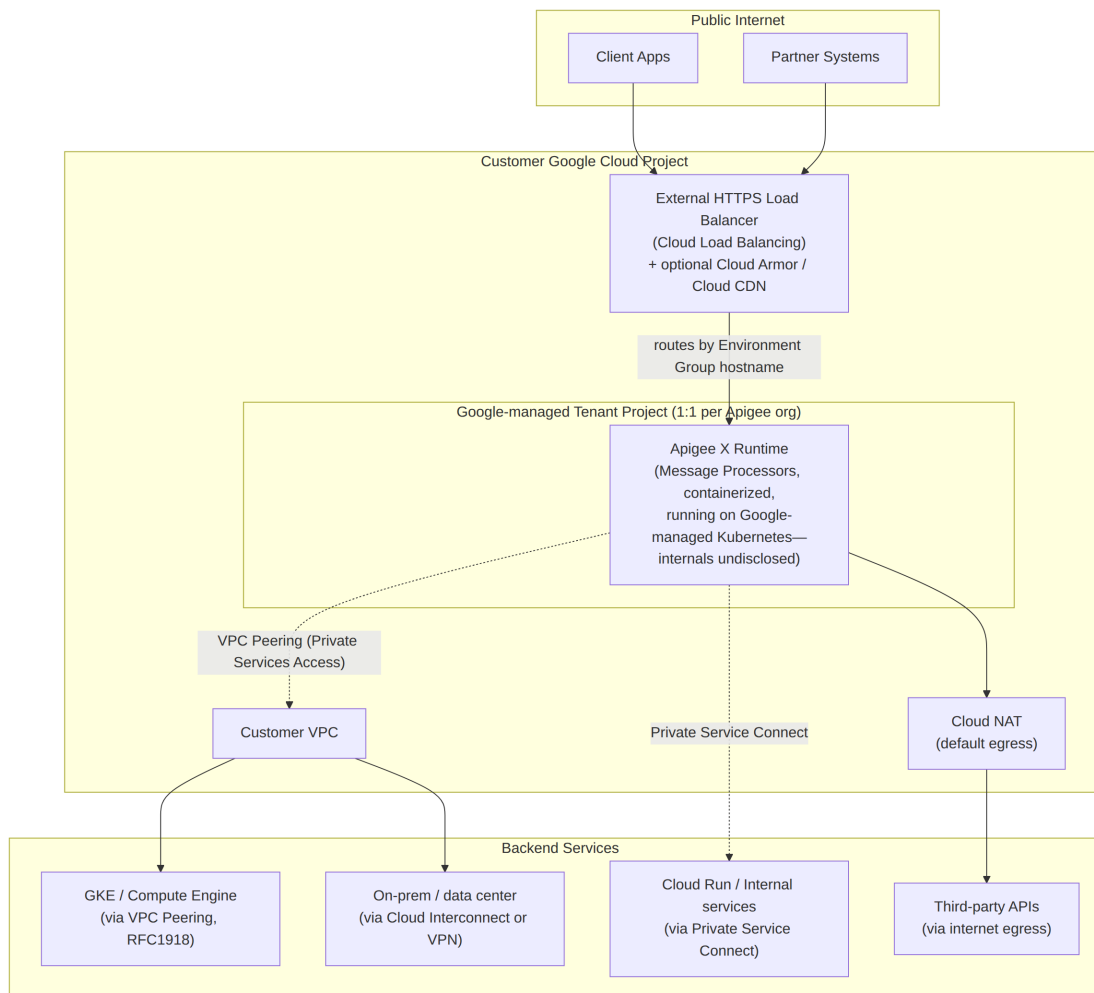


Diagram: original synthesis, redrawn from the northbound/southbound pattern described in [Best practices for securing your applications and APIs using Apigee](#) (Cloud Architecture Center) and [Apigee X design for multiple distinct VPC connections](#) (Google Cloud Blog). Not an official Google Cloud diagram image.

The single most important architectural fact about Apigee X is this: **the runtime is Google’s, running in a Google-owned, single-tenant “tenant project,” not the customer’s Kubernetes cluster.** Google states plainly that “Apigee runtime: A set of containerized runtime services in a Kubernetes cluster that Google maintains” (Source: [What is Apigee?](#)), and separately that “ApigeeX is a Private Service Access (PSA) based service that is deployed and managed in a tenant project. A tenant project is a project that is tied to the Apigee X Runtime, in a one-to-one relationship” running inside “a Google managed VPC” (Source: [Apigee X design for multiple distinct VPC connections](#)). Google does not disclose in customer-facing documentation whether that Kubernetes cluster is GKE specifically — this is a deliberate abstraction boundary, and it is the crux of “why Kubernetes is hidden in Apigee X” (Section 5 returns to this directly).

2.2 The Resource Hierarchy

2.2.1 Organization

An Apigee **Organization** is “a container for all the entities in an Apigee account, including API proxies, API products, API packages, apps, and developers,” and critically, “to install and use Apigee or Apigee hybrid, you must have an Apigee organization that is bound to a Google Cloud project” (Source: [Understanding commonly used Apigee terms](#)). This 1:1 binding to a GCP project is what lets Apigee X plug directly into Cloud IAM, Cloud Billing, Cloud KMS, and VPC networking as first-class citizens rather than through a separate identity/billing system, which was one of Edge’s structural limitations.

2.2.2 Instance

An **Instance** is the unit of *regional runtime capacity* inside an organization. Apigee X enforces a hard architectural constraint here that every architect needs memorized: **exactly one instance per Google Cloud region, per organization**, with a maximum of 10 instances/regions per organization (11 for Hybrid) (Source: [Apigee Limits reference](#)). This single fact is *why* Apigee X’s multi-region story is “add another region” rather than “add another instance in the same region” — there is no horizontal instance-level sharding within a region; all capacity growth within a region happens inside that one instance’s Google-managed runtime, invisibly to the customer.

2.2.3 Environment

An **Environment** is “a runtime execution context for APIs. An API must be deployed to an environment before it can be accessed at runtime” (Source: [Understanding commonly used Apigee terms](#)). Environments are commonly named test, prod, dev, and so on, and each environment must be attached to an instance in a specific region before proxies deployed to it can actually serve traffic there. An organization can have up to 85 environments (contract-dependent) (Source: [Apigee Limits reference](#)).

2.2.4 Environment Group

An **Environment Group** is the hostname-to-environment routing layer that sits above environments: “the basic mechanism for defining the way requests are routed to individual environments. You define hostnames on your environment groups (not on individual environments), and Apigee routes requests to the environments within a group using those hostname definitions” (same source). This indirection is deliberate — it lets an organization change which environment answers for a given hostname (for example, during a blue/green cutover) without touching DNS or the load balancer.

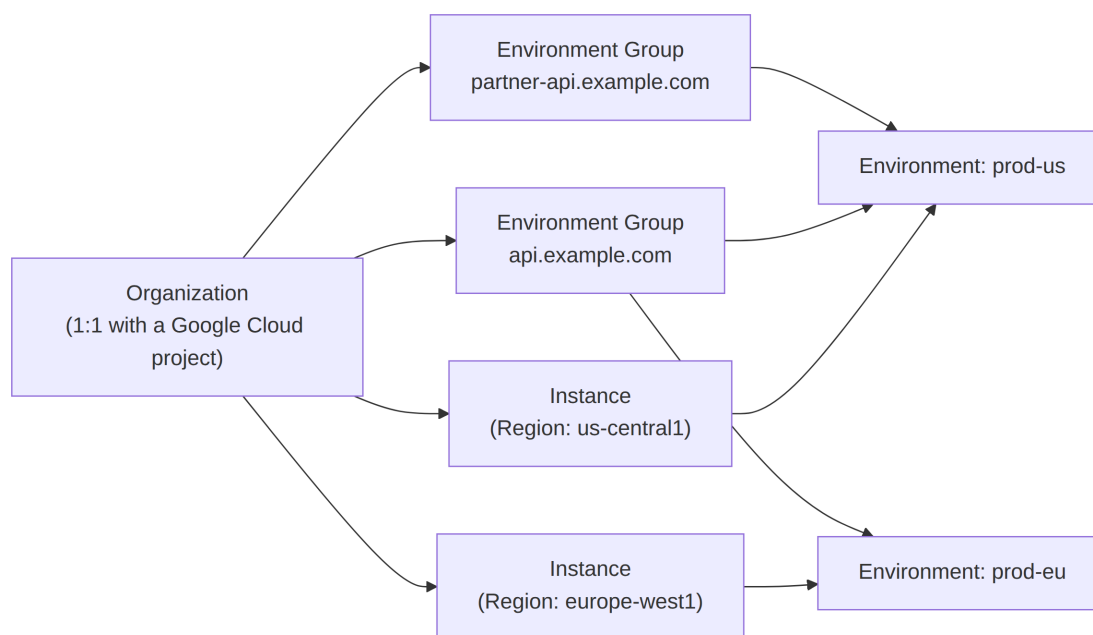


Diagram: original synthesis based on the verified definitions of Organization, Instance, Environment, and Environment Group above. An environment can belong to multiple environment groups, and a group’s hostnames can route

to environments in more than one region — this is the structural basis for Apigee X multi-region deployments covered in Section 8.

2.2.5 API Proxy, Policy, API Product, Developer App

- **API Proxy** — “a proxy that acts as a facade for your existing API... This facade decouples your public interface from your backend API” (Source: [Understanding commonly used Apigee terms](#)). A proxy is versioned as a sequence of **revisions** — “revisions are sequentially numbered, enabling you to revert a change by deploying a previous revision,” and “only one revision of an API proxy can be deployed in an environment at a given time if revisions share the same API proxy base path” (Source: [Understanding deployment](#), legacy Apigee domain, concept unchanged in current Apigee X).
- **Policy** — “a processing step that executes as an atomic, reusable unit of logic within an API flow,” covering routing, message transformation, access control, remote calls, data masking, threat inspection, and caching (Source: [Understanding commonly used Apigee terms](#)). The Cloud Architecture Center groups the most commonly used policies into three families: traffic management (SpikeArrest, Quota, ResponseCache), message-level protection (OASValidation, JSONThreatProtection, XMLThreatProtection), and security (BasicAuthentication, VerifyAPIKey, OAuthV2, JWT, CORS) (Source: [Best practices for securing your applications and APIs using Apigee](#)).
- **API Product** — “a collection of API resources (URIs) combined with a service plan and presented to developers as a bundle” (Source: [Understanding commonly used Apigee terms](#)). API Products are the unit an external developer actually subscribes to — they bundle one or more proxy resources with a quota/rate-limit plan.
- **Developer App** — a credential-bearing registration, created by a **Developer**, that is granted access to one or more API Products; the app receives a consumer key/secret pair used at runtime (typically via the `VerifyAPIKey` or `OAuthV2` policies) to authenticate and authorize calls. This model is standard, well-established Apigee behavior consistent across Edge, X, and Hybrid.

2.3 The Apigee X Runtime, in Detail

Inside the Google-managed tenant project, the workhorse component is the **Message Processor**. Google’s own error catalog is direct about its role: “The Message Processor is the core component of Apigee that processes the policies and interacts with the backend servers” (Source: [Message Processor error catalog](#)), and Google’s troubleshooting documentation confirms “the message processor is part of the apigee-runtime component,” running as “pods that can autoscale or restart” (Source: [Troubleshooting playbook: message processor](#)). This is the clearest official acknowledgment that “Message Processor” remains current, official terminology for Apigee X — it is simply implemented as containerized pods rather than as the standalone VM process it was under Edge.

Two terms that appear frequently in Hybrid documentation and in older Edge/community material are worth explicitly *not* attaching to Apigee X: **MART** (a Hybrid-only synchronization component covered in Section 3) does not appear in current Apigee X customer-facing documentation, and “UAP”/“**Unified Analytics Platform**” does not appear as an official term in any source reviewed for this handbook. Avoid using either term when describing Apigee X specifically.

What Google does **not** disclose about the Apigee X runtime, by design, includes: whether the underlying Kubernetes cluster is GKE; the exact autoscaling trigger metrics and thresholds; and the exact internal request-processing topology beyond “Message Processor.” This is intentional platform abstraction, not a documentation gap — see Section 5.4 for a full treatment of *why*.

2.4 Networking Architecture

2.4.1 VPC Peering / Private Services Access (PSA)

Apigee X’s default connectivity model is **Private Services Access**, implemented with **VPC Network Peering** between the customer’s VPC and the Google-managed tenant project: “VPC peering is the easiest option. The restriction is that connectivity wise, only one peering hop is allowed, which is already consumed by a connection to the Apigee X Runtime. The Apigee X Runtime resides in a Google managed VPC” (Source: [Apigee X design for multiple distinct VPC connections](#)). Because VPC Peering is non-transitive (“only two VPCs can be peered in sequence” and Apigee already consumes the customer’s one available peering hop to the tenant project), reaching

multiple backend VPCs from Apigee X requires additional patterns layered on top of the base peering connection — Cloud VPN (which is transitive across hops), a Shared VPC hub design, or Private Service Connect.

Apigee X also natively provisions its own load balancer for **northbound** (client-facing) traffic and, by default, egresses non-private traffic through **Cloud NAT** in the tenant project: “The Apigee X deployment natively manages a load balancer in the customer’s VPC for ingress traffic. By default, egress to non-private routes from Apigee X is through Cloud NAT in the tenant project” (same source).

2.4.2 Private Service Connect (PSC)

Private Service Connect is the newer, more scalable alternative to raw VPC Peering for **southbound** (Apigee-to-backend) connectivity: “Private Service Connect (PSC) allows private consumption of services across VPC networks... PSC is the missing piece that will natively support Apigee X backends in other VPCs,” with the key advantage over peering that “the underlying infrastructure is not exposed: connecting and managing services is much simpler, more secure and private” (Source: [Connecting to Cloud Run from Apigee X using PSC](#)). A representative PSC southbound pattern connects Apigee X through a PSC Endpoint Attachment to a PSC Service Attachment, fronting an internal Application Load Balancer, which in turn fronts the backend (for example, a serverless NEG in front of Cloud Run) — with the constraint that “these two attachments (Endpoint and Service Attachments) along with the L7 ILB must be part of the same GCP region” (same source). PSC is also the current mechanism Google documents for **northbound** networking as an alternative to a Managed Instance Group-based load balancer target (Source: [Northbound networking with Private Service Connect](#), title confirmed, full body not independently re-verified in this research pass — consult this page directly for current step-by-step configuration).

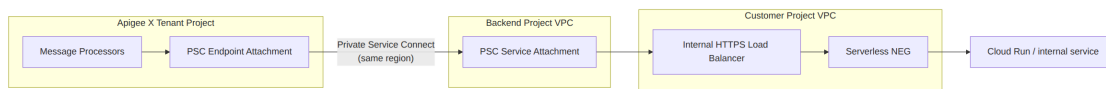


Diagram: redrawn from the PSC southbound pattern described in [Connecting to Cloud Run from Apigee X using PSC](#). Not an official Google Cloud diagram image.

2.4.3 Northbound vs. Southbound — the Official Security Pattern

The Cloud Architecture Center’s one dedicated Apigee reference article frames the whole northbound/southbound picture concretely: external clients hit an **External Application Load Balancer** (which terminates TLS), which is layered with **Cloud Armor** (WAF — “evaluates every call made by the client that hits the external Application Load Balancer”) and optionally **reCAPTCHA Enterprise** in a combined WAAP (Web App and API Protection) pattern, in front of Apigee, which “is provisioned in a Google Cloud project and the runtime is provisioned and peered in a tenant project using VPC Network Peering.” Google explicitly recommends layering Cloud Armor rather than relying solely on Apigee’s own policies for this class of protection: “We recommend that you use Cloud Armor for IP address-based and geo-based access control.” On the southbound side, the same article documents three distinct backend connectivity patterns: GKE backends reached “via RFC 1918 internal IPs over VPC Network Peering,” on-premises/data-center services reached over **Cloud Interconnect**, and third-party APIs reached via NAT egress. (Source: [Best practices for securing your applications and APIs using Apigee](#).) Cloud CDN is layered at the load balancer to absorb cacheable read traffic ahead of Apigee entirely.

2.4.4 Scaling Past One Peering Hop: Hub-and-Spoke and Shared VPC

Because Apigee X consumes the customer’s one VPC-peering hop to reach the tenant project, connecting Apigee to *many* backend VPCs at scale requires an explicit topology decision. Google’s product engineering team documents this problem and two concrete solutions in detail:

1. **Hub-and-spoke over Cloud VPN.** Since “Cloud VPN supports transitivity across multiple VPC network hops,” a hub project can terminate VPN tunnels to many spoke VPCs, with Apigee X peered to the hub.
2. **Shared VPC hub design.** The Apigee X tenant project’s host VPC can itself be a Shared VPC, since “Shared VPC subnets do not violate the VPC peering transitivity limit” — this nests an “inner hub-and-spoke” (the Shared VPC containing the Apigee X host project) inside an “outer hub-and-spoke” (VPN-connected backend spokes), a pattern Google’s own engineers describe as “analogous to cascading hub-and-spoke architecture.” A further evolution — a “wheel” pattern where spoke VPCs peer directly to each other — removes the hub dependency entirely for high-fanout, multi-tenant productization scenarios.

(Source for this subsection: [Apigee X design for multiple distinct VPC connections](#).) Google also documents a dedicated [Shared VPC networks](#) configuration page and a [Peering ranges](#) reference for sizing the CIDR block Apigee X consumes from the customer’s address space — both confirmed to exist at these URLs; consult them directly for current step-by-step values.

2.5 Component Responsibility and Request-Flow Summary

Component	Responsibility	Who operates it
External Load Balancer (+ Cloud Armor/CDN)	TLS termination, WAF, edge caching	Customer (provisioned in customer project, per Architecture Center guidance)
Environment Group	Hostname → environment routing	Customer configures; Google executes the routing
Message Processor / apigee-runtime	Policy execution, backend communication	Google (fully managed, Kubernetes internals undisclosed)
Environments / Instances	Logical + regional partitioning of runtime capacity	Customer configures topology; Google provisions and scales the underlying capacity
Cloud NAT / VPC Peering / PSC	Southbound/northbound connectivity	Customer configures the network attachment; Google operates the tenant-project side
Analytics pipeline	Collect and expose per-transaction analytics	Google (fully managed)
API Proxies, Policies, Products, Apps	The API contract and business logic itself	Customer (this is what you build)

This table is the seed of the full responsibility matrix in Section 5; the key takeaway for Apigee X specifically is that **everything to the right of “your VPC edge” is Google’s operational responsibility** — which is precisely what makes Kubernetes invisible in this product, and precisely what Section 3’s Apigee Hybrid architecture inverts for the runtime half of the stack.

Section 3 — Apigee Hybrid Architecture

3.1 The Split: Management Plane and Runtime Plane

Apigee Hybrid’s defining architectural move is to split what Apigee X keeps unified. Google’s official “shared responsibility model” documentation states the split directly, in terms of *who does what* rather than just *what runs where* — which is the most useful framing for an architect:

Google’s responsibilities (the management/control plane): > “Deliver Apigee hybrid packages including runtime, mart, Cassandra, and Kubernetes controllers and management tools.” “Keeping Management APIs and Analytics compatible with customer deployments.” “Continually scan components with the Container Analysis API and patch known vulnerabilities.” “Providing upgrade notifications with documented steps.” “Providing the connections to the management plane and Google Cloud Observability.” (Source: [Apigee hybrid shared responsibility model](#))

Customer’s responsibilities (the runtime/data plane): > “Maintaining connectivity between the Apigee hybrid data plane and the Google managed Apigee control plane.” “Provisioning of the underlying compute, storage and network infrastructure as per Apigee’s recommendations.” “Keep the Kubernetes platform and underlying infrastructure up to date with latest software updates and patches.” Load balancer provisioning and TLS certificate lifecycle for APIs. “Multi-region deployment and disaster recovery.” (same source)

Notice something important embedded in the first Google-responsibility quote: Google explicitly says it delivers “runtime, mart, Cassandra, and Kubernetes controllers” as *packages* — meaning Google builds and ships the container images and the Kubernetes operator/CRDs that define the runtime plane’s shape, but the customer is the one who runs `kubectl apply` (or `apigeectl`, or Helm) and operates the actual cluster. This is the single fact that answers “where does Kubernetes fit into Apigee Hybrid”: **Kubernetes is the customer’s own compute substrate, into which Google ships a pre-built application (the Apigee runtime plane) as container images plus an Operator that manages custom resources.**

Google’s own definition of the product frames the same split in plain product language: “Apigee hybrid lets you manage APIs on-premises, on Google Cloud Platform (GCP), or a mix of both... All API traffic is processed within the boundaries of your network and under your control, while management services such as the UI and API analytics run in the cloud and are maintained by Google.” The management plane is “a set of services hosted in the cloud and maintained by Google” (UI, Management APIs, analytics processing); the runtime plane is “a set of containerized runtime services that you set up and maintain in your own Kubernetes cluster,” explicitly designed so the runtime plane can “function independently from the management plane” for some period of time if connectivity is lost. (Source: [What is Apigee hybrid?](#))

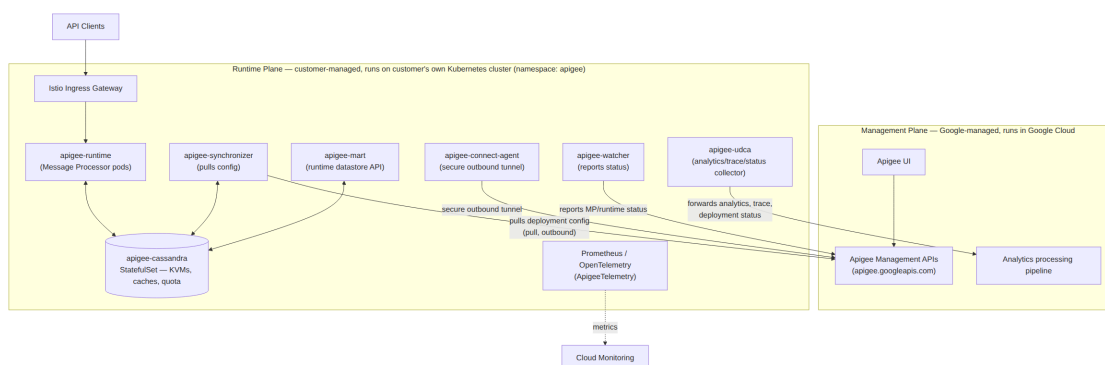


Diagram: original synthesis, redrawn from the management-plane/runtime-plane split described in [What is Apigee hybrid?](#) and [Apigee hybrid shared responsibility model](#), with component-level detail from the troubleshooting playbooks cited throughout this section. Not an official Google Cloud diagram image.

3.2 Runtime Plane Components, in Detail

Google’s own troubleshooting documentation is unusually explicit about how the runtime plane’s Kubernetes Operator materializes each component. The Operator manages a set of custom resources — **ApigeeOrganization**, **ApigeeEnvironment**, **ApigeeDatastore**, **ApigeeTelemetry**, and **ApigeeRedis** — and the following table is drawn directly from Google’s documented mapping (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)):

Component	Governing custom resource	Kubernetes deployment type
apigee-connect-agent	ApigeeOrganization	ApigeeDeployment → Deployment + HorizontalPodAutoscaler + PodDisruptionBudget + ReplicaSet
apigee-mart	ApigeeOrganization	ApigeeDeployment → Deployment
apigee-watcher	ApigeeOrganization	ApigeeDeployment → Deployment
apigee-runtime (Message Processor)	ApigeeEnvironment	ApigeeDeployment → Deployment
apigee-synchronizer	ApigeeEnvironment	ApigeeDeployment → Deployment
apigee-udca	ApigeeEnvironment	ApigeeDeployment → Deployment
apigee-cassandra	ApigeeDatastore	StatefulSet

The pattern is worth stating plainly because it is exactly the kind of thing an interviewer will ask: **every stateless runtime-plane component is a Deployment (wrapped in Apigee’s own ApigeeDeployment custom resource); Cassandra is the one and only StatefulSet**, because it is the only component that needs stable per-pod network identity and durable per-pod storage.

3.2.1 MART (Management API for Runtime [Data])

- **Purpose.** MART is the runtime-plane service that lets the management plane reach into runtime-plane state that only exists in the cluster — most notably Cassandra-backed data such as KVM reads/writes and certain runtime-side status queries. Apigee’s own release notes describe its role precisely in the context of connectivity: “Apigee Connect Alpha allows the Apigee hybrid MART service to connect to the management plane without requiring you to expose the MART endpoint” (Source: [Apigee hybrid 1.1.0 release notes](#), legacy Apigee domain, concept current).
- **Communication flow.** Historically, the management plane called *into* MART, which meant the customer had to expose a MART ingress endpoint to Google. **Apigee Connect** inverted this: the `apigee-connect-agent` opens a secure, outbound-initiated tunnel from the runtime plane to Google, so MART can be reached without any customer-exposed inbound endpoint for management traffic.
- **Data stored.** MART itself is stateless; it authenticates and validates requests before reading/writing to Cassandra.
- **Failure impact.** In the officially documented Cassandra-outage scenario, `apigee-mart` pods enter `CrashLoopBackOff` together with `apigee-runtime` and `apigee-synchronizer`, because all three depend on a healthy Cassandra connection (Source: [No active runtime pods warning](#)). No independently documented “MART alone is down” scenario was found; MART’s availability is effectively coupled to Cassandra’s.
- **Kubernetes deployment type.** `ApigeeDeployment` → Deployment, organization-scoped.

3.2.2 Synchronizer

- **Purpose.** The Synchronizer keeps the runtime plane’s local view of deployed configuration (“contracts”: proxy bundles, environment configuration, target servers, KVM metadata) up to date with what the control plane’s Management API says should be deployed.

- **Communication flow.** This is a **pull model**: the Synchronizer, running inside the runtime plane, initiates outbound calls to the Apigee Management API in the control plane and polls for changes — Google’s design goal is explicitly that the customer never has to open an inbound firewall port toward Google for this traffic. Google’s own troubleshooting documentation confirms the failure signature when this path breaks: “if no network route is available to the internet, the Apigee runtime plane components will not be able to communicate with Apigee control plane APIs,” naming `apigee-synchronizer` as the component that “downloads API product configs from the control plane” (Source: [Troubleshooting playbook: no network connectivity](#)). Google does not publish an exact polling interval in the sources reviewed for this handbook — describe the mechanism, not an invented number, when discussing this in an interview or design review.
- **Data stored.** The Synchronizer writes pulled configuration into the runtime plane’s Cassandra datastore, from which Message Processors read it at request time.
- **Failure impact.** New proxy deployments cannot complete (the Message Processor logs “no active contract is available for the message processor to serve the traffic” and fails its readiness probe, per [Troubleshooting playbook: message processor](#)); API Products/Developers/Apps stop populating in the Apigee UI; Management API reads/writes for those entities fail. Critically, **already-deployed traffic keeps flowing** — the Synchronizer is on the control path for *new* deployments, not on the data path for existing ones.
- **Kubernetes deployment type.** `ApigeeDeployment` → Deployment, environment-scoped.

3.2.3 Cassandra

- **Purpose.** Cassandra is the runtime plane’s persistent datastore for **Key Value Maps, response caches, and Quota counters** — confirmed directly: “runtime pods must connect to Cassandra to fetch resources like Key Value Maps and caches” (Source: [No active runtime pods warning](#)).
- **Communication flow.** `apigee-runtime`, `apigee-mart`, and `apigee-synchronizer` all read from and write to Cassandra directly over the cluster network.
- **Data stored.** KVMs, response caches, Quota counters. It is **not deprecated** — Google’s shared-responsibility page explicitly lists Cassandra as a package it continues to deliver alongside the runtime and Kubernetes controllers, and dedicated Cassandra backup/scaling documentation continues to be published for the current release line. A separate `ApigeeRedis` custom resource has been added in recent hybrid versions, but it supplements Cassandra narrowly — per a Google engineer’s public forum answer, “Quotas are stored in Cassandra with rate synchronized... on the Quota policies configuration” (long-term/contractual limits) while “Spike Arrest counts in memory on the Message Processors” and “will use Redis to coordinate counts when `UseEffectiveCount` is set” (short-term protection, coordinated across MP pods only for that one setting) — treat this specific division of labor as community-corroborated rather than a `cloud.google.com` quotation.
- **Failure impact.** If Cassandra is unreachable, `apigee-runtime`, `apigee-mart`, and `apigee-synchronizer` pods all enter `CrashLoopBackOff` with the log signature “Cannot build a cluster without contact points”; deployments fail with a “No active runtime pods” UI warning, a “Datastore Error” on the Products/Apps pages, and an “Internal Server Error” on the Developers page. **A single-replica Cassandra is an explicit single point of failure** — Google’s own guidance is to configure `cassandra.replicaCount: 3` for production. Recovery is automatic once Cassandra returns; no manual pod intervention is required. (Source: [No active runtime pods warning](#); [Troubleshooting playbook: Cassandra](#).)
- **Kubernetes deployment type.** `StatefulSet`, governed by the `ApigeeDatastore` custom resource — the one genuinely stateful component in the runtime plane.

3.2.4 Message Processor (`apigee-runtime`)

- **Purpose.** Executes proxy flows and policies against inbound requests and communicates with backend targets — functionally the API gateway itself, and architecturally identical in role to the Apigee X Message Processor described in Section 2.3.
- **Communication flow.** Reads deployed configuration from Cassandra (populated there by the Synchronizer); receives traffic from the Istio Ingress Gateway.
- **Data stored.** Stateless itself; reads runtime data from Cassandra per-request.
- **Failure impact.** Coupled to Cassandra and Synchronizer availability as described above; when healthy but no active deployment “contract” exists, the pod fails its readiness probe and is removed from traffic serving rather than serving broken responses.
- **Kubernetes deployment type.** `ApigeeDeployment` → Deployment, environment-scoped. Apigee hybrid v1.16’s release notes document a configurable `runtime.release.strategy` (rolling / scale-down-first / none) governing how upgrades to this Deployment roll out (Source: [Apigee hybrid release notes](#)).

3.2.5 Ingress Gateway

- **Purpose.** Terminates inbound API traffic and routes it to the correct Message Processor pod.
- **Communication flow.** The ingress layer is **Istio-based** (Cloud Service Mesh); community-documented object names include `istio-ingressgateway` for general API traffic and `apigee-mart-istio-ingressgateway` for routing to MART specifically, the latter of which becomes unnecessary once Apigee Connect is in use. Envoy is Istio’s data-plane proxy, and Google’s own security-patching documentation lists Envoy by name among the open-source projects it tracks for vulnerability disclosure — consistent with, though not an explicit official statement of, Envoy powering this ingress data plane.
- **Data stored.** None — routing configuration only.
- **Failure impact.** A documented, officially-verified failure mode: missing or misconfigured TLS secrets for an environment group cause SSL handshake failures and connection resets for **all traffic through that ingress endpoint** — unlike a Synchronizer outage, an ingress/TLS problem is traffic-affecting immediately (Source: [Troubleshooting playbook: API calls failing with TLS errors](#)).
- **Kubernetes deployment type.** Deployment + Service (typically LoadBalancer type) per the Istio ingress-gateway model; TLS certificates are held in Kubernetes Secrets, which `apigee-watcher` (below) helps reconcile.

3.2.6 Telemetry (Prometheus, OpenTelemetry, logging)

- **Purpose.** Collects operational metrics and logs from every runtime-plane component.
- **Communication flow.** Apigee hybrid documentation states directly that “Apigee hybrid uses the industry-standard Prometheus add-on for metrics collection,” governed by the `ApigeeTelemetry` custom resource; an OpenTelemetry Collector integration with HTTP proxy forwarding is also a documented, currently supported feature. Metrics are shipped to Cloud Monitoring; log shipping is commonly implemented via a Fluentd/Fluent Bit-based agent (named in Google Cloud Community material — not confirmed via a direct `cloud.google.com` sentence in this research pass, so treat “Fluentd” as community-corroborated rather than an official term).
- **Data stored.** Time-series metrics and structured logs, typically forwarded off-cluster rather than retained long-term in-cluster.
- **Failure impact.** Loss of telemetry components degrades observability but is not documented as traffic-affecting.
- **Kubernetes deployment type.** `ApigeeTelemetry` custom resource, materialized as Deployments/DaemonSets depending on the specific telemetry agent.

3.2.7 UDCA (Universal Data Collection Agent)

- **Purpose.** Extracts analytics, distributed-trace, and deployment-status data from the runtime plane and forwards it to the management plane’s analytics processing pipeline, making it visible in the same Apigee Analytics UI/API used by Apigee X.
- **Communication flow.** Outbound, from the runtime plane to the control plane, consistent with the Synchronizer’s and Watcher’s outbound-only design philosophy.
- **Data stored.** Transient buffering only; system of record is the control-plane analytics pipeline.
- **Failure impact.** If the runtime plane loses connectivity to the control plane, analytics/debug/deployment-status data queues locally rather than being lost outright, and is expected to catch up once connectivity returns — consistent with the broader resilience narrative in Section 9.
- **Kubernetes deployment type.** `ApigeeDeployment` → Deployment, environment-scoped.

3.2.8 Watcher (`apigee-watcher`)

- **Purpose.** Reports Message Processor and runtime status up to the management plane, and reconciles ingress-related TLS secret state inside the cluster.
- **Communication flow.** Outbound status reporting to the Management API (“mp status to mgmt”); a documented failure mode shows `apigee-watcher` logging `NOT_FOUND: failed to get secret...` in namespace `apigee` when a required TLS secret for an environment group is missing (Source: [Troubleshooting playbook: API calls failing with TLS errors](#)) — direct evidence Watcher, not a separate cert-manager-style controller, is responsible for this reconciliation in Hybrid.
- **Data stored.** None persisted; operates on live cluster/secret state.

- **Failure impact.** Stale or missing status reporting to the management plane; potential TLS secret drift for ingress.
- **Kubernetes deployment type.** `ApigeeDeployment` → Deployment, organization-scoped.

3.2.9 Apigee Connect Agent

- **Purpose.** Establishes the secure, outbound-initiated tunnel that lets the management plane reach MART/runtime services without the customer exposing an inbound ingress endpoint for management traffic.
- **Communication flow.** Outbound-only from the runtime plane to Google.
- **Data stored.** None.
- **Failure impact.** Without it, MART must be reached via a directly-exposed ingress endpoint instead.
- **Kubernetes deployment type.** `ApigeeDeployment` → Deployment + HorizontalPodAutoscaler + PodDisruptionBudget + ReplicaSet, organization-scoped — the only component the official troubleshooting playbook shows explicitly paired with an HPA in its default configuration.

3.3 Where Kubernetes Actually Fits

Bringing Sections 3.1 and 3.2 together: Kubernetes in Apigee Hybrid is not a hidden implementation detail — it is the **explicit, customer-operated substrate** that hosts every runtime-plane component in the table above, organized (by default) inside a single namespace called `apigee`. Google ships the container images and an Operator/CRD layer (`ApigeeOrganization`, `ApigeeEnvironment`, `ApigeeDatastore`, `ApigeeTelemetry`, `ApigeeRedis`) that translates high-level Apigee configuration into standard Kubernetes objects (Deployments, a `StatefulSet` for Cassandra, Services, Secrets, ConfigMaps, HPAs). The customer is responsible for the cluster itself — its nodes, its patching, its networking, its storage classes — exactly as stated in the shared-responsibility model in Section 3.1. Section 4 goes one level deeper into exactly how each Kubernetes primitive is used.

3.4 Supported Kubernetes Platforms

Apigee Hybrid’s most heavily documented target is **GKE**. **EKS** (Amazon) is confirmed supported, with a documented EKS-specific dependency: Cassandra pods can get stuck in a Pending state due to a “Missing Amazon EBS CSI driver (for EKS installations)” (Source: [Troubleshooting playbook: Cassandra](#)). **AKS** (Azure) is confirmed supported per Google-staff community guidance. **Anthos** is supported in some configurations but explicitly *not* on “Anthos attached clusters with arbitrary k8s distributions” nor “Anthos on-prem, on bare metal,” per the same community-sourced guidance — treat this as corroboration rather than a primary citation, and always check the current [Apigee hybrid supported platforms](#) page for the exact, version-specific compatibility matrix, since it changes with every hybrid release and this handbook does not reproduce specific minimum version numbers that could not be independently verified.

Section 4 — Kubernetes Deep Dive for Apigee Hybrid

This section explains each core Kubernetes primitive using the generic, upstream Kubernetes definition, and then immediately ties it to the specific Apigee Hybrid component that uses it, per the mapping established in Section 3.2. Generic definitions are quoted directly from the Kubernetes project’s own documentation.

4.1 Cluster and Node

A Kubernetes **cluster** is the set of machines (**nodes**) that run containerized workloads, coordinated by a control plane. In Apigee Hybrid, the cluster is entirely the customer’s responsibility to provision and patch — “Keep the Kubernetes platform and underlying infrastructure up to date with latest software updates and patches” is listed as a customer duty in the shared-responsibility model (Source: [Apigee hybrid shared responsibility model](#)). Google’s own Apigee product-team guidance gives concrete node-level sizing floors for production: Cassandra nodes need “a minimum of 8 vCPUs and 15Gi of RAM for production workloads,” while runtime (Message Processor) pods are recommended at “2 vCPUs and 2 Gi of RAM” each (Source: [Optimize managing APIs at scale across hybrid environments](#)). Google is explicit that there is no universal sizing formula — “there is no one optimal infrastructure size for all clusters running Apigee hybrid” — and recommends working with a Google Customer Engineer for capacity estimation, then validating with load tests rather than trusting static numbers (same source). The same source also states the baseline production topology bluntly: “Your production platform should have at least two Kubernetes clusters, each in a different region,” ideally “the same size” so either can absorb the other’s full load during a regional failure.

4.2 Namespace

“In Kubernetes, namespaces provide a mechanism for isolating groups of resources within a single cluster” (Source: [Kubernetes documentation — Namespaces](#)). Apigee Hybrid’s default install target is a single namespace named **apigee** — confirmed directly in Apigee’s own install documentation: “The default namespace is ‘apigee,’ but users can specify alternatives using the `--namespace` flag or by manually updating resource definitions” (Source: [Apigee hybrid installation and administration guide](#)). Every component in Section 3.2’s table runs inside this one namespace by default.

4.3 Pod

A Pod is the smallest deployable unit — one or more containers sharing network and storage. Every Apigee Hybrid runtime-plane component (Message Processor, Synchronizer, MART, UDCA, Watcher, Connect Agent, Cassandra) ultimately runs as one or more pods; what differs is the *controller* that manages those pods, covered next.

4.4 Deployment

“A Deployment manages a set of Pods to run an application workload, usually one that doesn’t maintain state” (Source: [Kubernetes documentation — Deployments](#)). This is the controller behind every stateless Apigee Hybrid component: `apigee-runtime`, `apigee-synchronizer`, `apigee-mart`, `apigee-udca`, `apigee-watcher`, and `apigee-connect-agent` are all instantiated through Apigee’s own `ApigeeDeployment` custom resource, which the Apigee Operator materializes into a standard Kubernetes Deployment (plus, for `apigee-connect-agent`, a `HorizontalPodAutoscaler`, `PodDisruptionBudget`, and `ReplicaSet`) (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)). Apigee hybrid v1.16 adds a configurable `runtime.release.strategy` (rolling / scale-down-first / none) that governs how the `apigee-runtime` Deployment rolls pods during an upgrade (Source: [Apigee hybrid release notes](#)).

4.5 StatefulSet

“A StatefulSet runs a group of Pods, and maintains a sticky identity for each of those Pods. This is useful for managing applications that need persistent storage or a stable, unique network identity” (Source: [Kubernetes documentation — StatefulSets](#)). **Cassandra is the only Apigee Hybrid component deployed as a StatefulSet**, governed by the ApigeeDatastore custom resource — precisely because a Cassandra ring depends on each node keeping a stable identity and durable storage across restarts. This is a high-value exam fact: if asked “which Apigee Hybrid component is a StatefulSet and why,” the answer is Cassandra, because it is the only genuinely stateful runtime-plane component.

4.6 Service

“Expose an application running in your cluster behind a single outward-facing endpoint, even when the workload is split across multiple backends” (Source: [Kubernetes documentation — Service](#)). Every Deployment and the Cassandra StatefulSet sit behind a Service so that, for example, the Message Processor Deployment can be addressed by Synchronizer and Cassandra by a stable in-cluster name regardless of which specific pods are currently healthy.

4.7 Ingress and the Istio Ingress Gateway

“Make your HTTP (or HTTPS) network service available using a protocol-aware configuration mechanism... lets you map traffic to different backends based on rules you define via the Kubernetes API” (Source: [Kubernetes documentation — Ingress](#)). Apigee Hybrid’s ingress layer is Istio-based rather than a bare Kubernetes Ingress object — community-documented object names include `istio-ingressgateway` (general API traffic, exposed as a Service of type `LoadBalancer`) and `apigee-mart-istio-ingressgateway` (routes to MART, unnecessary once Apigee Connect is enabled). TLS certificates for this gateway are held as Kubernetes Secrets (below), and Apigee’s install guide confirms both self-signed and CA-issued certificates are supported: “Users can either generate self-signed certificates or use certificates from recognized authorities for istio ingress gateway communication” (Source: [Apigee hybrid installation and administration guide](#)).

4.8 ConfigMap

“A ConfigMap is an API object used to store non-confidential data in key-value pairs... allows you to decouple environment-specific configuration from your container images” (Source: [Kubernetes documentation — ConfigMaps](#)). Apigee Hybrid uses ConfigMaps for non-sensitive per-component configuration, including forward-proxy settings for individual components, which Apigee documents as configurable “in the `apigee-hybrid-config.yaml` file” (Source: [Apigee hybrid installation and administration guide](#)).

4.9 Secret

“A Secret is an object that contains a small amount of sensitive data such as a password, a token, or a key... Using a Secret means that you don’t need to include confidential data in your application code” (Source: [Kubernetes documentation — Secrets](#)). Apigee Hybrid uses Secrets for two major categories of sensitive material:

1. **Google Cloud service-account credentials.** Apigee’s install guide confirms two supported approaches: “traditional Google Cloud service account keys stored as Kubernetes secrets, or workload identity for GKE clusters” — and, for environments requiring an external secret manager, “Vault integration allows storing service account keys in external secret management systems using CSI drivers” (same source).
2. **TLS certificates** for the Istio Ingress Gateway and for environment-group hostnames. A documented, officially verified failure mode shows exactly why this matters operationally: when a required TLS secret is missing for an environment group, `apigee-watcher` logs `NOT_FOUND: failed to get secret... in namespace apigee`, and clients see SSL handshake failures against that hostname (Source: [Troubleshooting playbook: API calls failing with TLS errors](#)).

4.10 PersistentVolume

“A PersistentVolume (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes... has a lifecycle independent of any individual Pod that uses the PV” (Source: [Kubernetes documentation — Persistent Volumes](#)). **Cassandra is the component that requires Persistent Volumes** — each Cassandra StatefulSet pod is backed by its own PV so that data survives pod rescheduling, which is also why the cloud-provider CSI driver matters operationally: Google’s own troubleshooting documentation flags a real-world failure mode where Cassandra pods get stuck Pending on EKS due to “Missing Amazon EBS CSI driver” (Source: [Troubleshooting playbook: Cassandra](#)) — i.e., the storage class/CSI driver prerequisite is a real, documented gotcha, not a theoretical concern.

4.11 HorizontalPodAutoscaler (HPA)

“A HorizontalPodAutoscaler automatically updates a workload resource (such as a Deployment or StatefulSet), with the aim of automatically scaling capacity to match demand. Horizontal scaling means that the response to increased load is to deploy more Pods” (Source: [Kubernetes documentation — HorizontalPodAutoscaler](#)). Google’s product-team guidance states plainly that Message Processor (apigee-runtime) pods “autoscale via Kubernetes’ horizontal pod autoscaler” by default, while — critically — **“Cassandra does not have the ability to autoscale. Cassandra can easily be scaled up and down, but it must be done on-demand and in sets of three”** (Source: [Optimize managing APIs at scale across hybrid environments](#)). The “sets of three” constraint is a direct consequence of Cassandra’s ring topology and default replication factor of 3: adding or removing nodes in multiples of 3 keeps the ring balanced across replicas. The official troubleshooting playbook also confirms the apigee-connect-agent Deployment is paired with an HPA by default (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)).

4.12 Putting It Together: the GKE Cluster Layout

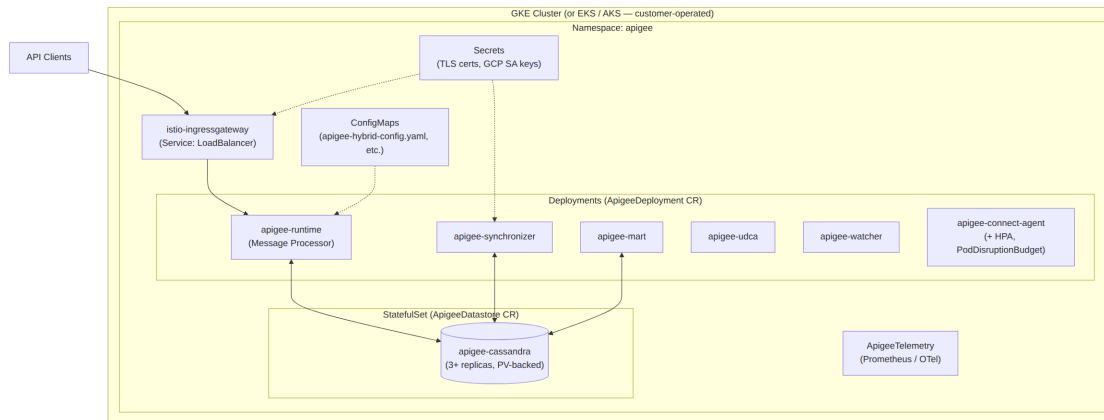


Diagram: original synthesis based on the verified component-to-Kubernetes-object mapping in [Troubleshooting Apigee hybrid stuck in creating or releasing state](#) and the install-guide details cited throughout this section. Not an official Google Cloud diagram image.

4.13 Illustrative Configuration Example

The example below is a **simplified, illustrative sketch** of the kind of overrides.yaml file apigeectl (or the Helm-based install path) consumes to configure a Hybrid runtime plane — it is written by the author of this handbook to teach the shape of the configuration, and is **not** a verbatim excerpt from Google’s documentation (this handbook could not independently re-verify the exact current schema for every field in this research pass; consult the current [Apigee hybrid configuration property reference](#) for exact, version-correct field names before using this in a real deployment):

```

# ILLUSTRATIVE EXAMPLE ONLY – verify exact schema against the current
# Apigee hybrid configuration property reference before real use.
gcp:
  projectID: my-gcp-project
  region: us-central1
k8sCluster:
  name: apigee-hybrid-prod-uc1
  region: us-central1
cassandra:
  replicaCount: 3          # Google's documented HA minimum
  storage:
    capacity: 100Gi
  resources:
    requests:
      cpu: "8"
      memory: 15Gi        # Google's documented production minimum
runtime:
  replicaCountMin: 3
  replicaCountMax: 10
  release:
    strategy: rolling
  resources:
    requests:
      cpu: "2"
      memory: 2Gi         # Google's documented recommendation
mart:
  replicaCountMin: 1
  replicaCountMax: 2
synchronizer:
  serviceAccountRef: synchronizer-sa
connectAgent:
  enabled: true            # avoids exposing MART ingress to Google
ingressGateways:
  - name: main-ingress
    svcType: LoadBalancer

```

This sketch illustrates the same structural facts established above in prose: `cassandra.replicaCount` maps directly to Cassandra’s StatefulSet replica count (Google’s documented minimum for HA is 3); `runtime.replicaCountMin/Max` maps to the HPA range for the Message Processor Deployment; `connectAgent.enabled` is the toggle that avoids exposing a MART ingress endpoint to Google.

Section 5 — Control Plane vs. Runtime Plane

5.1 Apigee X: Both Planes, One Owner

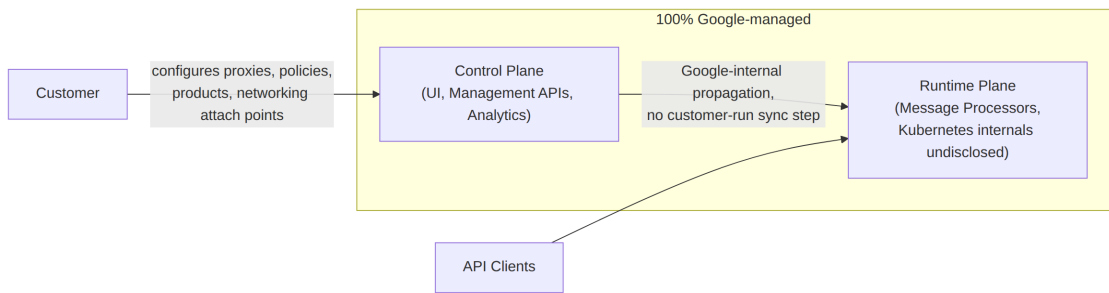


Diagram: original synthesis based on Section 2 of this handbook. Not an official Google Cloud diagram image.

In Apigee X, the control-plane/runtime-plane distinction still exists conceptually — Google’s engineering teams operate them as separate systems internally — but from the customer’s perspective **both are a single managed product**. There is no customer-run Synchronizer, no customer Kubernetes cluster, and no separate “runtime plane” the customer ever touches directly. The customer’s entire interaction surface is the Management API / UI (to configure proxies, products, environments) and the VPC networking attach points (to connect the tenant project to their own VPC).

5.2 Apigee Hybrid: Two Planes, Two Owners

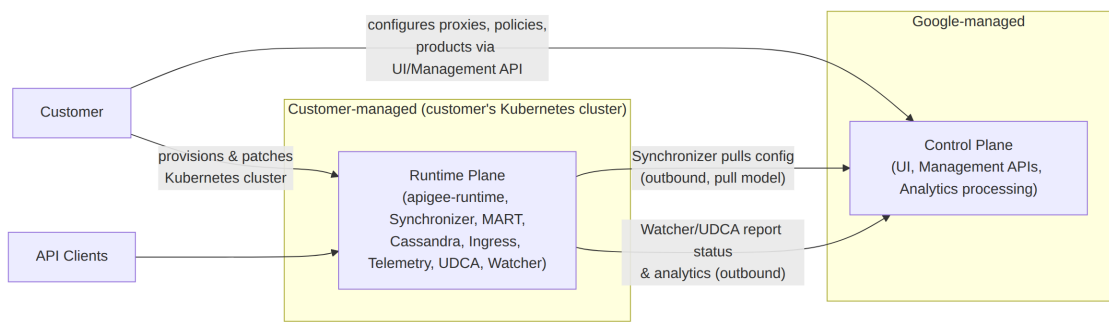


Diagram: original synthesis based on Section 3 of this handbook. Not an official Google Cloud diagram image.

In Apigee Hybrid, the same two planes are **owned by two different parties**, connected only by outbound-initiated calls from the runtime plane to the control plane (Synchronizer pulling config, Watcher/UDCA pushing status and analytics, Apigee Connect tunneling MART access) — a design explicitly chosen so the customer never has to open an inbound firewall port toward Google.

5.3 Responsibility Matrix

Question	Apigee X	Apigee Hybrid
Who owns the control plane (UI, Management API, analytics processing)?	Google	Google — identical to X for this half of the product

Question	Apigee X	Apigee Hybrid
Who owns the runtime/data plane (traffic processing)?	Google (fully managed, Kubernetes internals undisclosed)	Customer (their own Kubernetes cluster, namespace apigee)
Who manages Kubernetes itself?	N/A — not exposed to the customer at all	Customer — “Keep the Kubernetes platform and underlying infrastructure up to date with latest software updates and patches” is an explicit customer duty
Who manages upgrades of the runtime software?	Google (transparent to the customer)	Google ships the new container images/Operator version and “provid[es] upgrade notifications with documented steps,” but the customer executes the upgrade (e.g., <code>apigeectl apply / Helm upgrade</code>) against their own cluster
Who manages scaling of the runtime?	Google (invisible autoscaling within the single instance per region)	Customer , using Kubernetes-native mechanisms Google documents but does not operate: HPA for <code>apigee-runtime</code> (autoscales), manual scaling in multiples of 3 for Cassandra (does not autoscale)
Who manages TLS certificates for northbound traffic?	Customer configures the External Load Balancer and its certificates in their own project; Google operates the tenant-project side of the peering/PSC connection	Customer — “load balancer provisioning/management fronting APIs, including TLS certificate lifecycle” is an explicit customer duty; TLS material lives in Kubernetes Secrets in the customer’s cluster
Who patches vulnerabilities in Apigee’s own container images?	Google (fully managed)	Google — “continually scan components with the Container Analysis API and patch known vulnerabilities” — but the customer must pull and deploy the patched images to their cluster
Who is responsible for multi-region DR design?	Customer decides <i>whether</i> to go multi-region; Google does not auto-failover across regions (“services will not automatically move to an alternate region”)	Customer , fully — “multi-region deployment and disaster recovery” is an explicit customer duty, and there is no published Hybrid SLA at all
Who provisions compute/storage/network?	Google (inside the tenant project); customer only provisions their own VPC-side networking	Customer , entirely — “provisioning of the underlying compute, storage and network infrastructure” is an explicit customer duty
Is there a contractual SLA?	Yes — 99.99% (2+ regions with a multi-regional/global Cloud KMS key) or 99.9% (single region)	No — the Apigee SLA explicitly excludes “API calls processed by Apigee hybrid”

(Sources for this table: [Apigee hybrid shared responsibility model](#); [Apigee SLA](#); [Business continuity planning and disaster recovery](#); [Optimize managing APIs at scale across hybrid environments](#).)

5.4 Why Is Kubernetes Hidden in Apigee X?

This is one of the most commonly asked architect-level questions about Apigee, and the honest answer has three parts, each grounded in the facts established in Sections 2 and 3:

1. **It is a deliberate abstraction boundary, not a documentation gap.** Google states that the runtime is “a set of containerized runtime services in a Kubernetes cluster that Google maintains” — Google discloses *that* it’s Kubernetes-based, but not *which* distribution, *how* it’s configured, or *how* it scales internally. Compare this to Hybrid, where Google explicitly names GKE, EKS, and AKS as supported platforms and ships the Operator/CRDs the customer applies directly — a completely different disclosure posture for the exact same underlying orchestration technology.
2. **The product’s entire value proposition for Apigee X is “you concentrate on building your services.”** If Google exposed Kubernetes internals for the SaaS runtime, customers would inevitably start asking to tune node pools, override scheduling, or inspect pod-level behavior — reintroducing exactly the operational burden (“provisioning of the underlying compute, storage and network infrastructure,” “keep the Kubernetes platform... up to date”) that Hybrid explicitly assigns to the *customer* and X exists specifically to remove.
3. **Multi-tenancy and security isolation.** Apigee X runs each customer’s runtime inside a dedicated, Google-managed “tenant project,” in a 1:1 relationship with the customer’s Apigee organization. Disclosing and exposing the underlying cluster topology would either compromise that isolation model or force Google to support customer-facing operations against infrastructure it needs to be free to re-architect, re-balance, or migrate without a customer-visible contract.

The practical consequence for you as an architect: **when a client asks you to tune Kubernetes-level behavior (node pools, custom admission controllers, sidecar injection, custom autoscaling policies) for their API gateway layer, that requirement is a strong signal they need Apigee Hybrid, not Apigee X** — Section 10 turns this observation into a formal decision framework.

Section 6 — End-to-End Traffic Flow

6.1 Apigee X Request Flow

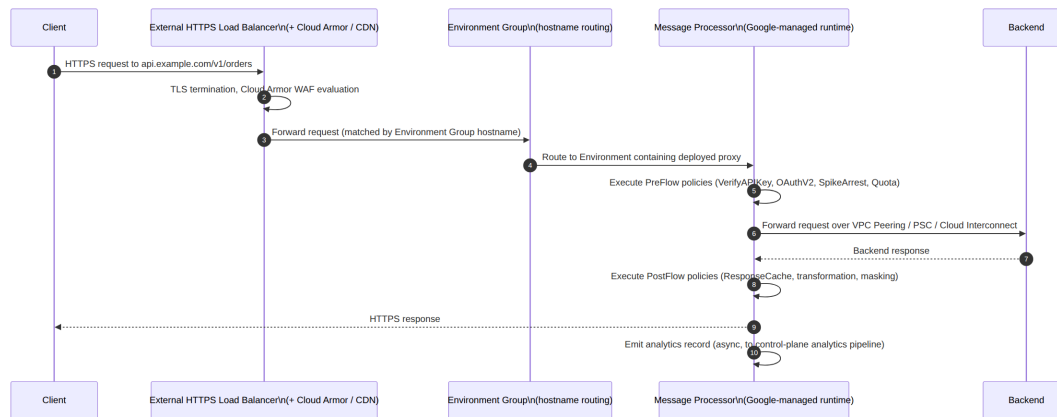


Diagram: original synthesis based on the northbound/southbound pattern in [Best practices for securing your applications and APIs using Apigee](#) and the component roles verified in Section 2. Not an official Google Cloud diagram image.

What happens at every stage:

1. **Client → Load Balancer.** The client resolves the API hostname via DNS and connects to Google Cloud's External HTTPS Load Balancer, which terminates TLS at the edge.
2. **WAF / bot protection.** If configured (Google's own recommendation per the Architecture Center article), Cloud Armor evaluates the request for IP/geo-based rules, and reCAPTCHA Enterprise may score the request as part of a WAAP pattern, before it ever reaches Apigee.
3. **Environment Group match.** Apigee inspects the Host header and matches it to an Environment Group; the group's member environments are the candidate set for base-path matching.
4. **Route to Message Processor.** Once the specific environment (and therefore the specific deployed proxy revision) is resolved by base path, the request lands on a Message Processor pod inside the Google-managed runtime.
5. **PreFlow policy execution.** Security (VerifyAPIKey, OAuthV2, JWT), traffic management (SpikeArrest, Quota), and message-level validation (OASValidation, threat protection) policies execute in the order configured in the proxy bundle.
6. **Backend call.** The Message Processor forwards the (possibly transformed) request to the backend target over whichever southbound path is configured — VPC Peering for GKE/Compute Engine, Private Service Connect for serverless/internal services, Cloud Interconnect for on-premises systems, or NAT egress for public third-party APIs.
7. **PostFlow policy execution and response.** Response transformation, caching (ResponseCache), and data masking policies run before the response streams back to the client.
8. **Analytics emission.** Independently of the response path, the Message Processor emits an analytics record asynchronously into Google's analytics processing pipeline, where it becomes visible in the Apigee Analytics UI.

6.2 Apigee Hybrid Request Flow

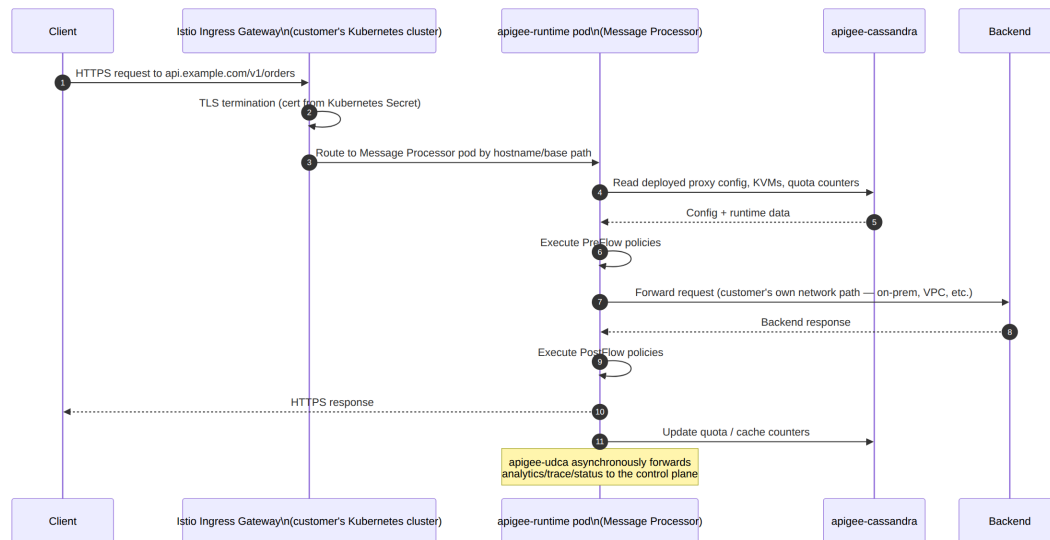


Diagram: original synthesis based on the verified runtime-plane component roles in Section 3.2. Not an official Google Cloud diagram image.

What happens at every stage:

1. **Client → Ingress.** The client connects to the Istio Ingress Gateway's Service (typically a Kubernetes Load-Balancer-type Service, fronted by whatever external load balancer the customer has provisioned per their own shared-responsibility duty). TLS is terminated using a certificate stored in a Kubernetes Secret.
2. **Routing to a Message Processor pod.** The ingress gateway routes to an apigee-runtime pod based on hostname/base-path matching, identical in concept to the Environment Group mechanism in Apigee X.
3. **Local config/data read.** Unlike Apigee X, the Message Processor here reads its deployed-proxy configuration, KVMs, and quota/cache state from the **local Cassandra StatefulSet** rather than from a Google-internal store — this local read is exactly what lets the runtime plane keep serving already-deployed traffic even if connectivity to the control plane is lost (see Section 9).
4. **PreFlow policy execution, backend call, PostFlow policy execution.** Identical in concept to Apigee X — the same policy engine and proxy model runs in both products; only the surrounding infrastructure differs.
5. **Response to client.**
6. **State update.** Cache/quota counters are updated back into Cassandra.
7. **Asynchronous telemetry.** apigee-udca forwards analytics, trace, and deployment-status data to the control plane out-of-band; apigee-watcher separately reports pod/runtime health status.

6.3 The One Structural Difference That Matters Most

Comparing 6.1 and 6.2 side by side, exactly one step differs in a way that has real operational consequences: **Apigee X's Message Processor reads runtime configuration from Google's internal systems; Apigee Hybrid's Message Processor reads it from a local, customer-operated Cassandra ring.** This is precisely why Hybrid's runtime plane can “function independently from the management plane” for a period of time (per Google's own product definition, Section 3.1) while Apigee X's runtime — being entirely Google's internal system end-to-end — has no equivalent customer-observable “disconnected mode” concept at all; from the customer's point of view in Apigee X, the runtime simply *is* always connected, because there is no separate plane boundary the customer ever crosses.

Section 7 — Configuration Flow and Synchronization

7.1 Proxy Deployment, End to End

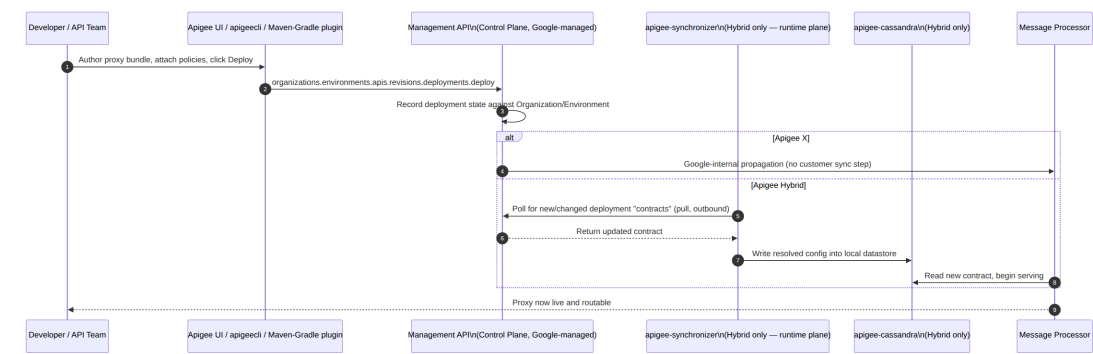


Diagram: original synthesis based on the deployment-revision mechanics documented at [Understanding deployment](#) (legacy Apigee domain, concept unchanged) and the Synchronizer/Cassandra roles verified in Section 3.2. Not an official Google Cloud diagram image.

The trigger mechanisms at the top of this flow are identical for both products: the Apigee UI, `apigeectl`, the Maven/Gradle Apigee plugins, `gcloud alpha apigee`, or a direct REST call all ultimately invoke the same Management API deployment operation. Deployment is **revision-based**: “revisions are sequentially numbered, enabling you to revert a change by deploying a previous revision,” and Apigee enforces that “only one revision of an API proxy can be deployed in an environment at a given time if revisions share the same API proxy base path” (Source: [Understanding deployment](#), legacy Apigee domain, concept current in Apigee X and Hybrid).

The fork in the flow is exactly the plane-ownership distinction from Section 5: in **Apigee X**, the Management API and the runtime are the same Google-managed system, so propagation is internal to Google with no customer-visible synchronization step. In **Apigee Hybrid**, the Synchronizer must actively pull the new deployment state across the control-plane/runtime-plane boundary before it reaches Cassandra and, from there, the Message Processor. Google does not publish an exact polling interval for this pull in the sources reviewed for this handbook — describe it as “the Synchronizer polls the Management API on a recurring basis” rather than citing a specific number.

7.2 Shared Flows

A Shared Flow lets you “combine policies and resources into a *shared flow* that you can consume from multiple API proxies, and even from other shared flows.” Unlike a proxy, “a shared flow has no endpoint. It can be used only from an API proxy or shared flow that’s in the same organization as the shared flow itself” (Source: [Shared flows](#), legacy Apigee domain, concept current). There are exactly two ways a shared flow gets invoked:

1. **Explicitly, via the `FlowCallout` policy** — placed anywhere inside a specific proxy’s own PreFlow, PostFlow, or conditional flow, giving per-proxy control over exactly when it runs.
2. **Implicitly, via a Flow Hook** — attached at the environment level, so it runs automatically for *every* proxy deployed to that environment, with no policy configuration required inside the proxies themselves.

Apigee defines four Flow Hook points (Source: [Flow hooks](#), legacy Apigee domain, concept current):

Flow Hook	Executes...
PreProxyFlowHook	“BEFORE a proxy endpoint executes”
PreTargetFlowHook	“BEFORE a target is called”

Flow Hook	Executes...
PostTargetFlowHook	“AFTER the target response executes”
PostProxyFlowHook	“AFTER the proxy endpoint and right before the response is sent out to the client”

“You can attach only one shared flow to each flow hook,” though “you can attach the same shared flow to multiple flow hooks” (same source) — this is the standard mechanism architects use to make security or logging logic *mandatory* across an entire environment (for example, forcing every proxy in prod through a common threat-protection shared flow) rather than trusting every proxy author to remember to include it.

7.3 Key Value Maps (KVMs)

“A KVM is a custom collection of key/value String pairs that is either encrypted or unencrypted” (Source: [Key Value Maps](#), legacy Apigee domain, concept current). Three scopes exist: **organization** (visible to any proxy in any environment), **environment** (visible only to proxies deployed in that environment), and **API-proxy** (visible only to that one proxy) — with a practical wrinkle worth remembering: “the management UI only exposes environment-scoped KVMs,” so organization- and proxy-scoped KVMs must be managed via the Management API or `apigeec li`. Encrypted KVMs use an Apigee-generated AES-128 key, display as `*****` in the UI/API, must be referenced with a `private.` variable prefix in policies to avoid leaking into debug/trace tooling, and cannot have their encryption setting changed after creation (delete and recreate is required). At runtime, the `KeyVa lueMapOperations` policy reads and writes entries via its `<Get>`, `<Put>`, and `<InitialEntries>` elements. In Apigee Hybrid, KVM data is expected to travel the same Synchronizer → Cassandra path as other environment/organization configuration, consistent with the general synchronization model in Section 7.1 — this handbook could not independently re-verify KVM-specific sync mechanics beyond that general model in this research pass.

7.4 Environment Configuration and Routing

Environment Groups own hostnames; environments are members of one or more groups; a deployed proxy’s base path must be unique across every environment within the same group so that a `Host` + base-path match is always unambiguous (see Section 2.2.4). This routing layer is identical in concept between Apigee X and Apigee Hybrid — the only difference is *where* the matching and execution physically happen: inside Google’s tenant project for X, or inside the customer’s Kubernetes cluster’s Istio Ingress Gateway and Message Processor pods for Hybrid.

7.5 The Synchronization Process, Summarized

For an architect, the Hybrid synchronization story reduces to four properties, all previously established with citations in Sections 3.2.2 and 6.3, restated here as a checklist because they come up constantly in design reviews:

- **Direction: pull, not push.** The Synchronizer initiates outbound calls from the runtime plane to the control plane’s Management API — Google never needs an inbound path into the customer’s cluster.
- **Landing zone: Cassandra.** Whatever the Synchronizer pulls is written into the local Cassandra datastore, not consumed directly by the Message Processor from the network.
- **Decoupled failure domains.** Because the Message Processor reads from local Cassandra rather than calling the control plane per-request, a control-plane/network outage stops *new deployments* but does not stop *already-deployed traffic* (Section 9 covers this in full).
- **No published interval.** Do not state an exact polling frequency as fact in a customer-facing design document — cite this handbook’s Section 3.2.2 caveat and point to the current [Configure the Synchronizer](#) page for the authoritative, version-current number if a client needs it.

Section 8 — Production Enterprise Architecture

8.1 An Honest Starting Point

Before the diagrams: unlike GKE or BigQuery, Apigee does not have a large published catalog of Cloud Architecture Center reference architectures. A targeted search of the Architecture Center turned up exactly **one** Apigee-specific page — [Best practices for securing your applications and APIs using Apigee](#) — and it is focused on security layering (Cloud Armor, reCAPTCHA, WAAP), not topology, HA, or DR. The architectures in this section are therefore built by combining Google’s **product documentation** (the Limits reference, the SLA, the Business Continuity page), Google’s **product-team blog posts**, and standard, well-established distributed-systems patterns for the parts Google’s own docs leave to the customer — each labeled accordingly. An architect presenting these patterns to a client should say plainly that Google’s own published topology guidance for Apigee is comparatively thin, and that several of the specific sizing/interval numbers below come from Google’s blog rather than a numbered reference-architecture page.

8.2 Single-Region Apigee X

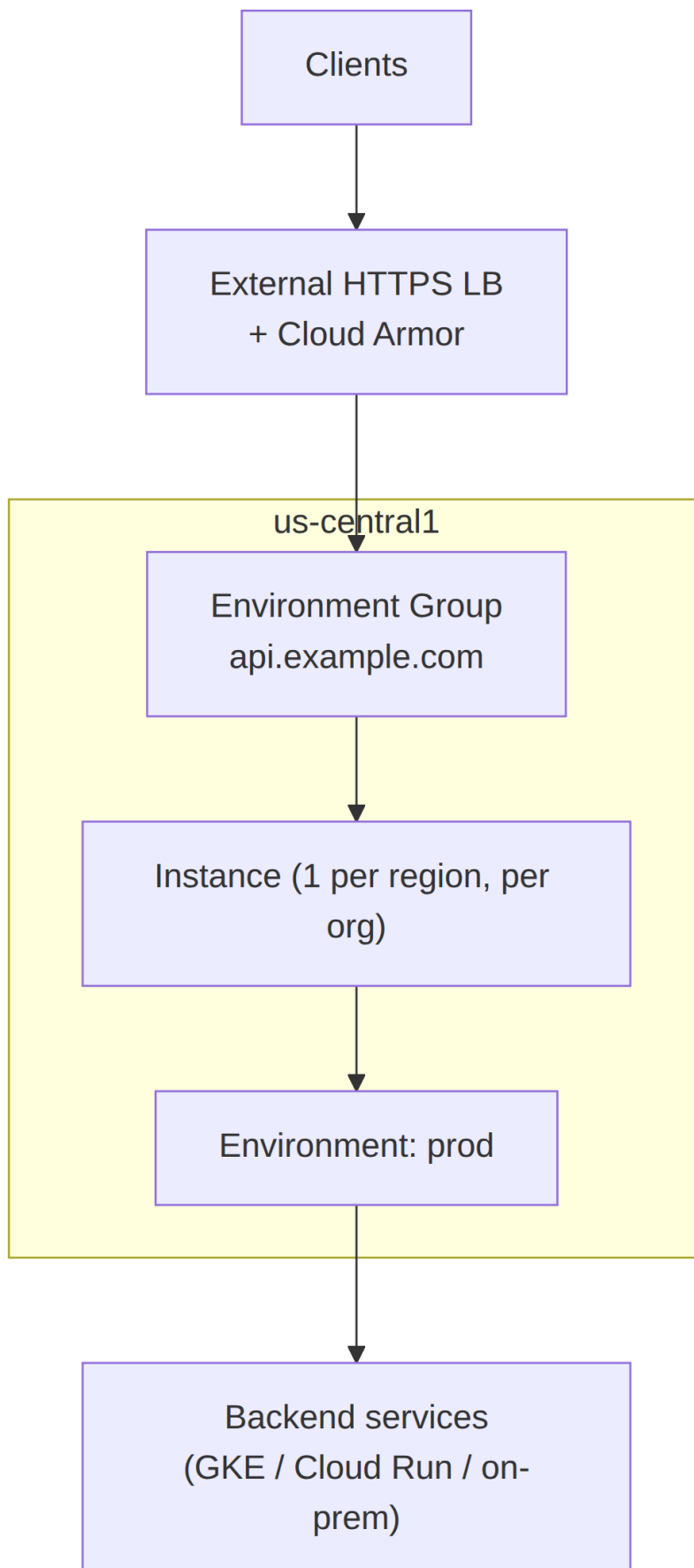


Diagram: original synthesis based on the resource hierarchy in Section 2.2 and the Limits reference. Not an official Google Cloud diagram image.

A single-region Apigee X deployment is bound by the hard limit established in Section 2.2.2: **exactly one Instance per region per organization** (Source: [Apigee Limits reference](#)). All scale-out within the region happens invisibly inside that one Instance’s Google-managed capacity. This topology carries the **99.9%** monthly uptime tier under the Apigee SLA (Source: [Apigee SLA](#)), and Google’s own Business Continuity documentation is explicit about the residual risk: “if an entire region were to go offline, a single-region customer could experience an outage of API processing services” (Source: [Business continuity planning and disaster recovery](#)).

8.3 Multi-Region Apigee X

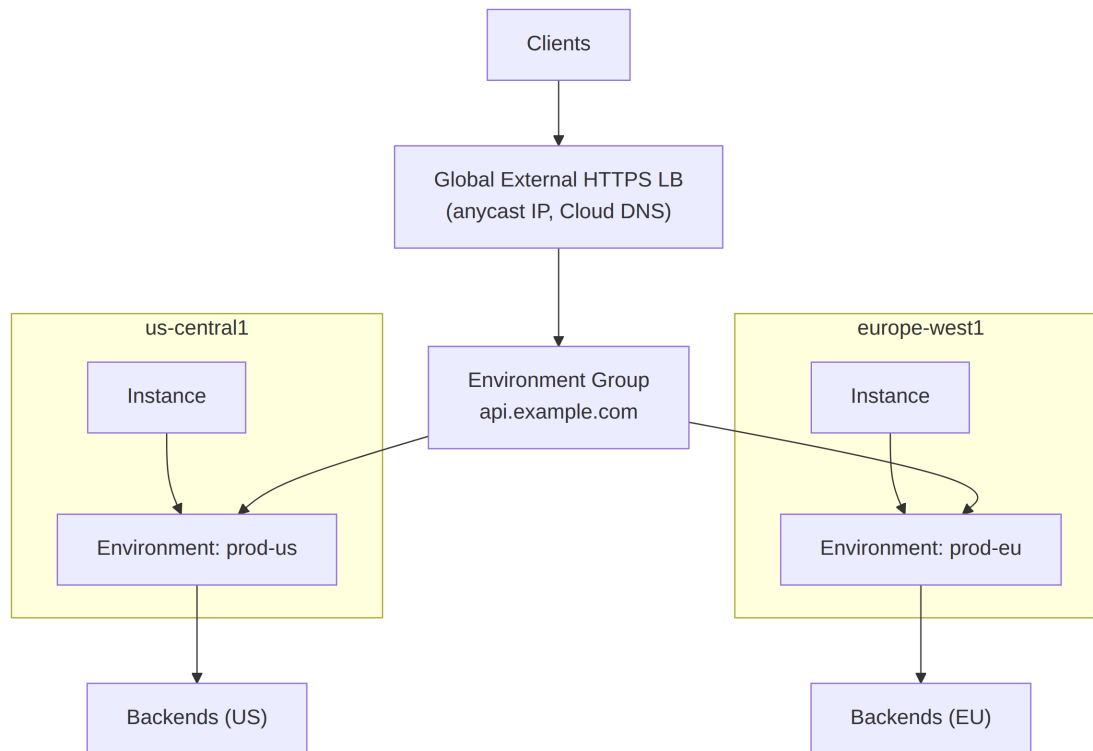


Diagram: original synthesis based on the Environment Group routing mechanism in Section 2.2.4, combined with the SLA’s multi-region tiering. Not an official Google Cloud diagram image.

Because an Environment Group can route its hostname’s traffic to environments that live in different regions/instances (Section 2.2.4), Apigee X’s multi-region pattern is architecturally simple to describe: add another region’s Instance, attach a new Environment to it, and add that Environment to the same Environment Group that already serves the hostname. The concrete payoff for doing this is contractual, not just aspirational: the Apigee SLA jumps to **99.99%** monthly uptime specifically “for environments provisioned in 2 or more Regions with a dual-region, multi-regional, or global Cloud KMS encryption key,” versus 99.9% otherwise (Source: [Apigee SLA](#)) — meaning the encryption-key topology, not just the region count, is part of what Google contractually requires for the higher tier. Google is equally explicit about what it will **not** do for you here: “services will not automatically move to an alternate region, as this could potentially compromise Google’s commitments to its customers,” and “customers can and should consider how Apigee can redirect traffic as needed” (Source: [Business continuity planning and disaster recovery](#)) — the global load balancer and DNS failover behavior shown above is standard Google Cloud load-balancing practice applied by the customer, not a bespoke Apigee capability Google operates on the customer’s behalf.

8.4 Single-Region Apigee Hybrid

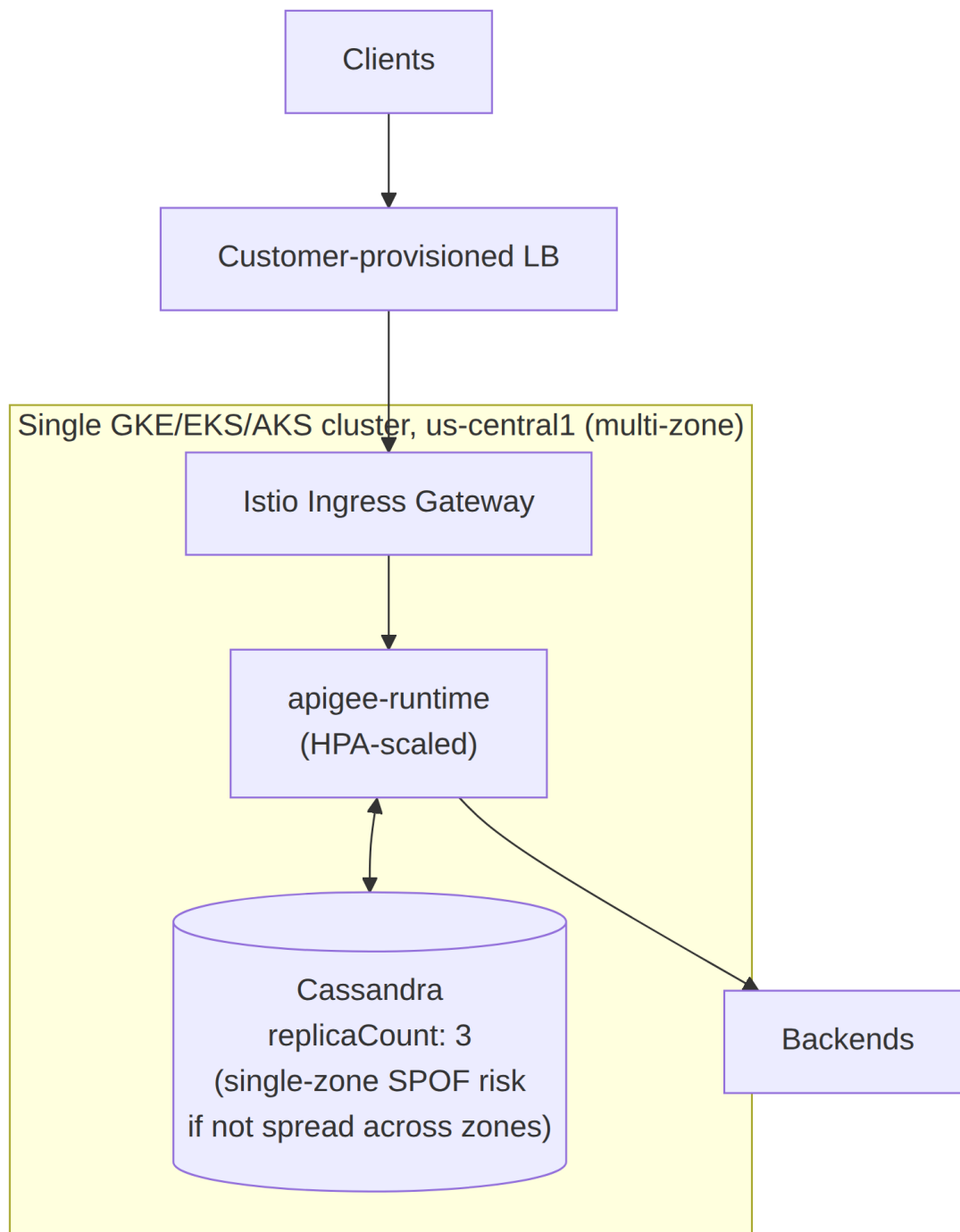


Diagram: original synthesis based on Google's product-team sizing guidance in [Optimize managing APIs at scale across hybrid environments](#) and the component facts in Section 3.2. Not an official Google Cloud diagram image.

A single-region Hybrid deployment is one Kubernetes cluster, with a Cassandra `replicaCount` of at least 3 to avoid the documented single-point-of-failure risk of a 1-pod Cassandra (Source: [No active runtime pods warning](#)). Google does not publish Apigee-specific guidance in the sources reviewed for this handbook on whether the cluster itself should be zonal or regional — this is standard GKE production guidance applied by the architect, not an Apigee-specific citation, and should be presented to a client as such: a **regional cluster spreading nodes across at least three zones** is the conventional choice for any stateful production workload, Cassandra very much included.

8.5 Multi-Region Apigee Hybrid

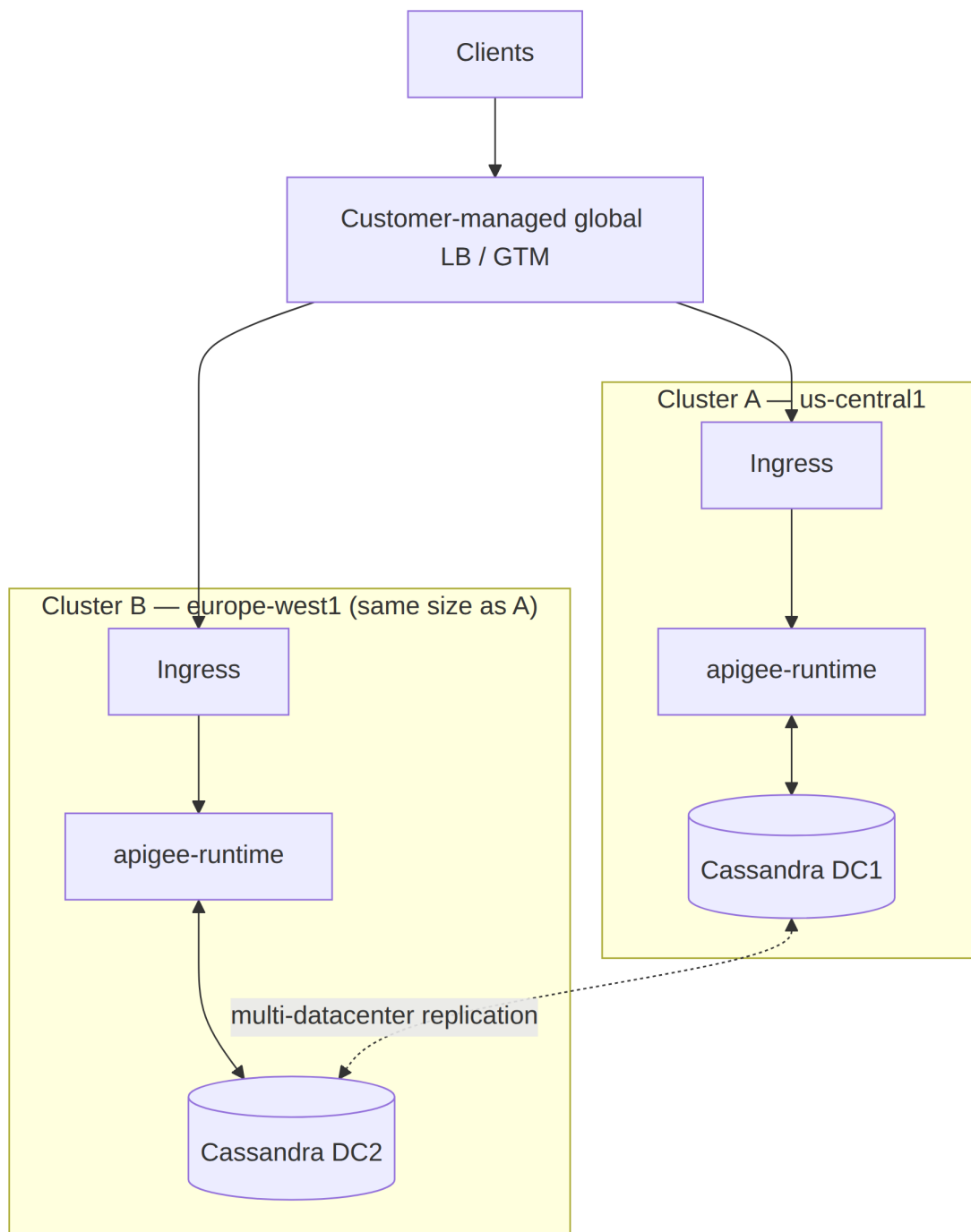


Diagram: original synthesis based directly on Google’s stated production baseline in [Optimize managing APIs at scale across hybrid environments](#). Not an official Google Cloud diagram image.

Google’s product engineering team states the production baseline for Hybrid directly and unambiguously: **“Your production platform should have at least two Kubernetes clusters, each in a different region,”** sized identically — “we recommend that they are the same size. That way, one can handle the full load if the other cluster fails” (same source). This is a **one-cluster-per-region** model, with Cassandra configured for multi-datacenter replication across the clusters (standard Cassandra ring-of-rings topology; the exact `dc_name/seed-host` configuration mechanics live in versioned Apigee documentation this handbook could not independently re-verify in this research pass — consult [Multi-region deployment](#) directly for the current, version-correct procedure). The same source’s guidance on Cassandra scaling applies at every region: it “must be done on-demand and in sets of three,” and does not autoscale.

8.6 Disaster Recovery Architecture

	Apigee X	Apigee Hybrid
Published SLA?	Yes — 99.99% (multi-region + multi-regional/global KMS key) or 99.9% (single region)	No — the SLA explicitly excludes “API calls processed by Apigee hybrid”
Published RTO/RPO?	No — “Apigee does not offer recovery point and recovery time objectives (RPO/RTO) for customers or in contracts related to BCP/DR activities,” though Google frames its own live/live architecture as making RTO/RPO “real-time” from its side	No published figure; entirely a customer design responsibility
Auto cross-region failover?	No — explicitly disclaimed for data-sovereignty reasons	No — entirely customer-built (global LB / GTM in front of multiple regional clusters)
Customer’s DR responsibility	Decide whether to deploy multi-region; design the traffic-redirection strategy on top of Apigee	Design and build the entire multi-cluster topology, Cassandra multi-DC replication, and failover mechanism

(Source: [Business continuity planning and disaster recovery](#); [Apigee SLA](#); [Apigee hybrid shared responsibility model](#).)

The clearest way to frame this for a client: Google’s underlying philosophy — stated directly in the Business Continuity documentation — is a **live/live** model (“two (or more) data centers constantly and simultaneously servicing customer traffic in each region globally”) rather than a traditional active/passive DR pair, for the portion of the stack Google itself operates. For Apigee X, that means the *region* is your unit of failure, and the *customer*’s job is deciding how many regions to buy and how to route around a dead one. For Apigee Hybrid, Google gives you none of that live/live infrastructure for the runtime plane — you are building the equivalent of Google’s own live/live posture yourself, cluster by cluster, region by region.

8.7 High Availability Architecture — Summary Checklist

- **Apigee X:** multi-region Environment Group attachment + a dual-region/multi-regional/global Cloud KMS key (required for the 99.99% SLA tier) + a customer-operated global load balancer/DNS failover strategy in front of it.
- **Apigee Hybrid:** at least two same-sized regional Kubernetes clusters, each running the full component stack from Section 3.2, Cassandra `replicaCount` ≥ 3 per cluster with multi-datacenter replication across clusters, HPA-scaled `apigee-runtime`, a customer-managed global load-balancing layer in front of the clusters’ ingress gateways, and CI/CD/GitOps automation for proxy/config rollout — Google’s own guidance recommends automation specifically to make “adding additional regions easy” and to simplify “recreating clusters during disasters” (Source: [Optimize managing APIs at scale across hybrid environments](#)).

Section 9 — Troubleshooting Architecture

This section applies to Apigee Hybrid, since Apigee X’s runtime failure modes are internal to Google and not customer-observable or customer-actionable in the same way. Every failure scenario below is sourced directly from Google’s own official troubleshooting playbooks.

9.1 Runtime Pod (apigee-runtime / Message Processor) Failure

- **Symptom.** The Message Processor pod fails its readiness probe and is removed from traffic serving.
- **Root cause pattern.** Most commonly, this is a *downstream* symptom rather than a primary failure: either Cassandra is unreachable (Section 9.3) or the Synchronizer has never delivered an active “contract” for that environment, logged as “no active contract is available for the message processor to serve the traffic” (Source: [Troubleshooting playbook: message processor](#)).
- **Impact.** New deployments cannot go live in the affected environment; if enough replicas are affected, existing traffic capacity in that environment is reduced or lost.
- **Recovery.** Because apigee-runtime is a standard Deployment, Kubernetes will restart failed pods automatically; recovery is otherwise gated on resolving the underlying Cassandra or Synchronizer issue.

9.2 Synchronizer Failure

- **Symptom.** New API Products, Developers, and Apps stop appearing in the Apigee UI; new proxy deployments never complete.
- **Root cause pattern.** Network connectivity from the runtime plane to the control plane’s Management API is broken — Google’s documentation states plainly: “if no network route is available to the internet, the Apigee runtime plane components will not be able to communicate with Apigee control plane APIs” (Source: [Troubleshooting playbook: no network connectivity](#)). Common concrete causes include missing firewall allow-listing for apigee.googleapis.com, a misconfigured forward proxy, or — for private GKE clusters — Private Google Access not being enabled (“Private Google Access needs to be enabled to allow the Apigee runtime plane components access to Google APIs internally,” same source).
- **Impact.** **Already-deployed traffic is unaffected** — this is the single most important fact to remember about Synchronizer failures, and the one most likely to be tested in an interview. Only the *control path* for new/changed configuration is broken.
- **Recovery.** Restore network connectivity (route, firewall rule, forward-proxy config, or Private Google Access setting); the Synchronizer will resume pulling on its own once the path is restored, with no data loss for already-deployed proxies.

9.3 Cassandra Failure

- **Symptom.** apigee-runtime, apigee-mart, and apigee-synchronizer pods all enter CrashLoopBack-Off simultaneously, with the log signature “Cannot build a cluster without contact points”; the Apigee UI shows a “No active runtime pods” warning, a “Datastore Error” on the Products/Apps pages, and an “Internal Server Error” on the Developers page (Source: [No active runtime pods warning](#)).
- **Root cause pattern.** Most commonly a single-replica Cassandra configuration acting as a documented single point of failure, insufficient disk space (there is a dedicated official playbook for this exact scenario), or — on EKS specifically — a missing Amazon EBS CSI driver leaving Cassandra pods stuck Pending (Source: [Troubleshooting playbook: Cassandra](#)).
- **Impact.** This is the most severe of the documented failure modes: it takes down MART, Synchronizer, *and* the Message Processor’s ability to serve or deploy, because all three depend directly on a healthy Cassandra connection.
- **Recovery.** Google’s own guidance is preventive: configure `cassandra.replicaCount: 3` (or higher, in multiples of 3) so no single pod failure can take the ring down; ensure adequate persistent-volume disk capacity; on EKS, confirm the EBS CSI driver is installed. Once Cassandra is healthy again, dependent pods recover automatically without manual intervention.

9.4 MART Failure

- **Symptom.** Documented as occurring *together with* the Cassandra-outage scenario above, rather than as an independently documented standalone failure mode — apigee-mart pods enter CrashLoopBackOff alongside apigee-runtime and apigee-synchronizer when Cassandra is unreachable.
- **Impact.** Management-plane operations that need to reach into runtime-plane state (for example, certain KVM or runtime-status operations) fail.
- **Recovery.** Because MART’s documented failure mode is coupled to Cassandra, the recovery path is the same as Section 9.3: restore Cassandra health.

9.5 Ingress Failure

- **Symptom.** SSL handshake failures and connection resets for clients hitting a specific environment group’s hostname.
- **Root cause pattern.** A missing or misconfigured TLS secret for that environment group — the official play-book shows apigee-watcher logging NOT_FOUND: failed to get secret... in namespace apigee in this exact scenario (Source: [Troubleshooting playbook: API calls failing with TLS errors](#)).
- **Impact.** Unlike a Synchronizer outage, this **is** traffic-affecting immediately — every client attempting to reach that hostname fails at the TLS layer before ever reaching a Message Processor.
- **Recovery.** Recreate/correct the Kubernetes Secret holding the environment group’s TLS certificate and key; apigee-watcher reconciles the ingress gateway configuration once the secret is present and valid.

9.6 Failure-Impact Comparison Table

Failing component	New deployments?	Existing traffic?	UI/Management API?	Severity
Synchronizer / control-plane connectivity	Blocked	Unaffected	Products/Developers/Apps stop populating	Low –Medium
Cassandra	Blocked	Affected (runtime pods CrashLoop)	Datastore errors across UI	High
MART	Coupled to Cassandra	Coupled to Cassandra	KVM/runtime-status operations fail	High (when co-occurring with Cassandra failure)
Ingress / TLS	N/A	Affected immediately (TLS handshake failures)	N/A	High , but scoped to the affected environment group
Message Processor (apigee-runtime)	Blocked in affected environment	Reduced capacity or affected, depending on replica health	N/A	Medium–High, scoped to the affected environment

9.7 The Broader Resilience Narrative

A useful mental model, corroborated (though not itself an official `cloud.google.com` citation — sourced from a Google Cloud Community discussion, flagged accordingly) by community guidance consistent with every officially-verified mechanism above: **“Apigee Hybrid is built for resilience, not isolation.”** Existing configuration continues to work because it is cached locally in Cassandra; if pods restart, “the data is pulled from cassandra to resume runtime operations.” What breaks during an extended control-plane disconnection is specifically the ability to deploy *new or modified* proxies, populate *new* Developer/App/Product entities in the UI, and get *timely* analytics/debug visibility — not the serving of already-deployed traffic. This is the direct, practical payoff of the local-Cassandra-read architecture established in Section 6.3, and it is the single cleanest way to explain to a client why Hybrid’s failure domains are structured the way they are.

Section 10 — Apigee X vs. Apigee Hybrid

10.1 Comparison Matrix

Dimension	Apigee X	Apigee Hybrid
Runtime ownership	Google (fully managed, single-tenant tenant project)	Customer (their own Kubernetes cluster)
Kubernetes ownership	Not exposed to the customer at all	Customer provisions, patches, scales, and secures the cluster; Google ships container images + Operator/CRDs
Security model	Google-managed runtime isolation (tenant project); customer configures Cloud Armor/IAM/policies at the edge and in the proxy layer	Customer owns cluster-level security (RBAC, network policy, node hardening, Secrets/Vault) in addition to the same proxy-layer policy configuration
Networking	VPC Peering (PSA) or Private Service Connect between customer VPC and Google's tenant project; Google-provisioned northbound LB	Fully customer-defined — on-premises networks, any cloud VPC, hybrid Interconnect/VPN topologies, customer-provisioned ingress
Cost model	Consumption-based Apigee pricing; no separate infrastructure bill for the runtime	Apigee licensing plus the customer's own Kubernetes cluster compute/storage/networking costs, plus the operational (headcount) cost of running that cluster
Scalability	Google-managed autoscaling inside each region's single Instance; customer scales by adding regions/environments	Customer-managed: HPA for apigee-runtime, manual multiples-of-three scaling for Cassandra, cluster-level node pool scaling
Operations	Near-zero infrastructure operations for the runtime; customer operates proxies/policies/products only	Full Kubernetes operations burden (upgrades, patching, monitoring, incident response) on top of proxy/policy/product operations
Maintenance	Google patches and upgrades the runtime transparently	Google ships patched images and upgrade instructions; customer executes every upgrade against their own cluster
Compliance	Runtime processing happens inside Google's infrastructure — acceptable for most compliance regimes, but data-in-transit/at-rest leaves the customer's own network boundary during processing	All API traffic "is processed within the boundaries of your network and under your control" (Source: What is Apigee hybrid?) — the primary reason regulated industries and data-sovereignty-constrained customers choose Hybrid

Dimension	Apigee X	Apigee Hybrid
Data residency	Constrained by which Google Cloud regions Apigee X supports; traffic processing occurs in Google’s tenant project in that region	Traffic processing occurs on infrastructure the customer chooses and controls — on-premises, any supported cloud, any region, including locations Apigee X may not support at all
Upgrades	Transparent, Google-managed, no customer action required	Google provides “upgrade notifications with documented steps”; customer executes (apigeectl/Helm) against their own cluster on their own schedule
SLA	Yes — 99.99% (multi-region + qualifying Cloud KMS key) or 99.9% (single region) (Source: Apigee SLA)	None published — Apigee’s SLA explicitly excludes Hybrid

10.2 Architect Recommendations

Choose Apigee X when:

- The organization wants to minimize operational headcount and infrastructure ownership for the API gateway layer — this is the product’s entire design intent (“allowing you to concentrate on building your services”).
- All backend systems the APIs need to reach already live inside Google Cloud (or are reachable via VPC Peering, Private Service Connect, or Cloud Interconnect from a Google Cloud tenant project) — Section 2.4’s networking patterns are all designed around this assumption.
- A contractual, numeric SLA (99.9%/99.99%) is a procurement or compliance requirement, and the organization is comfortable with API traffic being processed inside Google-managed infrastructure.
- The team has no strong requirement to customize Kubernetes-level behavior (custom admission controllers, node-level tuning, non-standard service mesh configuration) underneath the gateway — if they *do* have such a requirement, that is the clearest single signal to route them to Hybrid instead (Section 5.4).
- Time-to-value matters more than infrastructure control: there is no cluster to stand up before the first proxy can be deployed.

Choose Apigee Hybrid when:

- Data residency, sovereignty, or regulatory requirements mandate that API traffic never leave a specific network boundary (on-premises, a specific sovereign cloud region, or an air-gapped environment) — Hybrid’s entire value proposition is that “all API traffic is processed within the boundaries of your network and under your control.”
- The organization already operates Kubernetes at scale and has the platform-engineering capability to absorb another production workload (patching, monitoring, incident response, capacity planning) — Hybrid trades Google’s operational burden for the customer’s, and that trade is only a win if the customer’s platform team is genuinely strong.
- Backend systems are on-premises, in another cloud, or split across multiple environments in a way that a single Google-managed tenant project’s networking model cannot cleanly reach.
- The organization needs Kubernetes-level customization underneath the API gateway — custom service-mesh policy, specific node pool isolation for compliance, integration with an existing internal PKI/Vault-based secrets model, or coexistence with other workloads on the same cluster.
- There is no hard requirement for a contractual SLA on the gateway layer itself, and the organization is comfortable building its own multi-region/DR posture from scratch, since Google publishes none for Hybrid.

The single most reliable diagnostic question to ask a client: “When something goes wrong with the API gateway at 2 a.m., who do you want holding the pager — Google’s SRE team, or your own platform team?” If the honest answer is “Google’s,” recommend Apigee X. If the honest answer is “ours, because we need the control,” recommend

Apigee Hybrid — and be candid that the price of that control is Kubernetes operations, not just an Apigee license fee.

Section 11 — Interview Preparation

This section is a standalone study reference for architect-level interviews, design-review panels, and certification-style oral exams on Apigee X and Apigee Hybrid. It is organized into three question banks — Apigee X architecture, Apigee Hybrid architecture, and Kubernetes as it specifically applies to Apigee Hybrid — each numbered independently so it can be studied as a self-contained set. Every answer that states a specific documented fact (a limit, an SLA percentage, a named component behavior, a quoted definition) carries the same inline citation format used throughout this handbook; where a question calls for architectural judgment rather than a documented fact — “why would you choose X over Hybrid,” “how would you design DR” — the answer is given confidently from the architecture established in Sections 1–10, without a fabricated citation. Where Google does not publish an exact number (a polling interval, a minimum Kubernetes version, an autoscaling threshold), the answer says so plainly rather than inventing one — this is intentional, and matches the discipline this handbook has followed throughout; carrying that same discipline into a live interview is, itself, a mark of seniority.

11.1 Top 50 Apigee X Architecture Questions

Q1. What is an Apigee organization, and why does it matter that it is bound 1:1 to a Google Cloud project?

An Apigee **Organization** is “a container for all the entities in an Apigee account, including API proxies, API products, API packages, apps, and developers,” and Google requires that “to install and use Apigee or Apigee hybrid, you must have an Apigee organization that is bound to a Google Cloud project” (Source: [Understanding commonly used Apigee terms](#)). This binding is architecturally significant because it is what lets Apigee X plug directly into Cloud IAM for access control, Cloud Billing for consumption-based pricing, Cloud KMS for encryption keys, and VPC networking for connectivity — as first-class Google Cloud citizens rather than through a bolted-on identity or billing system. It also means an organization’s lifecycle (creation, deletion, project-level quota) is tied to a single GCP project, which has real implications for how you structure environments across business units or subsidiaries — most enterprises choose one organization per major business domain rather than one per team, precisely because the org boundary is also a project boundary.

Q2. What is an Apigee Environment?

An **Environment** is “a runtime execution context for APIs. An API must be deployed to an environment before it can be accessed at runtime” (Source: [Understanding commonly used Apigee terms](#)). In practice, environments are how you separate test, dev, and prod traffic within the same organization, each with its own deployed proxy revisions, KVMs, and target servers. An environment only serves traffic once it is attached to an Instance in a specific region, so an environment is a logical execution context, while the Instance it is attached to is the physical regional capacity that actually runs it. Apigee X allows up to 85 environments per organization, contract-dependent (Source: [Apigee Limits reference](#)).

Q3. What is an Environment Group, and why does it exist as a separate layer above Environments?

An Environment Group is “the basic mechanism for defining the way requests are routed to individual environments. You define hostnames on your environment groups (not on individual environments), and Apigee routes requests to the environments within a group using those hostname definitions” (Source: [Understanding commonly used Apigee terms](#)). This indirection exists because hostnames and environments have different lifecycles: you want to be able to swap which environment answers for `api.example.com` — for a blue/green cutover, a canary, or a disaster-recovery failover — without touching DNS or your load balancer configuration. Because an environment can belong to more than one group, and a group’s hostnames can route to environments in different regions, the Environment Group is also the structural mechanism that makes Apigee X’s multi-region topology possible.

Q4. What is an Instance, and what is the hard limit around instances per region?

An Instance is the unit of regional runtime capacity inside an Apigee organization. Apigee X enforces exactly **one instance per Google Cloud region, per organization**, with a maximum of 10 instances/regions per organization (11 for Apigee Hybrid) (Source: [Apigee Limits reference](#)). This single constraint is why Apigee X's regional growth story is “add another region,” not “add another instance in the same region” — there is no horizontal instance-level sharding within a region; capacity growth inside one region happens invisibly inside that one instance's Google-managed runtime.

Q5. Can an environment belong to multiple environment groups, and what does that enable architecturally?

Yes — this is standard, well-established Apigee behavior consistent across the resource model. Belonging to multiple groups lets a single environment serve traffic under more than one hostname (for example, a legacy domain and a rebranded domain both routing to the same prod environment during a migration window), and, conversely, lets an Environment Group route the same hostname's traffic to environments in more than one region for multi-region deployments. The base-path uniqueness constraint that Apigee enforces across environments in the same group is what keeps this routing unambiguous — no two proxies with the same base path can be simultaneously deployed to two environments in the same group.

Q6. Walk through the full Apigee X resource hierarchy from organization down to API proxy.

The hierarchy starts at the **Organization**, which is bound 1:1 to a Google Cloud project and contains everything else. Inside an organization you provision **Instances** (regional runtime capacity, capped at one per region) and **Environments** (logical execution contexts, such as test or prod); each environment is attached to an instance in a specific region so it can actually serve traffic there. **Environment Groups** sit above environments as the routing layer, owning hostnames and directing incoming requests to their member environments by hostname match. Finally, **API Proxies** — versioned as sequential **revisions** — are deployed into one or more environments, where they execute **Policies** and are exposed to consumers through **API Products**, which bundle proxy resources with a rate-limit/quota plan and are what a **Developer App** is actually granted access to.

Q7. What is the difference between an API Product and an API Proxy?

An API Proxy is “a proxy that acts as a facade for your existing API... This facade decouples your public interface from your backend API, shielding developers from backend changes” (Source: [Understanding commonly used Apigee terms](#)) — it is the technical implementation artifact: the actual flows, policies, and target endpoints. An API Product is “a collection of API resources (URIs) combined with a service plan and presented to developers as a bundle” (same source) — it is the commercial/governance packaging layer that developers actually subscribe to. A single proxy's resources can be exposed through multiple products with different quota plans (for example, a free tier and a paid tier), which is precisely why the two concepts are kept separate rather than collapsed into one.

Q8. What is a Developer App, and how does it relate to API Products?

A Developer App is a credential-bearing registration, created by a registered Developer, that is granted access to one or more API Products; the app receives a consumer key/secret pair that is presented at runtime — typically validated via the `VerifyAPIKey` or `OAuthV2` policies — to authenticate and authorize each call. This model is standard, well-established Apigee behavior consistent across Edge, X, and Hybrid: Developer → Developer App → (authorized against) → API Product → (bundles) → API Proxy resources. It is the backbone of Apigee's developer-portal-driven, self-service API monetization and governance story.

Q9. What is a Revision, and can two revisions of the same proxy be live in an environment at once?

Revisions are how Apigee versions an individual API proxy: “revisions are sequentially numbered, enabling you to revert a change by deploying a previous revision” (Source: [Understanding deployment](#), legacy Apigee domain, concept unchanged in current Apigee X). Critically, “only one revision of an API proxy can be deployed in an environment at a given time if revisions share the same API proxy base path” (same source) — so no, you cannot

have two revisions with the same base path simultaneously live in one environment; deploying a new revision to that same base path in that environment implicitly supersedes the old one. This is why blue/green or canary rollouts at the proxy level are typically done by deploying the new revision to a *different* environment first (or by manipulating base paths/hostnames), not by trying to run two conflicting revisions side by side in the same environment.

Q10. What are the environment and environment-group limits per organization, and when would you actually hit them?

Apigee X allows up to 85 environments per organization (contract-dependent) and up to 85 environment groups per organization, with a maximum of 100 environment-group attachments (Source: [Apigee Limits reference](#)). In practice, large enterprises with many independent business units, each wanting their own dev/test/prod triplet inside a single shared Apigee organization, are the ones who approach this ceiling — a common architectural response is to split into multiple Apigee organizations (and, by extension, multiple GCP projects) once a single business's environment count starts crowding a shared org, rather than trying to squeeze every division into one organization's environment budget.

Q11. What is the Message Processor in Apigee X, and is it still current, official terminology?

Yes — the Message Processor remains current, official terminology, not a legacy Edge-era term that's been quietly retired. Google's own error catalog states directly: "The Message Processor is the core component of Apigee that processes the policies and interacts with the backend servers" (Source: [Message Processor error catalog](#)), and Google's troubleshooting documentation confirms "the message processor is part of the apigee-runtime component," running as "pods that can autoscale or restart" (Source: [Troubleshooting playbook: message processor](#)). It is the component that actually executes a proxy's flows and policies and talks to backend targets — functionally, the API gateway engine itself.

Q12. Where does the Message Processor physically run in Apigee X?

It runs inside a Google-managed, single-tenant "tenant project" — "ApigeeX is a Private Service Access (PSA) based service that is deployed and managed in a tenant project. A tenant project is a project that is tied to the Apigee X Runtime, in a one-to-one relationship," and that runtime "resides in a Google managed VPC" (Source: [Apigee X design for multiple distinct VPC connections](#)). Google's own definition of the base "Apigee" runtime confirms it is "a set of containerized runtime services in a Kubernetes cluster that Google maintains" (Source: [What is Apigee?](#)) — it is Kubernetes underneath, but that cluster is entirely outside the customer's own project and control.

Q13. Does Google disclose whether Apigee X's runtime is GKE specifically?

No. Google discloses that the runtime is Kubernetes-based but never names the specific distribution, and does not expose cluster-level configuration, node pools, or scheduling behavior to the customer at all. This is a deliberate abstraction boundary rather than a documentation gap: exposing that detail would invite exactly the kind of Kubernetes-tuning requests (custom admission controllers, node-pool overrides, sidecar injection) that Apigee X's entire value proposition — "allowing you to concentrate on building your services" (Source: [What is Apigee?](#)) — is designed to make unnecessary. This contrasts sharply with Apigee Hybrid, where Google explicitly names GKE, EKS, and AKS as supported platforms because the customer runs the cluster directly.

Q14. What is a "tenant project" in Apigee X, and why does the customer never see it directly?

A tenant project is a Google-managed Google Cloud project, created and operated by Google, tied 1:1 to a customer's Apigee X organization, inside which the actual Message Processor runtime executes (Source: [Apigee X design for multiple distinct VPC connections](#)). The customer never logs into this project or manages resources inside it directly; their only interaction with it is indirect, through network attachment points — VPC Peering, Private Service Access, or Private Service Connect — that connect their own VPC to the tenant project's Google-managed VPC. This isolation model is what allows Google to re-architect, patch, or rebalance the underlying infrastructure without ever exposing that churn to the customer as a breaking change.

Q15. How does Apigee X scale within a single region — at the instance level or the pod level?

Entirely at the pod level, invisibly to the customer. Because there is a hard limit of exactly one Instance per region per organization (Source: [Apigee Limits reference](#)), an architect cannot add a second instance to absorb more load in the same region — all scale-out inside a region happens inside that single instance’s Google-managed Kubernetes runtime, which Google’s own documentation confirms runs “as pods that can autoscale or restart” (Source: [Troubleshooting playbook: message processor](#)). The customer’s only scaling lever within a region is choosing which regions to deploy environments into at all; capacity planning and autoscaling behavior within a region are entirely Google’s operational responsibility.

Q16. Why can’t you inspect or SSH into the Kubernetes cluster backing Apigee X?

Because you don’t own it — it lives inside a Google-managed tenant project, and Google’s shared-responsibility posture for Apigee X (in contrast to Hybrid) places both the control plane and the runtime plane entirely under Google’s operation, with no customer Kubernetes access surface at all. This is architecturally the single most important distinction between Apigee X and Apigee Hybrid: in Hybrid, “Keep the Kubernetes platform and underlying infrastructure up to date with latest software updates and patches” is an explicit customer duty (Source: [Apigee hybrid shared responsibility model](#)); in Apigee X, there is no equivalent customer duty because there is no customer-facing cluster to patch, inspect, or SSH into in the first place.

Q17. What is Private Services Access (PSA), and how does Apigee X use it?

Private Services Access is the default connectivity model Apigee X uses to let the customer’s VPC reach the Google-managed tenant project by internal IP address, implemented via VPC Network Peering: “VPC peering is the easiest option. The restriction is that connectivity wise, only one peering hop is allowed, which is already consumed by a connection to the Apigee X Runtime” (Source: [Apigee X design for multiple distinct VPC connections](#)). Because Apigee X consumes the customer’s single available peering hop just to reach the tenant project, any additional backend VPCs the customer wants to reach require an additional pattern layered on top — Cloud VPN, Shared VPC, or Private Service Connect — rather than simply peering again.

Q18. Why is pure VPC Peering a scaling problem when Apigee X needs to reach many backend VPCs?

Because VPC Peering is fundamentally non-transitive — “only two VPCs can be peered in sequence” (Source: [Apigee X design for multiple distinct VPC connections](#)) — and Apigee X has already consumed its one peering hop connecting to the tenant project. If ten different backend VPCs each need direct connectivity, you cannot simply peer the tenant project to all ten; peering doesn’t chain. This is precisely the problem that motivates hub-and-spoke patterns over Cloud VPN (which is transitive), Shared VPC hub designs, and eventually Private Service Connect as the more scalable native solution.

Q19. What is the Shared VPC hub pattern, and why does it get around the peering-transitivity limit?

In this pattern, the Apigee X tenant project’s host VPC is itself made a Shared VPC, because “Shared VPC subnets do not violate the VPC peering transitivity limit” (Source: [Apigee X design for multiple distinct VPC connections](#)). This creates what Google’s engineers describe as an “inner hub-and-spoke” — the Shared VPC containing the Apigee X host project — nested inside an “outer hub-and-spoke” of VPN-connected backend spokes, a design the source calls “analogous to cascading hub-and-spoke architecture.” A further evolution, a “wheel” pattern where spoke VPCs peer directly to one another, removes the hub dependency entirely for high-fanout, multi-tenant scenarios.

Q20. What is Private Service Connect, and how does it differ from VPC Peering for Apigee X southbound traffic?

Private Service Connect (PSC) “allows private consumption of services across VPC networks,” and unlike PSA/VPC Peering, “PSC is the missing piece that will natively support Apigee X backends in other VPCs” without exposing the underlying network infrastructure: “unlike VPC Peering, the underlying infrastructure is not exposed: connecting and managing services is much simpler, more secure and private” (Source: [Connecting to Cloud Run from Apigee X](#)

using PSC). A key architectural constraint is that “these two attachments (Endpoint and Service Attachments) along with the L7 ILB must be part of the same GCP region” (same source), so PSC connectivity design has to respect regional co-location even though it removes the peering-transitivity problem entirely.

Q21. Describe the PSC pattern for reaching a backend like Cloud Run from Apigee X.

The documented pattern is: Apigee X’s Message Processors call out to a **PSC Endpoint Attachment** in the tenant project, which connects to a **PSC Service Attachment** in the backend project (same region as required), which fronts an **Internal HTTPS Load Balancer**, which in turn routes to a **Serverless NEG** in front of the actual Cloud Run service (Source: [Connecting to Cloud Run from Apigee X using PSC](#)). This layered approach lets the backend team fully control what is exposed through the Service Attachment without ever exposing their VPC’s internal topology to the Apigee-side consumer, which is exactly the isolation property PSC is designed to provide over raw peering.

Q22. What is the distinction between northbound and southbound traffic in Apigee X, and what handles each?

Northbound (ingress) traffic is external client traffic arriving at Apigee: the documented pattern routes it through an **External Application Load Balancer** (TLS termination), layered with **Cloud Armor** (WAF) and optionally **reCAPTCHA Enterprise** in a combined WAAP pattern, before it reaches Apigee, which “is provisioned in a Google Cloud project and the runtime is provisioned and peered in a tenant project using VPC Network Peering” (Source: [Best practices for securing your applications and APIs using Apigee](#)). Southbound (egress) traffic is Apigee reaching backends: the same source documents GKE backends reached “via RFC 1918 internal IPs over VPC Network Peering,” on-premises/data-center services reached over **Cloud Interconnect**, and third-party APIs reached via NAT egress.

Q23. What is the recommended pattern for edge security in front of Apigee X, and why not rely purely on Apigee’s own policies?

Google’s Cloud Architecture Center recommends layering **Cloud Armor** — which “evaluates every call made by the client that hits the external Application Load Balancer” — in front of Apigee, explicitly stating “we recommend that you use Cloud Armor for IP address-based and geo-based access control” rather than depending solely on Apigee’s native `AccessControl` policy (Source: [Best practices for securing your applications and APIs using Apigee](#)). The architectural rationale is defense-in-depth and efficiency: blocking malicious or geo-restricted traffic at the load balancer, before it ever consumes a Message Processor cycle, is both more scalable and gives you a broader WAF rule surface (with optional reCAPTCHA Enterprise for bot mitigation) than Apigee’s own proxy-level policies alone can provide.

Q24. How does Apigee X handle egress to the public internet by default?

By default, non-private egress traffic — calls Apigee X makes out to third-party APIs over the public internet, rather than into a peered or PSC-connected private backend — routes through **Cloud NAT** provisioned in the tenant project: “By default, egress to non-private routes from Apigee X is through Cloud NAT in the tenant project” (Source: [Apigee X design for multiple distinct VPC connections](#)). This means the source IPs seen by third-party backends are the tenant project’s Cloud NAT addresses, which is a common allow-listing consideration when a partner API requires a fixed, known source IP range.

Q25. When would you use a “wheel” topology instead of hub-and-spoke for Apigee X’s VPC connectivity?

A wheel topology — where spoke VPCs peer directly to each other rather than routing everything back through a central hub — is the right choice when you have many high-throughput, tightly coupled backend VPCs that need to talk to each other independently of Apigee, and you want to avoid the hub becoming a throughput or operational bottleneck, particularly in high-fanout, multi-tenant productization scenarios where a large number of spokes would otherwise all funnel through one hub project. In most enterprise deployments, though, a Shared VPC hub-and-spoke design is preferable because it centralizes network policy and simplifies auditing; the wheel pattern is a specialized choice you reach for once hub-centric topology genuinely becomes the bottleneck, not a default starting point.

Q26. How do you reach an on-premises backend from Apigee X?

Via **Cloud Interconnect** (or, for lower-throughput needs, Cloud VPN), which the Cloud Architecture Center documents as the standard southbound pattern for on-premises/data-center services, illustrated with a scenario of payments and accounts services in a data center reached from a money-transfer service in Google Cloud (Source: [Best practices for securing your applications and APIs using Apigee](#)). This is a genuinely hybrid enterprise topology even though the API gateway itself is the fully-managed Apigee X product — a useful reminder that “Apigee X” and “hybrid enterprise architecture” are not mutually exclusive; the product is SaaS, but its backends can span anywhere Cloud Interconnect or VPN can reach.

Q27. What are the three policy families the Cloud Architecture Center groups Apigee’s most common policies into?

The Architecture Center’s security guidance groups policies into **Traffic Management** (SpikeArrest, Quota, ResponseCache), **message-level protection** (OASValidation, JSONThreatProtection, XMLThreatProtection), and **Security** (BasicAuthentication, VerifyAPIKey, OAuthV2, JWT, CORS) (Source: [Best practices for securing your applications and APIs using Apigee](#)). This grouping is a useful mental model for a design review because it maps directly to three different failure modes you’re protecting against: capacity abuse, malformed/malicious payloads, and unauthorized access, respectively — and a mature proxy design typically has at least one policy from each family, not just an authentication policy alone.

Q28. What’s the architectural difference between OAuthV2 and VerifyAPIKey as security policies?

VerifyAPIKey is a simple, static-credential check: the consumer key issued to a Developer App is validated against Apigee’s stored app/product association, with no token issuance, refresh, or scope model involved — it answers “is this a known, authorized app?” and nothing more. OAuthV2 is a full token-based flow that supports issuing, validating, and refreshing access tokens, optionally scoped to specific permissions, and is the appropriate choice whenever you need delegated authorization (a user granting a third-party app limited access on their behalf) or short-lived, revocable credentials rather than a long-lived static key. As an architecture decision, VerifyAPIKey is adequate for simple machine-to-machine or partner integrations where the key itself is tightly controlled; OAuthV2 is the right choice the moment end-user delegation or fine-grained, revocable scope enforcement enters the picture.

Q29. When would you use SpikeArrest versus Quota?

SpikeArrest protects against short-term traffic spikes and burst abuse by smoothing request rate over very short intervals (for example, evening out a burst of 100 requests in one second so the backend sees a steadier rate) — it is a traffic-shaping control, not a business-level entitlement. Quota enforces a longer-term, business-level usage ceiling (for example, 10,000 calls per day per API Product) tied to the developer’s plan or contract, and is what actually governs how much of the API a given consumer is entitled to use over time. In a well-designed proxy you typically want both: SpikeArrest to protect the backend from sudden bursts regardless of who’s calling, and Quota to enforce the commercial/contractual usage limit per consumer.

Q30. Why does Google recommend Cloud Armor over relying solely on Apigee’s native AccessControl policy for IP- and geo-based blocking?

Because Cloud Armor operates at the load balancer, ahead of Apigee entirely, and is purpose-built as a WAF with a broader and more efficiently enforced rule surface, whereas Apigee’s AccessControl policy only evaluates once a request has already reached a Message Processor and consumed proxy execution resources — Google states plainly, “we recommend that you use Cloud Armor for IP address-based and geo-based access control” (Source: [Best practices for securing your applications and APIs using Apigee](#)). Blocking at the edge is both cheaper (no wasted Message Processor cycles on traffic that should never have been let through) and more scalable, which is the general architectural principle behind “reject bad traffic as early in the pipeline as possible.”

Q31. What do JSONThreatProtection and XMLThreatProtection guard against?

These are message-level protection policies that inspect the structure of incoming JSON or XML payloads for patterns designed to exhaust backend resources or exploit parser vulnerabilities — for example, excessively deep object

nesting, extremely large arrays, oversized attribute counts, or entity-expansion attacks in XML (the classic “XML bomb” pattern), all of which the Cloud Architecture Center groups under message-level protection alongside OAS-Validation (Source: [Best practices for securing your applications and APIs using Apigee](#)). Architecturally, they matter because authentication and authorization policies (like `VerifyAPIKey` or `OAuthV2`) only establish *who* is calling — they say nothing about whether the payload itself is safe to parse and forward to a backend, which is precisely the gap these threat-protection policies close.

Q32. How would you use a Shared Flow plus a Flow Hook to enforce a mandatory security policy across an entire environment?

You would author the security logic (say, a mandatory JWT validation and threat-protection sequence) once as a Shared Flow, then attach that Shared Flow to a Flow Hook — for example, `PreProxyFlowHook` — at the environment level, rather than requiring every proxy author to remember to add a `FlowCallout` to that shared flow inside their own proxy. Because “you can attach only one shared flow to each flow hook” but that attached flow then executes “for every proxy deployed to that environment” (Source: [Flow hooks](#), legacy Apigee domain, concept current), this is the standard mechanism for making a security or logging policy organization-mandatory and audit-proof, rather than merely convention-based and dependent on individual proxy authors remembering to include it.

Q33. How does Apigee X’s analytics pipeline work at a high level?

Message Processors generate per-transaction analytics records asynchronously as they process traffic — capturing latency, status codes, fault information, and custom analytics variables — and because both the control plane and runtime plane in Apigee X are Google-managed, these records flow directly into Google’s analytics processing pipeline without any customer-run collection step, visible through the same Apigee Analytics UI and API. This is architecturally simpler than the Hybrid equivalent, where a dedicated UDCA component in the customer-run runtime plane must actively forward that data to the control plane — in Apigee X there is no separate forwarding hop because there is no separate customer-operated runtime plane to forward it from.

Q34. What data does Apigee’s analytics pipeline capture per transaction?

At minimum, per-transaction analytics captures request/response latency, HTTP status codes, fault/error information when a policy or backend call fails, and any custom analytics variables a proxy developer chooses to populate (for example, a specific header value, a business identifier extracted from the payload, or a custom dimension used for downstream reporting). This data is what populates the standard dashboards (traffic volume, error rates, latency percentiles) as well as any custom reports built against the Analytics API, and is the same underlying model whether the proxy is deployed in Apigee X or Apigee Hybrid — the difference is purely in how the data travels from the Message Processor to the analytics store.

Q35. Is there a component called “UAP” or “Unified Analytics Platform” in Apigee X?

No — this term does not appear in any official Google source reviewed for this handbook, and using it in an interview or design document risks signaling unfamiliarity with current terminology rather than expertise. The correct, defensible way to describe this part of the architecture is generically, as “Apigee’s analytics processing pipeline” or “Apigee’s control-plane analytics processing” — Google’s own documentation does not name a discrete “UAP” component, and this handbook deliberately avoids the term for that reason.

Q36. How would you expose a custom business metric through Apigee analytics?

You would populate a custom analytics variable inside the proxy flow — typically by extracting a value from the request, response, or a policy’s outcome (for example, a customer tier, a product SKU, or a partner identifier) and assigning it to a flow variable that Apigee’s analytics collection framework is configured to capture as a custom dimension. Once captured, that dimension becomes queryable through the Analytics API and reportable through custom dashboards, exactly like the built-in dimensions (status code, proxy name, environment) — the mechanism is the same in both Apigee X and Apigee Hybrid, since it is a proxy-configuration-level concept rather than something tied to which product deployed the Message Processor.

Q37. Walk through what happens end-to-end when you deploy a proxy revision in Apigee X.

The trigger — whether it's the Apigee UI, `apigeecli`, a Maven/Gradle plugin, `gcloud alpha apigee`, or a direct REST call — ultimately invokes the same Management API deployment operation (`organizations.environments.apis.revisions.deploy`). Because in Apigee X the Management API and the runtime are part of the same Google-managed system, the control plane records the new deployment state against the organization/environment and propagates it to the Google-managed Message Processors internally, with no customer-visible synchronization step in between — in practice, from the moment the deploy call succeeds, the new revision is live and routable, architecturally consistent with the deployment mechanics described generically at [Understanding deployment](#) (legacy Apigee domain, concept unchanged). This is the single biggest operational simplification Apigee X offers over Hybrid, where the same flow requires an active Synchronizer pull before the new revision reaches a Message Processor.

Q38. What tools can trigger a proxy deployment in Apigee X?

The Apigee UI, `apigeecli` (the community/Google-maintained CLI), the Maven and Gradle Apigee plugins for CI/CD pipelines, `gcloud alpha apigee` commands, and direct REST calls against the Management API all converge on the same underlying deployment operation. Most mature enterprise deployments avoid the UI for production changes entirely and drive deployment exclusively through CI/CD using `apigeecli` or the Maven/Gradle plugin, precisely so that proxy promotion between environments is auditable, repeatable, and gated behind the same code-review and pipeline controls used for any other production change.

Q39. What happens if you try to deploy two revisions that share the same base path to the same environment?

Apigee will not let both be simultaneously active — “only one revision of an API proxy can be deployed in an environment at a given time if revisions share the same API proxy base path” (Source: [Understanding deployment](#), legacy Apigee domain, concept unchanged in current Apigee X). Deploying the new revision either fails until the conflicting one is undeployed, or (depending on the deployment call's `override` flag) supersedes the previously deployed revision at that base path. This constraint exists because base-path-plus-hostname is exactly how the routing layer disambiguates which proxy should handle an incoming request, so two live revisions with the same base path in the same environment would make that routing decision ambiguous.

Q40. How would you implement a blue/green deployment using Apigee X's resource model?

Because two conflicting revisions can't coexist at the same base path in one environment, the standard architectural pattern is to deploy the new (“green”) revision into a *separate* environment from the current (“blue”) one, then shift traffic between them at the Environment Group / hostname routing layer rather than at the proxy-revision layer itself — for example, by adjusting which environment a group's hostname routes to, or by fronting both environments with a customer-managed load balancer that can weight traffic between them. This pushes the cutover decision up to a layer Apigee is explicitly designed to make cheap to change — Environment Group hostname routing — rather than trying to force it through the proxy-revision mechanism, which is intentionally single-active-revision-per-base-path.

Q41. What is a Shared Flow, and how does it differ from an API proxy?

A Shared Flow lets you “combine policies and resources into a *shared flow* that you can consume from multiple API proxies, and even from other shared flows,” but “although it's like a proxy, a shared flow has no endpoint. It can be used only from an API proxy or shared flow that's in the same organization as the shared flow itself” (Source: [Shared flows](#), legacy Apigee domain, concept current). Architecturally, an API proxy is a directly invocable, endpoint-bearing unit that clients call; a Shared Flow is a reusable library of policy logic with no independent endpoint, invoked either explicitly via the `FlowCallout` policy from inside a proxy, or implicitly and environment-wide via a Flow Hook.

Q42. What is a KVM, and what are its three scopes?

A Key Value Map is “a custom collection of key/value String pairs that is either encrypted or unencrypted” (Source: [Key Value Maps](#), legacy Apigee domain, concept current), accessed at runtime through the `KeyValueMapOperations` policy. The three scopes are **organization** (visible to any proxy in any environment), **environment** (visible

only to proxies deployed in that specific environment), and **API-proxy** (visible only to that single proxy) — with the practical wrinkle that “the management UI only exposes environment-scoped KVMs,” so organization- and proxy-scoped KVMs must be managed through the Management API or `apigeecli`. Encrypted KVMs use an Apigee-generated AES-128 key, are masked as `*****` in the UI/API, must be referenced with a `private.` variable prefix to avoid leaking into debug/trace tooling, and cannot have their encryption setting changed after creation without deleting and recreating the KVM.

Q43. What is the deployed-proxies-per-environment limit in Apigee X, and why does it matter architecturally?

Apigee X allows up to 6,000 deployed proxies per environment (Source: [Apigee Limits reference](#)). This matters because at true enterprise scale — hundreds of backend services each fronted by their own proxy, across dozens of teams — it is a real number architects need to check against, not a theoretical ceiling; organizations approaching it typically respond by splitting proxies across additional environments or reconsidering whether every backend genuinely needs its own dedicated proxy rather than being consolidated behind shared, parameterized proxies.

Q44. How many key stores are allowed per environment, and why does that matter for TLS-heavy, multi-tenant designs?

Apigee X allows up to 100 key stores per environment (Source: [Apigee Limits reference](#)). This is relevant to any design where each backend target, or each partner integration, requires its own distinct TLS keystore/truststore pair for mutual TLS — a multi-tenant B2B integration platform with many partners, each needing dedicated client certificates for outbound mTLS to their systems, needs to plan environment-per-tenant-group topologies carefully once the number of distinct keystores starts to approach this ceiling.

Q45. What is the max number of regions/instances per organization, and how does it tie into the underlying Cassandra pod constraint mentioned in Apigee’s limits documentation?

Apigee’s Limits reference caps regions per organization at 10 (11 for Hybrid), and notes this is “tied to a max of 150 Cassandra pods per organization” (Source: [Apigee Limits reference](#)). The Cassandra reference here reflects underlying platform capacity constraints documented on the same limits page rather than a customer-operated component in Apigee X specifically — Cassandra is the concept the customer directly operates in Apigee Hybrid (Section 3.2.3), and its appearance in this general limits note is best understood as evidence that similar underlying datastore scaling constraints shape both products’ regional ceilings, not as a claim that Apigee X customers run or see Cassandra themselves.

Q46. What is the API base-path limit per organization, and what design pattern helps avoid hitting it?

Apigee X allows up to 6,000 API base paths per organization (Source: [Apigee Limits reference](#)). The design pattern that helps avoid crowding this limit is consolidating related backend operations behind fewer, more generically parameterized proxies (using path parameters and conditional flows instead of a dedicated base path per narrow operation), rather than defaulting to “one proxy, one base path, per microservice endpoint” — a pattern that scales linearly with backend count and can burn through this budget quickly in a large microservices estate.

Q47. What is the exact Apigee X SLA tiering?

The Apigee SLA guarantees a Monthly Uptime Percentage of “≥ 99.99% for environments provisioned in 2 or more Regions with a dual-region, multi-regional, or global Cloud KMS encryption key” versus “≥ 99.9% for all other [single-region] environments,” with a 99.5% carve-out for the Mexico and Stockholm regions specifically (Source: [Apigee SLA](#)). Critically, the SLA’s “Covered Service” definition explicitly excludes “API calls processed by Apigee hybrid or Apigee Adapter for Envoy,” which means **Apigee Hybrid has no published SLA at all** — the 99.99%/99.9% tiering applies only to Apigee X.

Q48. Does Apigee X publish RTO/RPO figures, and what philosophy does Google offer instead?

No — Google states plainly that “Apigee does not offer recovery point and recovery time objectives (RPO/RTO) for customers or in contracts related to BCP/DR activities” (Source: [Business continuity planning and disaster recovery](#)). Instead, Google frames its own architecture as **live/live** rather than active/passive — “two (or more) data centers constantly and simultaneously servicing customer traffic in each region globally” — arguing that from Google’s side, RTO and RPO “can both be seen as ‘real-time’ ” because there is no cold or warm standby to fail over to in the first place; the residual risk this leaves the customer is entirely at the region level, addressed in Q49.

Q49. If an entire GCP region goes offline, does Apigee X automatically fail your traffic over to another region?

No, and Google is explicit about this: “services will not automatically move to an alternate region, as this could potentially compromise Google’s commitments to its customers” — a statement grounded in data-sovereignty concerns, not a technical limitation (Source: [Business continuity planning and disaster recovery](#)). The same source is direct about the consequence for single-region customers: “if an entire region were to go offline, a single-region customer could experience an outage of API processing services.” The customer’s own architecture — a global load balancer, DNS-based failover, or Environment Group routing to environments in a second region — is what determines whether cross-region failover happens at all; Google states this responsibility plainly: “customers can and should consider how Apigee can redirect traffic as needed for customers to maintain end-user services.”

Q50. When would you recommend a customer accept a single-region 99.9% Apigee X deployment rather than paying for multi-region 99.99%?

This is a judgment call, not a documented rule, and it comes down to the actual business cost of an outage versus the added cost and complexity of running multi-region: for an internal-facing API estate, a low-traffic partner integration, or an early-stage product where 99.9% (roughly 43 minutes of allowed downtime a month) is genuinely tolerable, the added cost of a second region, cross-region KVM/config consistency planning, and a global-load-balancer failover design often isn’t justified. The moment the API in question is customer-facing at meaningful volume, tied to revenue, or contractually obligated to a downstream SLA that Apigee’s own 99.9% single-region tier can’t comfortably support, the calculus flips — and the fact that the higher 99.99% tier also requires a dual-region/multi-regional/global Cloud KMS key (not just a second region) means this decision has to be made before key provisioning, not retrofitted later.

11.2 Top 50 Apigee Hybrid Architecture Questions

Q1. What is the fundamental architectural split in Apigee Hybrid?

Apigee Hybrid splits the product into a **management plane**, hosted in the cloud and maintained by Google, and a **runtime plane**, which the customer sets up and maintains in their own Kubernetes cluster. Google’s own definition states it directly: “Apigee hybrid lets you manage APIs on-premises, on Google Cloud Platform (GCP), or a mix of both... All API traffic is processed within the boundaries of your network and under your control, while management services such as the UI and API analytics run in the cloud and are maintained by Google” (Source: [What is Apigee hybrid?](#)). The runtime plane is explicitly designed to “function independently from the management plane” for a period of time, which is the architectural basis for Hybrid’s resilience story covered later in this section.

Q2. What does Google operate in Apigee Hybrid, contractually?

Per Google’s shared-responsibility documentation, Google’s responsibilities include delivering “Apigee hybrid packages including runtime, mart, Cassandra, and Kubernetes controllers and management tools,” “keeping Management APIs and Analytics compatible with customer deployments,” continually scanning components with the Container Analysis API and patching known vulnerabilities, “providing upgrade notifications with documented steps,” and “providing the connections to the management plane and Google Cloud Observability” (Source: [Apigee hybrid shared responsibility model](#)). Note the precise phrasing: Google “delivers” the runtime, MART, and Cassandra as packages — container images and an Operator/CRD layer — it does not claim to operate them once they’re running on the customer’s cluster.

Q3. What does the customer operate in Apigee Hybrid, contractually?

The customer’s documented responsibilities include “maintaining connectivity between the Apigee hybrid data plane and the Google managed Apigee control plane” (including VPN/Interconnect), “provisioning of the underlying compute, storage and network infrastructure as per Apigee’s recommendations,” keeping “the Kubernetes platform and underlying infrastructure up to date with latest software updates and patches,” load balancer provisioning and TLS certificate lifecycle, application code/config/container images/data/RBAC/IAM policies and the running containers themselves, logging/monitoring configuration, and “multi-region deployment and disaster recovery” (Source: [Apigee hybrid shared responsibility model](#)). This is, in effect, everything downstream of “Google ships you the container images and an Operator” — the customer runs the cluster.

Q4. Why is it significant that Google’s shared-responsibility page says it “delivers packages” rather than “operates the runtime”?

Because that precise wording is the textual proof of exactly where the operational boundary sits: Google builds and ships the container images and the Kubernetes Operator/CRDs that define the runtime plane’s shape, but it is the customer who actually runs `kubectl apply` (or `apigeectl`, or Helm) against their own cluster and keeps that cluster alive. This is the single fact that most cleanly answers “where does Kubernetes fit into Apigee Hybrid”: Kubernetes is the customer’s own compute substrate, into which Google ships a pre-built application as container images plus an Operator that reconciles custom resources — not a Google-operated service the customer merely configures, as it is in Apigee X.

Q5. What does it mean in practice that “the runtime plane can function independently from the management plane”?

It means the runtime plane is engineered to keep serving already-deployed API traffic even if its connection to the Google-managed management plane is lost, because the Message Processor reads deployed configuration from its local Cassandra datastore rather than calling the control plane on the request path. What breaks during a management-plane disconnection is the ability to *change* things — new deployments can’t complete because the Synchronizer can’t pull new “contracts,” and new Developers/Apps/Products stop populating in the UI — but existing, already-synced traffic keeps flowing. This decoupled-failure-domain design is the architectural payoff of the pull-based Synchronizer model covered later in this section.

Q6. What’s the single diagnostic question that determines whether a client can realistically absorb Hybrid’s split responsibility model?

Ask them directly whether their platform-engineering team already runs production Kubernetes workloads at scale, with real on-call, patching, and incident-response discipline — because Hybrid does not remove Kubernetes operations, it merely gives you a pre-built application to run on top of your own cluster. If the honest answer is “we don’t have that muscle today,” recommending Hybrid without first building that muscle sets the client up to inherit exactly the kind of production incident this handbook’s Section 9 catalogs (Cassandra single-point-of-failure, missing CSI drivers, TLS secret drift) without the operational maturity to resolve them quickly.

Q7. What is MART, and what does it do?

MART (Management API for Runtime [Data]) is the runtime-plane service that lets the management plane reach into runtime-plane state that only exists inside the customer’s cluster — most notably Cassandra-backed data such as KVM reads/writes and certain runtime-side status queries. Apigee’s own historical release notes describe it in the context of secure connectivity: “Apigee Connect Alpha allows the Apigee hybrid MART service to connect to the management plane without requiring you to expose the MART endpoint” (Source: [Apigee hybrid 1.1.0 release notes](#), legacy Apigee domain, concept current). MART itself is stateless — it authenticates and validates requests before reading from or writing to Cassandra.

Q8. What happens to MART when Cassandra becomes unreachable?

`apigee-mart` pods enter `CrashLoopBackOff` together with `apigee-runtime` and `apigee-synchronizer`, because all three depend directly on a healthy Cassandra connection (Source: [No active runtime pods warning](#)). No

independently documented “MART alone is down” failure scenario exists in Google’s official troubleshooting playbooks — MART’s availability is, in the documented failure modes, effectively coupled to Cassandra’s, so diagnosing a MART problem in practice almost always means diagnosing the underlying Cassandra ring first.

Q9. What is the Synchronizer, and why is it described as a “pull model”?

The Synchronizer keeps the runtime plane’s local view of deployed configuration — proxy bundles, environment configuration, target servers, KVM metadata, collectively referred to as “contracts” — up to date with what the control plane’s Management API says should be deployed. It is a pull model because the Synchronizer, running inside the runtime plane, initiates outbound calls to the Management API and polls for changes; Google’s own troubleshooting documentation confirms the failure signature when this breaks — the Synchronizer “downloads API product configs from the control plane” and, if the network path is broken, “the Apigee runtime plane components will not be able to communicate with Apigee control plane APIs” (Source: [Troubleshooting playbook: no network connectivity](#)). The deliberate design goal is that the customer never has to open an inbound firewall port toward Google for this traffic.

Q10. What is the actual traffic impact when the Synchronizer loses connectivity to the control plane?

Already-deployed traffic keeps flowing unaffected — this is the single most important fact about Synchronizer failures and the one most commonly tested in interviews. What breaks is strictly the control path: new proxy deployments cannot complete (the Message Processor logs “no active contract is available for the message processor to serve the traffic” and fails its readiness probe, per [Troubleshooting playbook: message processor](#)), and API Products/Developers/Apps stop populating in the Apigee UI or fail on Management API reads/writes. Google does not publish an exact polling interval for the Synchronizer in customer-facing docs reviewed for this handbook — describe the mechanism as “polls the control plane on a recurring interval,” not with an invented number.

Q11. What data does Cassandra store in Apigee Hybrid, and where does that documentation come from?

Cassandra is the runtime plane’s persistent datastore for **Key Value Maps, response caches, and Quota counters** — confirmed directly: “runtime pods must connect to Cassandra to fetch resources like Key Value Maps and caches” (Source: [No active runtime pods warning](#)). It is governed by the `ApigeeDatastore` custom resource and materialized as a Kubernetes **StatefulSet** — the one and only Hybrid component deployed that way, because it is the only component that needs stable per-pod network identity and durable per-pod storage.

Q12. What replicaCount does Google recommend for Cassandra, and why is a single-replica Cassandra dangerous?

Google’s own guidance is to configure `cassandra.replicaCount: 3` for production. A single-replica Cassandra is an explicit single point of failure: if that one pod becomes unreachable, `apigee-runtime`, `apigee-mart`, and `apigee-synchronizer` pods all enter `CrashLoopBackOff` with the log signature “Cannot build a cluster without contact points,” deployments fail with a “No active runtime pods” UI warning, and Products/Apps/Developers pages surface datastore and internal-server errors (Source: [No active runtime pods warning](#)). Recovery is automatic once Cassandra is healthy again — no manual pod intervention is required — but preventing the outage in the first place is entirely about running at least 3 replicas.

Q13. Is Cassandra being deprecated in favor of Redis in Apigee Hybrid?

No — this is a common misconception worth actively correcting in an interview. Google’s shared-responsibility page explicitly lists Cassandra as a package it continues to deliver alongside the runtime and Kubernetes controllers, and dedicated, currently-versioned Cassandra backup and scaling documentation continues to be published. A separate `ApigeeRedis` custom resource has been added in recent Hybrid versions, but per a Google engineer’s public forum answer, it supplements Cassandra for one narrow purpose: “Quotas are stored in Cassandra with rate synchronized... on the Quota policies configuration” (long-term, contractual limits) while “Spike Arrest counts in memory on the Message Processors” and “will use Redis to coordinate counts when `UseEffectiveCount` is set” (short-term protection, coordinated across MP pods only for that one setting) — treat this specific division of labor as

community-corroborated rather than an official `cloud.google.com` quotation, and never characterize it as Redis replacing Cassandra.

Q14. What is the Message Processor’s role in Apigee Hybrid, and how does it relate to configuration delivered by the Synchronizer?

The Message Processor (`apigee-runtime`) executes proxy flows and policies against inbound requests and communicates with backend targets — functionally the API gateway itself, architecturally identical in role to Apigee X’s Message Processor. It is stateless and reads deployed configuration from Cassandra at request time, configuration that was placed there by the Synchronizer’s pull from the control plane — the Message Processor never talks to the control plane directly for this purpose. When healthy but no active deployment “contract” exists for an environment, the pod fails its readiness probe and is removed from traffic serving rather than serving broken responses.

Q15. What is the `runtime.release.strategy` setting, and what does it control?

Apigee hybrid v1.16’s release notes document a configurable `runtime.release.strategy` (`rolling / scale-down-first / none`) that governs how the `apigee-runtime` Deployment rolls pods during an upgrade (Source: [Apigee hybrid release notes](#)). This is directly relevant to production change management: `rolling` performs a gradual pod-by-pod replacement, while `scale-down-first` fully removes old pods before bringing up new ones — an architect choosing between these is trading off upgrade speed and resource headroom against the risk window during which capacity is temporarily reduced.

Q16. What kind of ingress does Apigee Hybrid use, and is it a plain Kubernetes Ingress object?

No — Apigee Hybrid’s ingress layer is **Istio-based** (Cloud Service Mesh), not a bare Kubernetes Ingress resource. Community-documented object names include `istio-ingressgateway` for general API traffic (exposed as a Service of type `LoadBalancer`) and `apigee-mart-istio-ingressgateway` for routing specifically to MART, the latter of which becomes unnecessary once Apigee Connect is enabled. Envoy is Istio’s data-plane proxy, and Google’s own security-patching documentation lists Envoy by name among the open-source projects it tracks for vulnerability disclosure — consistent with, though not an explicit official statement of, Envoy powering this ingress data plane.

Q17. What is the documented failure mode for Hybrid’s ingress layer, and how severe is it?

Missing or misconfigured TLS secrets for an environment group cause SSL handshake failures and connection resets for **all traffic through that ingress endpoint** — the official playbook shows `apigee-watcher` logging `NOT_FOUND: failed to get secret... in namespace apigee` in this exact scenario (Source: [Troubleshooting playbook: API calls failing with TLS errors](#)). This is one of the most severe documented failure modes precisely because, unlike a Synchronizer outage, it **is** traffic-affecting immediately — every client attempting to reach that hostname fails at the TLS layer before ever reaching a Message Processor.

Q18. What telemetry stack does Apigee Hybrid use, and what custom resource governs it?

“Apigee hybrid uses the industry-standard Prometheus add-on for metrics collection,” and this is governed by the `ApigeeTelemetry` custom resource. An OpenTelemetry Collector integration with HTTP proxy forwarding is also a documented, currently supported feature. Metrics ultimately ship to Cloud Monitoring; log shipping is commonly implemented via a Fluentd/Fluent Bit-based agent, a detail named in Google Cloud Community material rather than confirmed via a direct `cloud.google.com` sentence in this research pass — treat “Fluentd” as community-corroborated, not an official term to cite as a direct Google statement.

Q19. Does losing telemetry components take down traffic in Apigee Hybrid?

No — loss of telemetry components (Prometheus, the OpenTelemetry Collector, or log-shipping agents) degrades observability but is not documented as traffic-affecting in Google’s official troubleshooting playbooks. This is architecturally consistent with everything else in the resilience story: components on the *reporting* path (Telemetry, UDCA, Watcher) can fail without stopping request processing, whereas components on the *serving* path (Cassandra, the Message Processor, the Ingress Gateway) failing does stop or degrade traffic.

Q20. What is UDCA, and what does it forward?

UDCA (Universal Data Collection Agent) extracts analytics, distributed-trace, and deployment-status data from the runtime plane and forwards it to the management plane's analytics processing pipeline, making it visible in the same Apigee Analytics UI/API used by Apigee X. Its communication is outbound-only, from the runtime plane to the control plane, consistent with the Synchronizer's and Watcher's outbound-only design philosophy — no inbound connection from Google into the customer's cluster is ever required for this data to flow.

Q21. What happens to analytics data during an extended control-plane connectivity loss?

If the runtime plane loses connectivity to the control plane, analytics, debug, and deployment-status data queues locally rather than being lost outright, and is expected to catch up once connectivity returns — consistent with the broader “built for resilience, not isolation” narrative documented in community guidance and consistent with every officially-verified mechanism in this handbook (Source, community corroboration: [Apigee Hybrid off the Grid](#)). The practical consequence for an operator is that the Apigee UI will not show correct, real-time status for APIs deployed during the outage — a visibility gap, not a data-loss event.

Q22. What does apigee-watcher do, and what evidence proves it (not a separate cert-manager) reconciles ingress TLS state?

Watcher reports Message Processor and runtime status up to the management plane, and also reconciles ingress-related TLS secret state inside the cluster. The direct evidence for its role in TLS reconciliation is a documented official troubleshooting playbook showing apigee-watcher logging `NOT_FOUND: failed to get secret...` in namespace `apigee` when a required TLS secret for an environment group is missing (Source: [Troubleshooting playbook: API calls failing with TLS errors](#)) — this is what tells us Watcher, not a separate open-source cert-manager-style controller, owns this reconciliation in Hybrid. No official confirmation of a distinct cert-manager component exists in the sources reviewed; do not assert its presence or absence as settled fact beyond what this evidence supports.

Q23. What Kubernetes deployment type does apigee-watcher use, and at what scope?

apigee-watcher is governed by the `ApigeeOrganization` custom resource and materialized by the Apigee Operator as an `ApigeeDeployment` → standard Kubernetes Deployment, at the organization scope rather than per-environment (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)). Organization scoping makes sense given its role — status reporting and TLS-secret reconciliation are naturally organization-wide concerns rather than something that needs a separate instance per environment.

Q24. What does the Apigee Connect Agent do, and why does it matter for a customer's firewall posture?

The Apigee Connect Agent (`apigee-connect-agent`) establishes a secure, outbound-initiated tunnel that lets the management plane reach MART and runtime services without the customer exposing an inbound ingress endpoint for management traffic. This matters directly for firewall posture: historically, the management plane called *into* MART, requiring the customer to expose a MART ingress endpoint to Google, which Apigee Connect eliminates entirely — “Apigee Connect Alpha allows the Apigee hybrid MART service to connect to the management plane without requiring you to expose the MART endpoint” (Source: [Apigee hybrid 1.1.0 release notes](#), legacy Apigee domain, concept current).

Q25. What Kubernetes deployment type backs the Apigee Connect Agent, and what's unusual about it compared to the other components?

`apigee-connect-agent` is org-scoped and materialized by the Apigee Operator as an `ApigeeDeployment` → Deployment **plus a HorizontalPodAutoscaler, PodDisruptionBudget, and ReplicaSet** — the only component the official troubleshooting playbook shows explicitly paired with an HPA in its default configuration (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)). This makes it stand apart from most other stateless components in the documented mapping, which are shown simply as Deployments without an explicitly named default HPA.

Q26. Which Apigee Hybrid components are Deployments, and which is the one StatefulSet — and why?

Nearly every stateless component — `apigee-connect-agent`, `apigee-mart`, `apigee-watcher` (all `ApigeeOrganization`-scoped), and `apigee-runtime`, `apigee-synchronizer`, `apigee-udca` (all `ApigeeEnvironment`-scoped) — is materialized through Apigee’s own `ApigeeDeployment` custom resource into a standard Kubernetes Deployment. `apigee-cassandra`, governed by `ApigeeDatastore`, is the sole **StatefulSet** (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)). The reason is straightforward: a `StatefulSet` exists specifically for workloads that need a stable, sticky per-pod identity and durable per-pod storage, and Cassandra — as a ring-topology, replicated datastore — is the only component in the runtime plane with that requirement; everything else is stateless request-processing or reporting logic that can be freely rescheduled.

Q27. What custom resources does the Apigee Operator manage in Hybrid?

The documented set is **ApigeeOrganization**, **ApigeeEnvironment**, **ApigeeDatastore**, **ApigeeTelemetry**, and **ApigeeRedis** (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)). Each CR corresponds to a distinct scope of concern — organization-wide components (Connect Agent, MART, Watcher), environment-scoped components (Message Processor, Synchronizer, UDCA), the Cassandra datastore, the telemetry stack, and Redis’s narrow Spike-Arrest-coordination role — and the Operator’s job is to reconcile these high-level CRs down into the standard Kubernetes objects (Deployments, the one `StatefulSet`, Services, Secrets, ConfigMaps, HPAs) that actually run.

Q28. What is the default Kubernetes namespace for an Apigee Hybrid install, and can it be changed?

The default namespace is `apigee`, confirmed directly in Apigee’s install documentation: “The default namespace is ‘apigee,’ but users can specify alternatives using the `--namespace` flag or by manually updating resource definitions” (Source: [Apigee hybrid installation and administration guide](#)). Every component in the Section 3.2 component table runs inside this one namespace by default, which is convenient for a single-tenant install but is a consideration if the same cluster needs to host multiple isolated Apigee Hybrid installs or coexist with unrelated workloads under strict namespace-based RBAC.

Q29. How are TLS certificates and GCP service-account credentials stored and secured inside the Hybrid cluster?

TLS certificates for the Istio Ingress Gateway and for environment-group hostnames are held as Kubernetes Secrets, self-signed or CA-issued — Apigee’s install guide confirms “users can either generate self-signed certificates or use certificates from recognized authorities for istio ingress gateway communication” (Source: [Apigee hybrid installation and administration guide](#)). For GCP service-account credentials, two approaches are supported: “traditional Google Cloud service account keys stored as Kubernetes secrets, or workload identity for GKE clusters,” and for external secret management, “Vault integration allows storing service account keys in external secret management systems using CSI drivers” (same source).

Q30. What Kubernetes object gives Cassandra pods stable, addressable network identity within the cluster?

Cassandra runs as a `StatefulSet`, and `StatefulSets` are paired with a headless Service to give each pod a stable, predictable DNS identity that survives pod rescheduling — this is standard Kubernetes `StatefulSet` practice (not independently re-verified against Apigee-specific documentation in this research pass, so present it as the general mechanism `StatefulSets` use rather than a directly cited Apigee statement) and is precisely why Cassandra, alone among Hybrid’s components, needs a `StatefulSet` rather than a Deployment: a Cassandra ring depends on each node being addressable by a stable identity, not an ephemeral pod IP that changes on every restart.

Q31. Walk through the full failure cascade when Cassandra goes down in Apigee Hybrid.

`apigee-runtime`, `apigee-mart`, and `apigee-synchronizer` pods all enter `CrashLoopBackOff` simultaneously, with the log signature “Cannot build a cluster without contact points”; the Apigee UI shows a “No active runtime pods” warning, a “Datastore Error” on the Products/Apps pages, and an “Internal Server Error” on the Developers

page (Source: [No active runtime pods warning](#)). This is the most severe documented failure mode because it simultaneously takes down MART, the Synchronizer, and the Message Processor’s ability to both serve and deploy — Google’s preventive guidance is `cassandra.replicaCount: 3` (or higher, in multiples of 3), and recovery is automatic once Cassandra is healthy again, with no manual pod intervention required.

Q32. Walk through the failure impact when the Synchronizer loses control-plane connectivity, and contrast it with the Cassandra scenario.

Unlike the Cassandra scenario, **already-deployed traffic is unaffected** when the Synchronizer alone loses connectivity — new API Products, Developers, and Apps stop appearing in the UI, and new proxy deployments never complete, but existing traffic keeps flowing because the Message Processor reads its configuration from local Cassandra, not from a live call to the control plane. Root causes typically include missing firewall allow-listing for `apigee.googleapis.com`, a misconfigured forward proxy, or — for private GKE clusters — Private Google Access not being enabled (Source: [Troubleshooting playbook: no network connectivity](#)). The contrast with Cassandra is the whole point of this comparison: a control-path outage (Synchronizer) is low-to-medium severity because it doesn’t touch existing traffic; a data-path outage (Cassandra) is high severity because it does.

Q33. Is there a documented “MART alone is down” failure scenario independent of Cassandra?

No — Google’s official troubleshooting playbooks document MART’s failure as occurring *together with* the Cassandra-outage scenario, not as an independently documented standalone failure mode: `apigee-mart` pods enter `CrashLoopBackOff` alongside `apigee-runtime` and `apigee-synchronizer` specifically because all three depend on a healthy Cassandra connection (Source: [No active runtime pods warning](#)). If asked to diagnose “MART is failing” in isolation, the correct first move is to check Cassandra’s health before looking anywhere else, since no other documented independent MART failure path exists in the sources reviewed.

Q34. Walk through the failure impact when ingress TLS secrets are missing or misconfigured.

Clients attempting to reach the affected environment group’s hostname experience SSL handshake failures and connection resets — this **is** traffic-affecting immediately, in contrast to a Synchronizer outage. The root cause is a missing or misconfigured Kubernetes Secret holding the TLS certificate/key for that environment group, and the diagnostic signature is `apigee-watcher` logging `NOT_FOUND: failed to get secret...` in namespace `apigee` (Source: [Troubleshooting playbook: API calls failing with TLS errors](#)). Recovery is to recreate or correct the Secret; `apigee-watcher` reconciles the ingress gateway configuration once a valid secret is present, with no need to restart the gateway manually in the documented scenario.

Q35. What happens when the Message Processor itself fails its readiness probe, independent of Cassandra or Synchronizer issues?

Most commonly, a Message Processor readiness failure is a *downstream* symptom rather than a primary failure of its own — either Cassandra is unreachable, or the Synchronizer has never delivered an active “contract” for that environment, logged as “no active contract is available for the message processor to serve the traffic” (Source: [Troubleshooting playbook: message processor](#)). Because `apigee-runtime` is a standard Kubernetes Deployment, failed pods restart automatically, but that restart won’t resolve the underlying issue if it’s a Cassandra or Synchronizer problem — the pod will simply cycle back into the same failed readiness state until the root cause is fixed.

Q36. Rank the five documented Hybrid failure scenarios by severity and explain the ranking.

Cassandra failure is the most severe: it cascades into MART, Synchronizer, and Message Processor `CrashLoopBackOff` simultaneously and is the one scenario documented as affecting existing traffic capacity directly. Ingress/TLS failure is comparably severe but more narrowly scoped — it’s immediately traffic-affecting, but only for the specific environment group whose TLS secret is broken, not the whole cluster. Message Processor readiness failures sit in the medium-to-high range, scoped to the affected environment, and are usually a symptom of one of the two failures above rather than a root cause on their own. MART failure, as documented, is coupled entirely to Cassandra’s health

and inherits its severity only when co-occurring. Synchronizer/control-plane-connectivity failure is the least severe of the five specifically because Google’s documented behavior confirms already-deployed traffic is unaffected — only the ability to change configuration is lost.

Q37. What is the “resilience, not isolation” narrative, and why does it matter in a client-facing design conversation?

It’s the summary framing — corroborated by community guidance and consistent with every officially-verified failure mode in this handbook — that “Apigee Hybrid is built for resilience, not isolation”: existing configuration keeps working because it’s cached locally in Cassandra, and even if pods restart, “the data is pulled from cassandra to resume runtime operations” (Source, community corroboration: [Apigee Hybrid off the Grid](#)). It matters in a client conversation because it draws the correct boundary around what “hybrid resilience” actually promises — surviving a *temporary* control-plane disconnection without dropping existing traffic — versus what it does not promise, which is indefinite offline operation; extended outages still impair the ability to deploy new or modified proxies, and that distinction should be stated plainly rather than oversold.

Q38. Give three concrete Google responsibilities and three concrete customer responsibilities from the shared-responsibility model.

Google’s documented responsibilities include delivering the runtime/MART/Cassandra/Kubernetes-controller packages, continually scanning components for vulnerabilities and patching known issues, and providing upgrade notifications with documented steps (Source: [Apigee hybrid shared responsibility model](#)). The customer’s documented responsibilities include provisioning the underlying compute/storage/network infrastructure, keeping the Kubernetes platform and its infrastructure patched and up to date, and owning multi-region deployment and disaster recovery entirely (same source). The pattern across both lists is consistent: Google owns “build and ship the software correctly”; the customer owns “run it, on infrastructure they provision, patch, and secure.”

Q39. Who is responsible for patching the Kubernetes nodes underneath an Apigee Hybrid cluster?

The customer, unambiguously — “keep the Kubernetes platform and underlying infrastructure up to date with latest software updates and patches” is an explicit customer duty per the shared-responsibility model (Source: [Apigee hybrid shared responsibility model](#)). Google’s patching responsibility is scoped to its own container images and components (“continually scan components with the Container Analysis API and patch known vulnerabilities”), but pulling and deploying those patched images, and keeping the node OS, container runtime, and cluster control plane current, is the customer’s job.

Q40. Who owns TLS certificate lifecycle for northbound API traffic in Apigee Hybrid?

The customer — “load balancer provisioning/management fronting APIs, including TLS certificate lifecycle” is an explicit customer responsibility (Source: [Apigee hybrid shared responsibility model](#)). In practice this means the customer provisions and rotates the certificates stored as Kubernetes Secrets for the Istio Ingress Gateway and for each environment group’s hostnames, and — as covered in the failure-scenario questions above — a lapse in that lifecycle management is a documented, immediately traffic-affecting failure mode, not a theoretical risk.

Q41. Who is responsible for multi-region deployment and disaster-recovery design in Apigee Hybrid?

Entirely the customer — “multi-region deployment and disaster recovery” is listed as an explicit customer responsibility (Source: [Apigee hybrid shared responsibility model](#)), and unlike Apigee X, Hybrid has no published SLA at all, so there is no contractual floor Google is committing to underneath whatever the customer designs. This is the sharpest practical contrast with Apigee X: for X, the customer decides *whether* to add regions on top of Google’s live/live control-plane architecture; for Hybrid, the customer designs and builds the entire multi-cluster, multi-datacenter-Cassandra, failover topology from first principles.

Q42. What is Google’s documented production baseline for how many Kubernetes clusters a Hybrid deployment should run, and how should they be sized relative to each other?

Google’s product engineering team states this directly and unambiguously: “Your production platform should have at least two Kubernetes clusters, each in a different region,” and for DR sizing parity, “we recommend that they are the same size. That way, one can handle the full load if the other cluster fails” (Source: [Optimize managing APIs at scale across hybrid environments](#)). This establishes a **one-cluster-per-region** model as the documented production baseline, not an optional enhancement — a single-cluster Hybrid deployment should be understood and presented to a client as a deliberate, accepted regional-outage risk, not as a mistake, but the risk should be named explicitly.

Q43. How does Cassandra scale across multiple regions/clusters in a multi-region Hybrid deployment?

Cassandra is configured for multi-datacenter replication across the clusters — a standard Cassandra ring-of-rings topology where each regional cluster runs its own local Cassandra data center, replicated to the others. The exact `dc_name/seed-host` configuration mechanics live in versioned Apigee documentation that this handbook could not independently re-verify in this research pass — consult the current [Multi-region deployment](#) page directly for the version-correct procedure rather than relying on a generic description of Cassandra multi-DC replication alone. What is independently verified is the scaling constraint that applies at every region: Cassandra “does not have the ability to autoscale” and “must be done on-demand and in sets of three” (Source: [Optimize managing APIs at scale across hybrid environments](#)).

Q44. Does Apigee Hybrid have a published SLA?

No. The Apigee SLA’s “Covered Service” definition explicitly states it means “Apigee Standard, Apigee Pay-as-you-go, Apigee Enterprise, or Apigee Enterprise Plus, but excluding API calls processed by Apigee hybrid or Apigee Adapter for Envoy” (Source: [Apigee SLA](#)). This is a fact worth stating plainly and without hedging in any client conversation: whatever uptime a Hybrid deployment achieves is purely a function of how well the customer designs and operates their own cluster topology — there is no Google-backed contractual floor underneath it at all.

Q45. Given that Google publishes no SLA or RTO/RPO for Hybrid, how would you architect DR for it from first principles?

You would apply the same live/live philosophy Google uses for its own control plane, but build it yourself: at least two same-sized regional Kubernetes clusters (Google’s own documented baseline), each running the full component stack, with Cassandra configured for multi-datacenter replication across them, HPA-scaled Message Processors, and a customer-managed global load-balancing layer in front of the clusters’ ingress gateways to route around a failed region. Layer CI/CD and GitOps automation on top specifically because Google’s own guidance recommends it “to make adding additional regions easy” and “to simplify recreating clusters during disasters” (Source: [Optimize managing APIs at scale across hybrid environments](#)) — without that automation, “recreate the cluster” during an actual disaster becomes a multi-day manual exercise rather than a rehearsed, repeatable procedure.

Q46. What Kubernetes platforms are confirmed supported for Apigee Hybrid, and what’s the EKS-specific gotcha?

GKE is the primary, most heavily documented target. EKS (Amazon) is confirmed supported, with a documented EKS-specific dependency: Cassandra pods can get stuck in a Pending state due to a “Missing Amazon EBS CSI driver (for EKS installations)” (Source: [Troubleshooting playbook: Cassandra](#)). AKS (Azure) is confirmed supported per Google-staff community guidance. This EBS CSI driver dependency is a real, documented operational gotcha worth calling out specifically in any EKS-targeted Hybrid design review, not a theoretical edge case.

Q47. Is Apigee Hybrid supported on Anthos on-premises or bare-metal deployments?

Explicitly not, per Google-staff community guidance: Hybrid is supported in some Anthos configurations, but “Anthos attached clusters with arbitrary k8s distributions” and “Anthos on-prem, on bare metal” are named as unsupported (community-sourced corroboration, not a primary `cloud.google.com` citation). OpenShift support is not

independently confirmed either way in the sources reviewed for this handbook. Given how frequently the supported-platforms matrix changes with each Hybrid release, always verify current compatibility against the live [Apigee hybrid supported platforms](#) page rather than relying on any point-in-time list, including this one.

Q48. Walk through the full deployment flow in Apigee Hybrid, from trigger to traffic serving.

The same trigger mechanisms as Apigee X — UI, `apigeectl`, Maven/Gradle plugins, `gcloud`, or a direct REST call — invoke the same Management API deployment operation, and the control plane records the new deployment state exactly as it does for X. The fork happens next: in Hybrid, the customer-run **Synchronizer**, inside the runtime plane, must actively pull the new deployment state across the control-plane/runtime-plane boundary — it polls the Management API, receives the updated “contract,” and writes the resolved configuration into the runtime plane’s local **Cassandra**. Only then do **Message Processor** (`apigee-runtime`) pods pick up the new deployment from Cassandra and begin routing traffic, fronted by the **Istio Ingress Gateway** that directs incoming requests to the correct pod — meaning there is a real, if usually brief, propagation delay in Hybrid that simply doesn’t exist in Apigee X’s fully Google-internal propagation path.

Q49. What is `apigeectl`, and how does the newer Helm-based install path differ from it?

`apigeectl` is the traditional CLI/templating tool that renders and applies Kubernetes manifests from an `overrides.yaml` configuration file, historically the primary way to install and upgrade a Hybrid runtime plane. Recent Hybrid versions (v1.15/v1.16) also support Helm charts for install and upgrade — confirmed via a real v1.16.0-hotfix.1 release note describing a Helm-based upgrade procedure that references components named `ao` (Apigee Operator) and `mart` (`mart-server`) (Source: [Apigee hybrid release notes](#)). Architecturally, the shift toward Helm aligns Hybrid’s install/upgrade tooling with the broader Kubernetes ecosystem’s dominant packaging convention, giving customers who already standardize on Helm for other workloads a more consistent operational story rather than a bespoke, Apigee-only tool.

Q50. Why would an enterprise choose Apigee Hybrid over Apigee X purely for regulatory reasons, and what does that choice cost them operationally?

The core regulatory driver is data residency and sovereignty: Hybrid’s entire value proposition is that “all API traffic is processed within the boundaries of your network and under your control” (Source: [What is Apigee hybrid?](#)), which matters decisively when a regulator or contract mandates that API payloads never leave a specific network boundary — on-premises, a sovereign cloud region, or an air-gapped environment — something Apigee X’s Google-tenant-project processing model cannot satisfy no matter how the networking is configured. The operational cost is real and should never be understated to a client: the organization inherits full Kubernetes operations (provisioning, patching, scaling, incident response, capacity planning) on top of the same proxy/policy/product operations Apigee X customers already do, plus the loss of any published SLA — the price of data-boundary control is Kubernetes ownership, not just a different Apigee license line item.

11.3 Top 30 Kubernetes Questions Related to Apigee Hybrid

Q1. What is a Kubernetes Cluster and Node, and what are Google’s documented minimum sizing recommendations for Apigee Hybrid?

A Kubernetes cluster is the set of machines (nodes) that run containerized workloads, coordinated by a control plane; in Apigee Hybrid, the entire cluster is the customer’s responsibility to provision and patch. Google’s product-team guidance gives concrete node-level sizing floors for production: Cassandra nodes need “a minimum of 8 vCPUs and 15Gi of RAM for production workloads,” while runtime (Message Processor) pods are recommended at “2 vCPUs and 2 Gi of RAM” each (Source: [Optimize managing APIs at scale across hybrid environments](#)). Google is explicit that no universal sizing formula exists — “there is no one optimal infrastructure size for all clusters running Apigee hybrid” — and recommends load-testing rather than trusting static estimates.

Q2. What is a Pod, and which Apigee Hybrid components run as pods?

A Pod is the smallest deployable unit in Kubernetes — one or more containers sharing network and storage. Every Apigee Hybrid runtime-plane component — the Message Processor, Synchronizer, MART, UDCA, Watcher, Connect Agent, and Cassandra — ultimately runs as one or more pods; what differs between them is the controller that manages those pods (a Deployment for the stateless components, a StatefulSet for Cassandra alone), which is the more architecturally interesting distinction to draw out in an answer than the pod concept itself.

Q3. What is a Deployment, and which Hybrid components use it?

“A Deployment manages a set of Pods to run an application workload, usually one that doesn’t maintain state” (Source: [Kubernetes documentation — Deployments](#)). This is the controller behind every stateless Apigee Hybrid component — `apigee-runtime`, `apigee-synchronizer`, `apigee-mart`, `apigee-udca`, `apigee-watcher`, and `apigee-connect-agent` — each instantiated through Apigee’s own `ApigeeDeployment` custom resource, which the Apigee Operator materializes into a standard Kubernetes Deployment (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)).

Q4. What is a StatefulSet, and why is Cassandra the only Hybrid component that uses one?

“A StatefulSet runs a group of Pods, and maintains a sticky identity for each of those Pods. This is useful for managing applications that need persistent storage or a stable, unique network identity” (Source: [Kubernetes documentation — StatefulSets](#)). Cassandra is governed by the `ApigeeDatastore` custom resource and is the sole component deployed as a StatefulSet because it’s the only genuinely stateful runtime-plane component — a Cassandra ring needs each node to keep a stable, predictable identity and durable per-pod storage across restarts, which is exactly what a StatefulSet (paired with PersistentVolumes) provides and a Deployment does not.

Q5. What is a Service, and how do Hybrid components address each other internally?

“Expose an application running in your cluster behind a single outward-facing endpoint, even when the workload is split across multiple backends” (Source: [Kubernetes documentation — Service](#)). Every Deployment and the Cassandra StatefulSet sit behind a Service, so that, for example, the Synchronizer and MART can address Cassandra by a stable in-cluster name regardless of which specific Cassandra pods are currently healthy or being rescheduled — this is precisely what lets Kubernetes restart or reschedule individual pods without every dependent component needing to track raw pod IPs.

Q6. What is a Kubernetes Ingress, and why does Apigee Hybrid use an Istio-based gateway instead of a bare Ingress object?

“Make your HTTP (or HTTPS) network service available using a protocol-aware configuration mechanism... lets you map traffic to different backends based on rules you define via the Kubernetes API” (Source: [Kubernetes documentation — Ingress](#)). Apigee Hybrid’s ingress layer is Istio-based (Cloud Service Mesh) rather than a plain Kubernetes Ingress resource, because API-gateway-grade traffic management — the environment-group hostname routing, TLS termination, and policy execution handoff to Message Processor pods that Apigee needs — requires the richer traffic-management and mTLS capability a service-mesh data plane (Envoy, in Istio’s case) provides beyond what a bare Ingress controller offers.

Q7. What is a ConfigMap, and what does Apigee Hybrid store in one?

“A ConfigMap is an API object used to store non-confidential data in key-value pairs... allows you to decouple environment-specific configuration from your container images” (Source: [Kubernetes documentation — ConfigMaps](#)). Apigee Hybrid uses ConfigMaps for non-sensitive per-component configuration, including forward-proxy settings for individual components, documented as configurable “in the `apigee-hybrid-config.yaml` file” (Source: [Apigee hybrid installation and administration guide](#)) — the general architectural point being that anything non-secret and environment-specific belongs in a ConfigMap rather than baked into a container image or, worse, into a Secret.

Q8. What is a Kubernetes Secret, and what two categories of sensitive material does Hybrid store this way?

“A Secret is an object that contains a small amount of sensitive data such as a password, a token, or a key... Using a Secret means that you don’t need to include confidential data in your application code” (Source: [Kubernetes documentation — Secrets](#)). Apigee Hybrid uses Secrets for Google Cloud service-account credentials (either “traditional Google Cloud service account keys stored as Kubernetes secrets, or workload identity for GKE clusters,” with Vault/CSI-driver integration for external secret management) and for TLS certificates fronting the Istio Ingress Gateway and environment-group hostnames (Source: [Apigee hybrid installation and administration guide](#)).

Q9. What is a PersistentVolume, and why does Cassandra specifically need one?

“A PersistentVolume (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes... has a lifecycle independent of any individual Pod that uses the PV” (Source: [Kubernetes documentation — Persistent Volumes](#)). Cassandra is the component that requires PersistentVolumes — each Cassandra StatefulSet pod is backed by its own PV so that data survives pod rescheduling, which is also why the cloud-provider CSI driver matters operationally: on EKS specifically, Cassandra pods can get stuck Pending due to a “Missing Amazon EBS CSI driver” (Source: [Troubleshooting playbook: Cassandra](#)) — a real, documented gotcha, not a theoretical concern.

Q10. What is a HorizontalPodAutoscaler, and which Hybrid component explicitly cannot use one the way the Message Processor does?

“A HorizontalPodAutoscaler automatically updates a workload resource... with the aim of automatically scaling capacity to match demand” (Source: [Kubernetes documentation — HorizontalPodAutoscaler](#)). Runtime (apigee-runtime) pods autoscale via Kubernetes’ HPA by default, but Cassandra explicitly does not: “Cassandra does not have the ability to autoscale. Cassandra can easily be scaled up and down, but it must be done on-demand and in sets of three” (Source: [Optimize managing APIs at scale across hybrid environments](#)) — a direct consequence of its ring topology and default replication factor of 3, which requires scaling in multiples of 3 to keep the ring balanced.

Q11. What is a Kubernetes Namespace, and what is Apigee Hybrid’s default?

“Namespaces provide a mechanism for isolating groups of resources within a single cluster” (Source: [Kubernetes documentation — Namespaces](#)). Apigee Hybrid’s default install target is the single namespace apigee — “the default namespace is ‘apigee,’ but users can specify alternatives using the --namespace flag or by manually updating resource definitions” (Source: [Apigee hybrid installation and administration guide](#)) — and every component in the runtime-plane component table runs inside this one namespace by default.

Q12. What role does kubectl play in day-to-day Apigee Hybrid operations?

kubectl is the standard command-line tool for communicating directly with a Kubernetes cluster’s control plane (Source: [Kubernetes documentation — kubectl](#)), and for Apigee Hybrid it is the primary manual troubleshooting instrument — inspecting pod status and logs (kubectl get pods -n apigee, kubectl logs), checking Secret and ConfigMap contents, and confirming Deployment/StatefulSet health when diagnosing any of the failure scenarios in Section 9. apigeectl and Helm operate at a higher level (rendering and applying Apigee-specific manifests from configuration), but once those manifests are applied, kubectl is the tool you actually reach for to see what’s really happening inside the cluster.

Q13. What does the ApigeeOrganization custom resource govern?

ApigeeOrganization is the organization-scoped custom resource that governs apigee-connect-agent, apigee-mart, and apigee-watcher — the three components whose concerns are naturally organization-wide rather than tied to a specific environment (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)). It is the top-level CR the Apigee Operator reconciles first when standing up a Hybrid runtime plane, establishing the organization-level identity and configuration that the environment-scoped CRs then build on top of.

Q14. What does the `ApigeeEnvironment` custom resource govern?

`ApigeeEnvironment` is the environment-scoped custom resource that governs `apigee-runtime` (the Message Processor), `apigee-synchronizer`, and `apigee-udca` — the three components whose behavior is specific to a single Apigee environment (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)). A Hybrid installation typically has one `ApigeeEnvironment` resource per Apigee environment deployed to that cluster, which is why troubleshooting a single-environment problem (a Synchronizer failing to pull contracts for prod specifically, for example) starts by inspecting that environment’s own `ApigeeEnvironment`-managed pods rather than the organization-wide ones.

Q15. What does the `ApigeeDatastore` custom resource govern?

`ApigeeDatastore` governs `apigee-cassandra`, the single `StatefulSet` in the runtime plane (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)). It is the CR an architect edits to change `replicaCount` (in multiples of 3, per Google’s documented scaling constraint), storage sizing, and resource requests/limits for the Cassandra ring — and, because Cassandra failures are the most severe documented failure mode in Hybrid, `ApigeeDatastore`’s configuration deserves more production scrutiny than almost any other single CR in the install.

Q16. What does the `ApigeeTelemetry` custom resource govern?

`ApigeeTelemetry` governs the observability stack — the Prometheus add-on for metrics collection, and the OpenTelemetry Collector integration where enabled. Google’s documentation states directly that “Apigee hybrid uses the industry-standard Prometheus add-on for metrics collection,” with `ApigeeTelemetry` as the governing custom resource confirmed in Google’s documented CR list (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)). Because telemetry failures are not documented as traffic-affecting, misconfiguration here tends to surface as an observability blind spot rather than an outage — which makes it easy to under-prioritize in a review, a mistake worth calling out explicitly.

Q17. What does the `ApigeeRedis` custom resource govern, and is it a replacement for Cassandra?

`ApigeeRedis` is a more recently introduced custom resource with a narrow, specific purpose: per a Google engineer’s public forum answer, it is used so that Spike Arrest “will use Redis to coordinate counts when `UseEffectiveCount` is set” — coordinating in-memory Spike Arrest counters across Message Processor pods for that one specific setting, while Cassandra continues to store Quota counters and rate configuration (community-corroborated, not a direct `cloud.google.com` quotation). It is not a replacement for Cassandra in any sense — Cassandra remains the primary, currently-shipped, currently-documented datastore for KVMs, response caches, and Quota counters, and `ApigeeRedis` supplements it for this one coordination use case only.

Q18. What does `apigeectl` actually do under the hood?

`apigeectl` is a CLI/templating tool that reads the customer’s `overrides.yaml` configuration file, renders the corresponding Kubernetes manifests (or, in newer versions, Helm chart values), and applies them against the target cluster — effectively the tool that turns a customer’s declared configuration (Cassandra replica count, runtime autoscaling bounds, connect-agent settings, ingress gateway definitions) into the actual Kubernetes objects the Apigee Operator and its custom resources then reconcile. It is the traditional, longest-standing install/upgrade path for Hybrid, predating the more recently added Helm-based alternative.

Q19. What changed with the introduction of Helm support for Apigee Hybrid installs?

Recent Hybrid versions (v1.15/v1.16) added Helm chart support for install and upgrade as an alternative to the traditional `apigeectl` path — confirmed via a real v1.16.0-hotfix.1 release note describing a Helm-based upgrade procedure that references components named `ao` (Apigee Operator) and `mart` (`mart-server`) (Source: [Apigee hybrid release notes](#)). Architecturally, this doesn’t change what gets deployed — the same custom resources and underlying Kubernetes objects result either way — but it gives customers whose organizations have already standardized their CI/CD and GitOps tooling around Helm a consistent installation and upgrade experience rather than a bespoke, Apigee-specific tool.

Q20. What is the ApigeeDeployment custom resource, and how does the Apigee Operator use it?

ApigeeDeployment is Apigee’s own custom resource that wraps every stateless runtime-plane component — the Apigee Operator materializes it into a standard Kubernetes Deployment (and, for apigee-connect-agent specifically, additionally a HorizontalPodAutoscaler, PodDisruptionBudget, and ReplicaSet) (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)). This is Apigee’s own abstraction layered on top of the standard Kubernetes Deployment concept — the Operator pattern lets Apigee expose a simpler, Apigee-domain-specific configuration surface (replica counts, resource requests tied to Apigee semantics) while still producing ordinary, inspectable Kubernetes objects underneath that `kubectl` and any standard Kubernetes tooling can see directly.

Q21. What are Google’s documented minimum production sizing numbers for Cassandra nodes and runtime pods?

Google’s product-team guidance states Cassandra nodes need “a minimum of 8 vCPUs and 15Gi of RAM for production workloads,” while runtime (Message Processor) pods are recommended at “2 vCPUs and 2 Gi of RAM” each (Source: [Optimize managing APIs at scale across hybrid environments](#)). These are documented floors for production, not targets to size exactly to — Google’s own guidance is explicit that they should be treated as a starting point validated by load testing, not a formula that guarantees correct sizing for every workload profile.

Q22. Why must Cassandra be scaled in multiples of three rather than one node at a time?

Because Cassandra’s ring topology uses a default replication factor of 3, meaning each piece of data is stored on three nodes; adding or removing nodes in multiples of 3 keeps the ring balanced across replicas and avoids uneven data distribution or replica-count mismatches across the ring. Google states this constraint directly: Cassandra “does not have the ability to autoscale. Cassandra can easily be scaled up and down, but it must be done on-demand and in sets of three” (Source: [Optimize managing APIs at scale across hybrid environments](#)) — this is precisely why Cassandra cannot simply be attached to a HorizontalPodAutoscaler the way the Message Processor is.

Q23. Why does Google say there’s no single optimal cluster size for Apigee Hybrid?

Because the right size genuinely depends on workload-specific variables Google can’t standardize away — transactions per second, number of environments, policy complexity per proxy, and payload size all materially affect how much CPU/memory a given cluster actually needs. Google’s own guidance states this directly: “there is no one optimal infrastructure size for all clusters running Apigee hybrid, but there are some easy ways to figure out the right size for your organization,” recommending “consulting a trusted partner like your Google Customer Engineer” for capacity estimation (Source: [Optimize managing APIs at scale across hybrid environments](#)).

Q24. How would you validate a sizing estimate for an Apigee Hybrid cluster before going to production?

Google’s own guidance is explicit on this point: “never rely just on the estimates you used when you set up the cluster capacity” (Source: [Optimize managing APIs at scale across hybrid environments](#)) — load testing against representative traffic patterns (realistic TPS, realistic policy complexity, realistic payload sizes) before go-live is the documented recommendation, not a substitute for the documented node-level floors but a check that the *number* of nodes and pods at those floors is actually sufficient for the workload’s real behavior. In practice this means running a load test that specifically exercises Cassandra under production-like KVM/quota/cache access patterns, not just hammering the Message Processor’s request path in isolation.

Q25. How would you diagnose a CrashLoopBackOff on apigee-runtime pods?

Start by checking `kubectl logs` for the failing pod — the two most common documented signatures are “Cannot build a cluster without contact points” (Cassandra unreachable) or “no active contract is available for the message processor to serve the traffic” (no valid Synchronizer-delivered deployment) (Sources: [No active runtime pods warning](#); [Troubleshooting playbook: message processor](#)). If it’s the Cassandra signature, pivot immediately to checking Cassandra pod health and `replicaCount`; if it’s the “no active contract” signature, check Synchronizer pod health and its connectivity to the Management API next — the runtime pod itself is almost never the actual root cause.

Q26. How would you diagnose a “No active runtime pods” warning in the Apigee UI?

This warning, combined with a “Datastore Error” on Products/Apps pages and an “Internal Server Error” on the Developers page, is the documented signature of a Cassandra outage (Source: [No active runtime pods warning](#)). The diagnostic path is to check the Cassandra StatefulSet’s pod status directly with `kubectl get pods` in the `apigee` namespace, look for `CrashLoopBackOff` on `apigee-cassandra` pods specifically, and check for the two most common underlying causes: a single-replica Cassandra configuration acting as a documented single point of failure, or insufficient disk space (there’s a dedicated official playbook for exactly this scenario) (Source: [Troubleshooting playbook: Cassandra](#)).

Q27. How would you diagnose SSL handshake failures against a specific environment group’s hostname?

Check for a `NOT_FOUND: failed to get secret...` in namespace `apigee` entry in the `apigee-watcher` pod’s logs — this is the exact documented signature Google’s own troubleshooting playbook shows for a missing or misconfigured TLS secret for that environment group (Source: [Troubleshooting playbook: API calls failing with TLS errors](#)). The fix is to recreate or correct the Kubernetes Secret holding that environment group’s TLS certificate and key; `apigee-watcher` reconciles the ingress gateway configuration automatically once a valid secret is present, with no need to manually restart the Istio Ingress Gateway itself in the documented scenario.

Q28. How would you diagnose Synchronizer connectivity failures on a private GKE cluster specifically?

Given the documented failure statement that “if no network route is available to the internet, the Apigee runtime plane components will not be able to communicate with Apigee control plane APIs” (Source: [Troubleshooting playbook: no network connectivity](#)), the GKE-private-cluster-specific check is whether **Private Google Access** is enabled on the cluster’s subnet — the same source states “Private Google Access needs to be enabled to allow the Apigee runtime plane components access to Google APIs internally.” Beyond that, check firewall allow-listing for `apigee.googleapis.com` and any forward-proxy configuration the cluster is routed through, since a misconfigured forward proxy is one of the other commonly documented root causes for this exact symptom.

Q29. How would you diagnose Cassandra pods stuck in a Pending state on EKS specifically?

This is a documented, EKS-specific dependency issue: check whether the Amazon EBS CSI driver is installed on the cluster, since Google’s own troubleshooting playbook names “Missing Amazon EBS CSI driver (for EKS installations)” as the cause of Cassandra pods getting stuck Pending (Source: [Troubleshooting playbook: Cassandra](#)). The underlying reason is that Cassandra’s `PersistentVolumeClaims` can’t be dynamically provisioned without a working CSI driver for the cloud provider’s block storage, so `kubectl describe pod` on the affected Cassandra pod should show an `unbound-PVC` event pointing directly at this root cause once you know to look for it.

Q30. What’s the general troubleshooting mental model an architect should apply across all of these Apigee Hybrid failure scenarios?

Always separate the question “is the *management/control path* broken, or is the *runtime/data path* broken?” before diving into logs — because the two have fundamentally different severity profiles: a control-path problem (Synchronizer/connectivity) blocks new deployments and UI population but leaves existing traffic untouched, while a data-path problem (Cassandra, ingress/TLS) is directly traffic-affecting. Once you know which side of that split you’re on, the specific diagnostic path narrows quickly: `kubectl logs` and `kubectl get pods -n apigee` for the specific component named in the symptom (Cassandra, Synchronizer, Watcher, or the Message Processor itself), cross-referenced against the exact log signatures documented in Google’s official troubleshooting playbooks rather than guessing — every failure mode covered in this section has a named, quoted log signature precisely so this pattern-matching step is fast and unambiguous.

Section 12 — Study Notes, Memory Maps, and Cheat Sheets

This section is a dense, revision-oriented compression of Sections 1–10. It repeats no new facts that were not already established and cited earlier in the handbook — it exists purely to let you cram, self-test, and recall under pressure. Where a specific number appears, it is repeated from its original citation; where the research base does not publish an exact number, that gap is called out explicitly rather than filled in.

12.1 Apigee X Cheat Sheet

Resource hierarchy (top to bottom):

- **Organization** — 1:1 with a Google Cloud project; the container for all proxies, products, apps, developers (Source: [Understanding commonly used Apigee terms](#)).
- **Instance** — the unit of regional runtime capacity. **Exactly 1 instance per region per organization, hard limit.** Up to 10 instances/regions per organization (11 for Hybrid) (Source: [Apigee Limits reference](#)).
- **Environment** — a runtime execution context (“test,” “prod”); must be attached to an Instance in a region to serve traffic there.
- **Environment Group** — owns hostnames; routes by Host header to member Environments, then by base path to a deployed proxy revision.
- **API Proxy** → **Revision** — proxies are versioned as sequentially numbered revisions; only one revision per base path can be deployed in an environment at a time.
- **API Product** — bundles proxy resources + a quota/rate plan; what a Developer App actually subscribes to.

Runtime composition: the Message Processor (apigee-runtime) is the sole named runtime component Google discloses — “a set of containerized runtime services in a Kubernetes cluster that Google maintains” (Source: [What is Apigee?](#)). Runs inside a Google-managed, single-tenant **tenant project**, 1:1 with the Apigee organization. Google does **not** disclose whether it’s GKE, the autoscaling thresholds, or the internal topology — this is a deliberate abstraction boundary, not a documentation gap.

Networking — northbound (ingress):

- External clients → External Application Load Balancer (TLS termination) → Cloud Armor (WAF) / reCAPTCHA Enterprise (WAAP pattern) → Apigee, peered into a tenant project via VPC Network Peering by default.
- PSC is also documented as an alternative northbound mechanism to a MIG-based LB target.

Networking — southbound (egress):

- GKE/Compute Engine backends: RFC 1918 internal IPs over VPC Network Peering.
- Serverless/internal services in another VPC: Private Service Connect (Endpoint Attachment → Service Attachment → internal L7 LB, same region required).
- On-premises/data center: Cloud Interconnect.
- Public third-party APIs: Cloud NAT egress (default) from the tenant project.

Networking — the peering-hop problem: VPC Peering is non-transitive; only one hop is allowed and Apigee X already consumes it to reach the tenant project. Scale-out patterns: hub-and-spoke over Cloud VPN (VPN is transitive), or a Shared VPC hub design (“Shared VPC subnets do not violate the VPC peering transitivity limit”) — described by Google’s own engineers as a “cascading hub-and-spoke architecture” (Source: [Apigee X design for multiple distinct VPC connections](#)).

Key limits (Source: [Apigee Limits reference](#)):

Resource	Limit
Instances per region	1
Instances per organization	10 (11 for Hybrid)
Regions per organization	Max 10 (11 for Hybrid)
Environments per organization	Up to 85 (contract-dependent)
Environment groups per organization	Up to 85
Environment group attachments	100
Hostnames per environment group	100
API base paths	6,000
Key stores per environment	100
Developer portals	10
Apigee Spaces	20
Deployed proxies per environment	6,000
Deployed shared flows per environment	75
Deployments per instance	6,000

SLA tiers (Source: [Apigee SLA](#)):

- **99.99%** monthly uptime — environments in 2+ regions **with** a dual-region, multi-regional, or global Cloud KMS encryption key. Region count *alone* does not qualify — the KMS key topology is a contractual condition too.
- **99.9%** monthly uptime — all other (single-region) environments.
- **99.5%** carve-out for the Mexico and Stockholm regions specifically.
- The SLA **explicitly excludes** “API calls processed by Apigee hybrid or Apigee Adapter for Envoy” — Hybrid has no published SLA at all.

8 facts most likely to be forgotten under interview pressure:

1. Exactly **one Instance per region per organization** — there is no such thing as adding a second instance in the same region for more capacity; all in-region scale-out is invisible, inside that one instance.
2. Google **never confirms Apigee X’s Kubernetes runtime is GKE** — treat “Kubernetes cluster that Google maintains” as the full extent of the disclosure.
3. **MART is a Hybrid-only term.** Do not use it when describing Apigee X — it does not appear in current Apigee X customer documentation.
4. The 99.99% SLA tier requires multi-region **and** a qualifying Cloud KMS key — not region count by itself.
5. Apigee X publishes **no RTO/RPO** — Google frames its own live/live architecture as making recovery “real-time” from its side, but this is not a contractual number.
6. There is **no automatic cross-region failover** — Google explicitly will not move services across regions on your behalf (“this could potentially compromise Google’s commitments to its customers” — data-sovereignty framing); the customer must build the redirection strategy.
7. VPC Peering gives you exactly **one** peering hop to the tenant project — reaching many backend VPCs at scale requires Cloud VPN hub-and-spoke, Shared VPC, or PSC, not more direct peering.
8. “UAP” / “Unified Analytics Platform” is **not a confirmed official term** — say “Apigee’s analytics processing pipeline” instead.

12.2 Apigee Hybrid Cheat Sheet

The split, in one line: Google runs the **management/control plane** (UI, Management APIs, analytics processing); the customer runs the **runtime/data plane** on their own Kubernetes cluster, in the apigee namespace by default (Source: [Apigee hybrid shared responsibility model](#)).

Every runtime-plane component (Source: [Troubleshooting Apigee hybrid stuck in creating or releasing state](#)):

Component	Governing CR	K8s object	One-line purpose
apigee-runtime (Message Processor)	ApigeeEnvironment	Deployment	Executes policies/proxy flows against inbound requests; the gateway itself
apigee-synchronizer	ApigeeEnvironment	Deployment	Pulls (polls, outbound) deployment config from the control plane into local Cassandra
apigee-mart	ApigeeOrganization	Deployment	Runtime-plane API that lets the management plane read/write Cassandra-backed state (e.g. KVMs)
apigee-udca	ApigeeEnvironment	Deployment	Forwards analytics, trace, and deployment-status data to the control plane
apigee-watcher	ApigeeOrganization	Deployment	Reports MP/runtime status to the management plane; reconciles ingress TLS Secrets
apigee-connect-agent	ApigeeOrganization	Deployment + HPA + PodDisruptionBudget + ReplicaSet	Secure outbound tunnel so the management plane reaches MART without an exposed inbound endpoint
apigee-cassandra	ApigeeDatastore	StatefulSet	Persistent datastore for KVMs, response caches, quota counters — the only stateful component

Pattern to remember: **every stateless component is an ApigeeDeployment CR → Deployment; Cassandra alone is a StatefulSet**, because it is the only component needing stable per-pod identity and durable storage.

Cassandra facts:

- Stores **KVMs, response caches, and Quota counters** — nothing else.
- **Not deprecated.** Google's shared-responsibility page still explicitly lists Cassandra among the packages it delivers.
- Google-recommended production minimum: **replicaCount: 3**; a single-replica Cassandra is an explicit, documented single point of failure.
- **Cannot autoscale.** Must be scaled manually, and only **in multiples of three** (ring/replication-factor topology) (Source: [Optimize managing APIs at scale across hybrid environments](#)).
- Production node sizing floor: **minimum 8 vCPUs and 15Gi RAM per Cassandra node** (same source).
- Hard organization-wide cap: **150 Cassandra pods per organization**, which is what caps regions per organization at 10 (11 for Hybrid) (Source: [Apigee Limits reference](#)).
- ApigeeRedis **supplements** Cassandra for one narrow purpose only — coordinating in-memory Spike Arrest counts across Message Processor pods when UseEffectiveCount is set. Quota still lives in Cassandra. Redis does not replace Cassandra.
- EKS-specific gotcha: Cassandra pods can get stuck Pending due to a missing Amazon EBS CSI driver.

Failure-impact quick reference (Source: troubleshooting playbooks cited in Section 9):

Failing component	New deployments?	Existing traffic?	Severity
Synchronizer / control-plane connectivity	Blocked	Unaffected	Low–Medium
Cassandra	Blocked	Affected (CrashLoopBackOff ripples to runtime + mart + synchronizer)	High
MART	Coupled to Cassandra	Coupled to Cassandra	High when co-occurring with Cassandra failure
Ingress / TLS	N/A	Affected immediately (TLS handshake failures)	High, scoped to the affected environment group
Message Processor (apigee-runtime)	Blocked in affected environment	Reduced/affected by replica health	Medium–High

The single fact worth memorizing above all others: **a Synchronizer/control-plane outage never stops already-deployed traffic; a Cassandra outage does.** The Message Processor reads its “contract” from local Cassandra, not from a live control-plane call — this is *why* Hybrid can “function independently from the management plane.”

Shared responsibility, one-liners (Source: [Apigee hybrid shared responsibility model](#)):

- Google: ships runtime/mart/Cassandra/Kubernetes-controller container images; keeps Management APIs and Analytics compatible; scans and patches Google-provided component vulnerabilities; provides upgrade notifications; provides the control-plane/observability connection points.
- Customer: provisions and patches the Kubernetes cluster and its infrastructure; maintains connectivity from the runtime plane to Google (VPN/Interconnect); provisions and manages the load balancer and TLS certificate lifecycle; owns multi-region deployment and disaster recovery; owns RBAC/IAM and running containers; owns logging/monitoring configuration.
- No published SLA exists for Hybrid at all — DR and HA are entirely the customer’s design responsibility.

12.3 Kubernetes-for-Apigee-Hybrid Cheat Sheet

K8s primitive	Apigee Hybrid usage, in one line
Cluster	The entire customer-provisioned, customer-patched substrate the runtime plane runs on; Google’s own guidance recommends at least two, one per region, same size
Node	Where pods actually schedule; Cassandra nodes need a documented production floor of 8 vCPUs/15Gi RAM, runtime pods ~2 vCPUs/2Gi RAM
Pod	The unit every component ultimately runs as — Message Processor, Synchronizer, MART, UDCA, Watcher, Connect Agent, Cassandra all run as one or more pods
Deployment	The controller behind every stateless component (apigee-runtime, apigee-synchronizer, apigee-mart, apigee-udca, apigee-watcher, apigee-connect-agent), materialized via Apigee’s own ApigeeDeployment CR
StatefulSet	Cassandra only — the one component needing a sticky per-pod identity and durable storage, governed by ApigeeDatastore

K8s primitive	Apigee Hybrid usage, in one line
Service	Stable in-cluster addressing for every Deployment/StatefulSet, e.g. how Synchronizer reaches Cassandra regardless of which pods are healthy
Ingress	Apigee Hybrid uses an Istio-based ingress gateway rather than a bare K8s Ingress object — <code>istio-ingressgateway</code> , exposed as a LoadBalancer-type Service
ConfigMap	Non-sensitive per-component configuration, including forward-proxy settings in <code>apigee-hybrid-config.yaml</code>
Secret	Holds GCP service-account credentials (or Workload Identity/Vault-via-CSI alternatives) and TLS certificates for the ingress gateway and environment-group hostnames
PersistentVolume	Backs every Cassandra StatefulSet pod so data survives rescheduling — the EKS EBS-CSI-driver dependency lives here
HorizontalPodAutoscaler (HPA)	Autoscales <code>apigee-runtime</code> (Message Processor) by default; also paired with <code>apigee-connect-agent</code> . Cassandra is explicitly excluded — it cannot autoscale
Namespace	Default install target is a single namespace named apigee , holding every runtime-plane component by default

12.4 Memory Maps

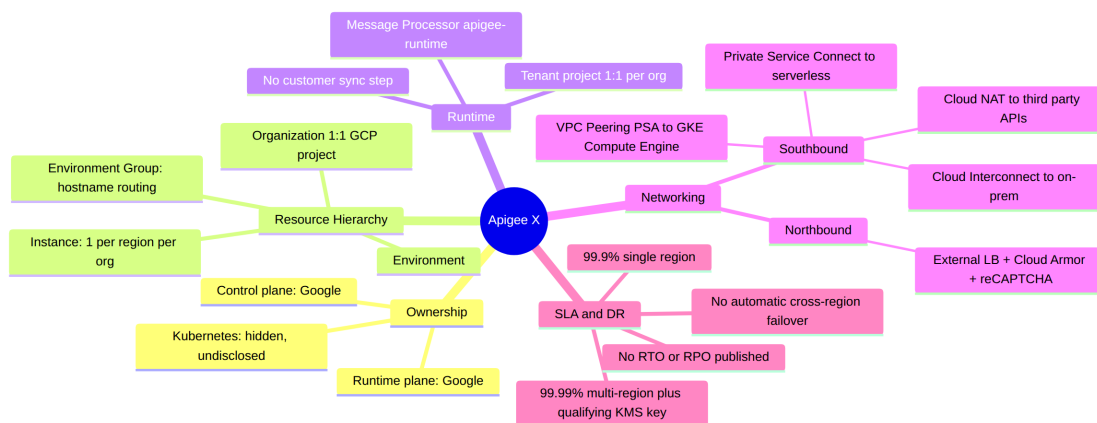


Diagram: original synthesis for revision purposes.

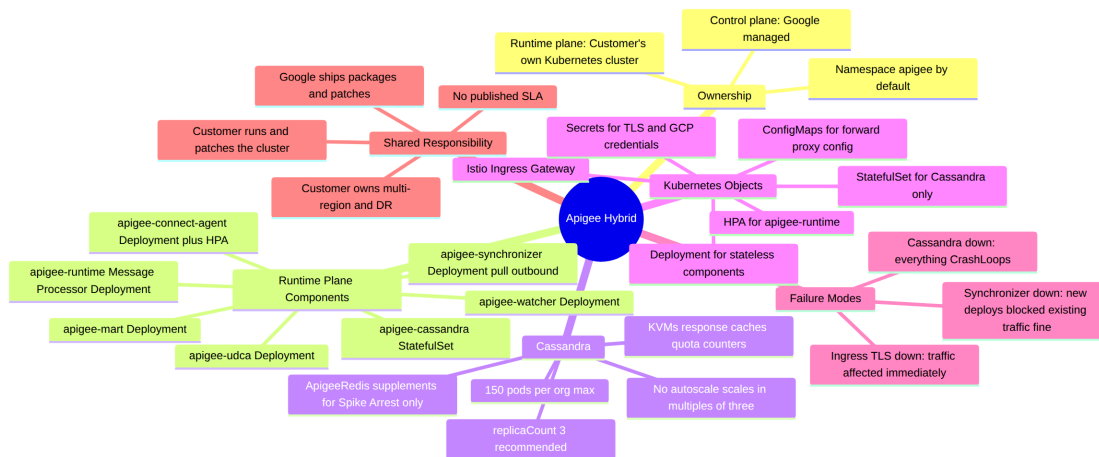


Diagram: original synthesis for revision purposes.

12.5 The One-Page Apigee X vs Hybrid Decision Cheat Sheet

Consistent with the full recommendations in Section 10.2 — use this table live in a client conversation:

Signal from the client	Lean toward
"We want to minimize infra ownership and headcount for the gateway layer."	Apigee X
"We need a contractual, numeric SLA on the gateway."	Apigee X (99.9%/99.99% — Hybrid has none)
"Our backends are already in Google Cloud, reachable by Peering/PSC/Interconnect."	Apigee X
"We have no need to tune Kubernetes underneath the gateway."	Apigee X
"Data must never leave our own network / a specific sovereign region / an air-gapped site."	Apigee Hybrid
"We already run Kubernetes at scale and have a strong platform team."	Apigee Hybrid
"Backends are on-prem, multi-cloud, or otherwise unreachable from a Google-managed tenant project."	Apigee Hybrid
"We need custom admission controllers, node-pool isolation, or a specific service-mesh/PKI/Vault setup under the gateway."	Apigee Hybrid
"We're fine building our own multi-region/DR posture and don't need Google's SLA."	Apigee Hybrid

The one diagnostic question: "When something goes wrong with the API gateway at 2 a.m., who do you want holding the pager — Google's SRE team, or your own platform team?" "Google's" → Apigee X. "Ours, because we need the control" → Apigee Hybrid, and be candid that the price of that control is Kubernetes operations, not just a license fee (Section 10.2).

Two things never to forget while making the recommendation: Apigee X has no customer Kubernetes exposure whatsoever — Hybrid's entire premise is that Kubernetes is customer-operated; and Cassandra, not "the cloud," is what makes Hybrid resilient to short control-plane outages — that resilience only exists because Hybrid *has* a local, customer-run datastore, which Apigee X architecturally does not need or expose.

Glossary

API Product — A collection of API resources (URIs) combined with a service plan and presented to developers as a bundle. This is the unit an external Developer actually subscribes to via a Developer App; it couples proxy resources to a quota/rate-limit plan.

API Proxy — A proxy that acts as a facade for an existing backend API. It decouples the public interface consumers depend on from the private backend implementation, shielding developers from backend changes while letting the backend evolve independently. Versioned as a sequence of numbered **Revisions**.

ApigeeDatastore — The custom resource (CRD) that governs the Cassandra component in Apigee Hybrid’s runtime plane; the Apigee Operator materializes it as a Kubernetes StatefulSet.

ApigeeDeployment — The custom resource (CRD) Apigee Hybrid uses to represent every stateless runtime-plane component; the Apigee Operator materializes each one as a standard Kubernetes Deployment (and, for `apigee-connect-agent`, also a HorizontalPodAutoscaler, PodDisruptionBudget, and ReplicaSet).

Apigee Connect (Agent) — A Hybrid runtime-plane component (`apigee-connect-agent`) that establishes a secure, outbound-initiated tunnel from the runtime plane to the management plane, letting Google reach MART/runtime services without the customer exposing an inbound ingress endpoint for management traffic.

Apigee Edge — The pre-Google-Cloud generation of Apigee, offered as Edge Public Cloud (SaaS) and Edge for Private Cloud (fully self-hosted on bare virtual machines, no containers or Kubernetes). Its named components — Message Processor, Router, Cassandra, ZooKeeper, Management Server — are the direct architectural ancestors of Apigee Hybrid’s runtime plane.

ApigeeEnvironment — The custom resource (CRD) scoping environment-level Hybrid runtime-plane components (`apigee-runtime`, `apigee-synchronizer`, `apigee-udca`), each materialized as a Deployment.

Apigee Hybrid — Google’s split-responsibility Apigee offering: a management/control plane hosted and maintained by Google in the cloud (UI, Management APIs, analytics processing), paired with a runtime/data plane that the customer installs, operates, and patches on their own Kubernetes cluster, on-premises or in any cloud. Designed so the runtime plane can keep serving already-deployed traffic independently of the management plane for a period of time.

ApigeeOrganization — The custom resource (CRD) scoping organization-level Hybrid runtime-plane components (`apigee-mart`, `apigee-watcher`, `apigee-connect-agent`), each materialized as a Deployment.

ApigeeRedis — A custom resource (CRD) added in recent Apigee Hybrid releases that provisions a narrow-purpose Redis instance used only to coordinate in-memory Spike Arrest counts across Message Processor pods when the `UseEffectiveCount` setting is enabled. It supplements Cassandra; it does not replace it, and Quota data continues to live in Cassandra.

ApigeeTelemetry — The custom resource (CRD) governing Apigee Hybrid’s metrics/telemetry stack, based on the Prometheus add-on, with a documented OpenTelemetry Collector integration.

Apigee X — The informal but widely used name (including in current Google Cloud Blog posts) for Apigee’s fully-managed SaaS offering, in which both the control plane and the runtime plane are operated entirely by Google. Current official documentation refers to the same product simply as “Apigee.” Deploys into a Google-managed, single-tenant tenant project bound 1:1 to the customer’s Apigee organization.

Base Path — The URL path segment that, combined with an Environment Group’s matched hostname, identifies which deployed API Proxy revision should handle a request. Apigee enforces base-path uniqueness across environments within the same Environment Group so this match is always unambiguous.

Cassandra — The persistent, distributed datastore in Apigee Hybrid’s runtime plane, storing Key Value Maps, response caches, and Quota counters. Deployed as the runtime plane’s only Kubernetes StatefulSet (governed by the `ApigeeDatastore` CRD), because it is the only component needing stable per-pod network identity and durable per-pod storage. Cannot autoscale; must be scaled manually in multiples of three. Not deprecated — still explicitly shipped by Google alongside the runtime and Kubernetes controllers.

Cloud Armor — Google Cloud’s Web Application Firewall (WAF), recommended by Google to be layered in front of Apigee at the External Application Load Balancer for IP- and geo-based access control, rather than relying solely on Apigee’s own policies for that purpose.

Cloud Interconnect — The dedicated, private connectivity service used for the southbound (Apigee-to-backend) path when backend systems live on-premises or in another data center.

Cloud KMS — Google Cloud’s key management service; the specific tier of Cloud KMS key attached to an Apigee X environment (dual-region, multi-regional, or global) is a contractual condition for qualifying for the 99.99% SLA tier, in addition to being deployed across two or more regions.

Cloud NAT — Apigee X’s default egress mechanism from the tenant project for southbound calls to non-private (public, third-party) destinations.

ConfigMap — A Kubernetes API object that stores non-confidential, key-value configuration data, decoupling environment-specific settings from container images. Apigee Hybrid uses ConfigMaps for non-sensitive per-component settings, including per-component forward-proxy configuration.

Control Plane — The half of the Apigee architecture responsible for administration: the UI, the Management APIs, and analytics processing. Always Google-managed, in both Apigee X and Apigee Hybrid — the one part of the architecture that is identical in ownership across both products.

CRD (Custom Resource Definition) / Custom Resource — A Kubernetes extension mechanism that lets an application (here, the Apigee Operator) define its own first-class API objects. Apigee Hybrid ships five: ApigeeOrganization, ApigeeEnvironment, ApigeeDatastore, ApigeeTelemetry, and ApigeeRedis.

Cluster (Kubernetes) — The set of machines (Nodes) running containerized workloads, coordinated by a Kubernetes control plane. In Apigee Hybrid, the cluster is entirely the customer’s responsibility to provision, size, and patch; Google’s own guidance recommends at least two clusters, in different regions, sized identically, for production.

DaemonSet — A Kubernetes controller that ensures a copy of a Pod runs on every (or every selected) Node in a cluster, typically used for node-local facilities such as logging or monitoring agents; some Apigee Hybrid telemetry agents may materialize this way depending on configuration.

Data Plane — See **Runtime Plane**; the two terms are used interchangeably in this handbook and in Apigee’s own documentation.

Deployment (Kubernetes) — A Kubernetes controller that manages a set of Pods running a stateless application workload. The controller behind every stateless Apigee Hybrid component (apigee-runtime, apigee-synchronizer, apigee-mart, apigee-udca, apigee-watcher, apigee-connect-agent), each instantiated via Apigee’s own ApigeeDeployment custom resource.

Developer — The registered entity, in Apigee’s model, that creates one or more Developer Apps to consume API Products.

Developer App — A credential-bearing registration created by a Developer and granted access to one or more API Products; issued a consumer key/secret pair used at runtime (typically via `VerifyAPIKey` or `OAuthV2`) to authenticate and authorize calls.

Environment — A runtime execution context for APIs. A proxy must be deployed to an environment before it can be accessed at runtime; each environment must be attached to an Instance in a specific region to actually serve traffic there.

Environment Group — The hostname-to-environment routing layer sitting above Environments. Hostnames are defined on the group, not on individual environments; Apigee routes incoming requests to the group’s member environments based on those hostname definitions. An environment can belong to more than one group.

Flow Hook — An environment-level attachment point that runs a Shared Flow automatically for every proxy deployed to that environment, with no per-proxy configuration required. Four hook points exist: `PreProxyFlowHook`, `PreTargetFlowHook`, `PostTargetFlowHook`, `PostProxyFlowHook`. Only one Shared Flow can be attached per hook, though the same Shared Flow can be attached to multiple hooks.

FlowCallout (policy) — The policy used to explicitly invoke a Shared Flow from inside a specific proxy’s own flow (`PreFlow`, `PostFlow`, or a conditional flow), as opposed to the automatic, environment-wide invocation performed by a Flow Hook.

Helm — A Kubernetes package-management tool; recent Apigee Hybrid versions support Helm charts as an alternative to `apigeectl` for installing and upgrading the runtime plane.

HorizontalPodAutoscaler (HPA) — A Kubernetes object that automatically updates a workload resource's Pod count to match demand. Apigee Hybrid's Message Processor (`apigee-runtime`) autoscales via HPA by default; Cassandra is explicitly excluded from HPA-based autoscaling and must be scaled manually, in multiples of three.

Hub-and-Spoke — A networking topology pattern used to work around the non-transitivity of VPC Peering for Apigee X southbound connectivity, implemented either over Cloud VPN (which is transitive) or via a Shared VPC hub design nested with an outer VPN-connected spoke topology.

Ingress (Kubernetes) — A Kubernetes API object that makes an HTTP/HTTPS service available via protocol-aware routing rules. Apigee Hybrid uses an Istio-based ingress layer rather than a bare Kubernetes Ingress object — see **Istio Ingress Gateway**.

Instance — The unit of regional runtime capacity within an Apigee organization. Exactly one instance is permitted per Google Cloud region per organization; an organization may have up to 10 instances/regions total (11 for Hybrid).

Istio Ingress Gateway — Apigee Hybrid's ingress layer, based on Istio (Cloud Service Mesh) rather than a bare Kubernetes Ingress object. Terminates inbound TLS (using certificates held in Kubernetes Secrets) and routes matched requests to Message Processor pods. Community-documented object names include `istio-ingressgateway` (general traffic) and `apigee-mart-istio-ingressgateway` (routing to MART, unnecessary once Apigee Connect is enabled).

Key Value Map (KVM) — A custom collection of key/value string pairs, either encrypted or unencrypted, scoped to an organization, an environment, or a single API proxy. Read and written at runtime via the `KeyValueMapOperations` policy. Encrypted KVMs use an Apigee-generated AES-128 key and cannot change their encryption setting after creation.

MART (Management API for Runtime [Data]) — The Apigee Hybrid runtime-plane service that authenticates and validates requests from the management plane before reading or writing Cassandra-backed runtime state (for example, KVM operations). Stateless itself; its availability is coupled to Cassandra's. A Hybrid-only, confirmed-current term — it does not appear in current Apigee X customer-facing documentation.

Message Processor — The core Apigee component that executes proxy flows and policies and communicates with backend targets — functionally, the API gateway itself. Implemented as containerized `apigee-runtime` pods in both Apigee X (Google-managed, internals undisclosed) and Apigee Hybrid (customer-managed Deployment, reading configuration from local Cassandra).

Namespace (Kubernetes) — A Kubernetes mechanism for isolating groups of resources within a single cluster. Apigee Hybrid's default install target is a single namespace named `apigee`, holding every runtime-plane component by default.

Node (Kubernetes) — A machine (virtual or physical) that is a member of a Kubernetes cluster and runs Pods. Apigee Hybrid product guidance gives documented production sizing floors: a minimum of 8 vCPUs and 15Gi of RAM for Cassandra nodes, and a recommended 2 vCPUs and 2Gi of RAM for runtime (Message Processor) pods.

Northbound — The client-facing (ingress) side of Apigee's traffic path: external clients through a load balancer and WAF layer into Apigee. Contrasted with **Southbound**.

Operator (Kubernetes / Apigee) — A Kubernetes pattern in which a custom controller manages custom resources to automate application-specific operational tasks. The Apigee Operator manages Apigee Hybrid's five custom resources (`ApigeeOrganization`, `ApigeeEnvironment`, `ApigeeDatastore`, `ApigeeTelemetry`, `ApigeeRedis`), translating them into standard Kubernetes objects (Deployments, a `StatefulSet`, Services, Secrets, ConfigMaps, HPAs).

Organization — A container for all the entities in an Apigee account, including API proxies, API products, apps, and developers. Bound 1:1 to a single Google Cloud project; this binding is what lets Apigee X integrate directly with Cloud IAM, Cloud Billing, and Cloud KMS.

PersistentVolume (PV) — A piece of cluster storage provisioned by an administrator or dynamically via a Storage Class, with a lifecycle independent of any individual Pod. Cassandra is the Apigee Hybrid component that requires PersistentVolumes, so that data survives pod rescheduling; the underlying cloud-provider CSI driver (for example, the Amazon EBS CSI driver on EKS) is a real, documented prerequisite.

Pod (Kubernetes) — The smallest deployable Kubernetes unit: one or more containers sharing network and storage. Every Apigee Hybrid runtime-plane component ultimately runs as one or more Pods, managed by either a Deployment or (for Cassandra) a StatefulSet.

PodDisruptionBudget — A Kubernetes object limiting how many Pods of a workload can be voluntarily disrupted at once; documented as part of the default configuration for the `apigee-connect-agent` Deployment.

Policy — A processing step that executes as an atomic, reusable unit of logic within an API flow — covering routing, message transformation, access control, threat detection, caching, and more. The fundamental building block of both API Proxies and Shared Flows.

PreFlow / PostFlow — The proxy-level flow points where policies execute before (PreFlow) or after (PostFlow) the core request/response processing of an API Proxy, distinct from the environment-wide Flow Hook points.

Private Service Connect (PSC) — A Google Cloud networking capability allowing private consumption of services across VPC networks without exposing underlying infrastructure. Used as the scalable alternative to VPC Peering for Apigee X southbound connectivity to backends in other VPCs (via Endpoint Attachment → Service Attachment → internal load balancer, all in the same region), and also documented as an alternative northbound connectivity mechanism.

Private Services Access (PSA) — The networking model underlying Apigee X's default VPC connectivity: Apigee X is deployed and managed in a Google-owned tenant project, connected to the customer's VPC via VPC Network Peering, in a 1:1 relationship between tenant project and Apigee organization.

Revision — A sequentially numbered version of a deployed API Proxy. Revisions let a proxy be updated and rolled back by deploying a previous revision number; only one revision sharing a given base path can be deployed in an environment at a time.

ReplicaSet — A Kubernetes controller ensuring a specified number of identical Pod replicas are running at any time; typically managed indirectly through a Deployment. Documented as part of the `apigee-connect-agent` Deployment's default materialization.

Router — The Apigee Edge for Private Cloud component that terminated inbound traffic and load-balanced it to Message Processors; its architectural role is filled by the Istio Ingress Gateway in Apigee Hybrid.

Runtime Plane (Data Plane) — The half of the Apigee architecture responsible for actually processing API traffic. In Apigee X, fully Google-managed inside a tenant project, internals undisclosed. In Apigee Hybrid, customer-provisioned and customer-operated on the customer's own Kubernetes cluster.

Secret (Kubernetes) — A Kubernetes API object holding a small amount of sensitive data (passwords, tokens, keys) outside application code. Apigee Hybrid uses Secrets for Google Cloud service-account credentials (or Workload Identity/Vault-via-CSI alternatives) and for TLS certificates used by the Istio Ingress Gateway and environment-group hostnames.

Service (Kubernetes) — A Kubernetes object exposing a set of Pods behind a single, stable outward-facing endpoint regardless of which specific backend Pods are currently healthy. Every Apigee Hybrid Deployment and the Cassandra StatefulSet sit behind a Service for stable in-cluster addressing.

Shared Flow — A reusable bundle of policies and resources that can be consumed from multiple API proxies (and from other Shared Flows). Unlike a proxy, it has no endpoint of its own and can only be used from within the same organization. Invoked either explicitly via the `FlowCallout` policy or implicitly by attaching it to a Flow Hook.

Shared VPC — A Google Cloud networking construct in which a host project shares its VPC network with other (service) projects. Used as a hub design for scaling Apigee X's southbound connectivity beyond the single VPC-peering hop available to the tenant project.

Southbound — The backend-facing (egress) side of Apigee's traffic path: from Apigee out to GKE/Compute Engine backends, serverless/internal services, on-premises systems, or third-party APIs. Contrasted with **Northbound**.

StatefulSet (Kubernetes) — A Kubernetes controller that runs a group of Pods while maintaining a sticky identity and stable storage for each Pod, used for applications needing persistent storage or a stable network identity. Cassandra is the only Apigee Hybrid component deployed as a StatefulSet, governed by the `ApigeeDataStore` custom resource.

Synchronizer — The Apigee Hybrid runtime-plane component (`apigee-synchronizer`) that pulls (polls) deployment configuration — proxy bundles, environment configuration, KVM metadata — from the control plane's Man-

agement API and writes it into the local Cassandra datastore. Operates as an outbound-only pull model so no inbound firewall port needs to be opened toward Google. Google does not publish an exact polling interval.

Tenant Project — The Google-owned, single-tenant Google Cloud project that hosts an Apigee X organization's runtime, in a strict 1:1 relationship with that organization. Resides in a Google-managed VPC, connected to the customer's VPC via Private Services Access/VPC Peering or Private Service Connect.

UDCA (Universal Data Collection Agent) — The Apigee Hybrid runtime-plane component (`apigee-udca`) that extracts analytics, distributed-trace, and deployment-status data from the runtime plane and forwards it to the management plane's analytics processing pipeline, making it visible in the same Apigee Analytics UI/API used by Apigee X.

VPC Peering — Virtual Private Cloud Network Peering; the default connectivity mechanism between a customer's VPC and Apigee X's Google-managed tenant project. Non-transitive — only one peering hop is available, and Apigee X already consumes it, which is why hub-and-spoke and Private Service Connect patterns exist for reaching additional backend VPCs.

Watcher (`apigee-watcher`) — The Apigee Hybrid runtime-plane component that reports Message Processor and runtime status up to the management plane, and reconciles ingress-related TLS Secret state inside the cluster (for example, detecting a missing TLS secret required by an environment group).

ZooKeeper — The coordination service used by Apigee Edge for Private Cloud for cluster membership and configuration propagation among Edge nodes. Dropped in Apigee Hybrid, whose coordination role is filled by Kubernetes itself.

Bibliography

Primary sources — directly fetched and quoted, [cloud.google.com / docs.cloud.google.com](https://cloud.google.com/docs.cloud.google.com)

1. What is Apigee? — <https://cloud.google.com/apigee/docs/api-platform/get-started/what-apigee-old>
2. Apigee hybrid shared responsibility model — <https://cloud.google.com/apigee/docs/hybrid/shared-responsibility-model>
3. Apigee Limits reference — <https://docs.cloud.google.com/apigee/docs/api-platform/reference/limits>
4. Apigee SLA — <https://cloud.google.com/apigee/sla>
5. Business continuity planning and disaster recovery — <https://docs.cloud.google.com/apigee/docs/api-platform/reference/business-continuity>
6. Best practices for securing your applications and APIs using Apigee (Cloud Architecture Center) — <https://cloud.google.com/architecture/best-practices-securing-applications-and-apis-using-apigee>
7. Message Processor error catalog — <https://docs.cloud.google.com/apigee/docs/api-platform/errorcatalog/mp-runtime-errorcatalog>
8. Troubleshooting playbook: message processor — <https://docs.cloud.google.com/apigee/docs/api-platform/troubleshoot/playbooks/ts-message-processor>
9. Troubleshooting playbook: no network connectivity — <https://docs.cloud.google.com/apigee/docs/api-platform/troubleshoot/playbooks/no-network-connectivity>
10. Troubleshooting playbook: Apigee hybrid stuck in creating or releasing state — <https://docs.cloud.google.com/apigee/docs/api-platform/troubleshoot/playbooks/troubleshooting-apigee-hybrid>
11. Troubleshooting playbook: no active runtime pods warning — <https://docs.cloud.google.com/apigee/docs/api-platform/troubleshoot/playbooks/deployment/no-active-runtime-pods-warning>
12. Troubleshooting playbook: Cassandra (ts-cassandra) — <https://docs.cloud.google.com/apigee/docs/api-platform/troubleshoot/playbooks/cassandra/ts-cassandra>
13. Troubleshooting playbook: API calls failing with TLS errors — <https://docs.cloud.google.com/apigee/docs/api-platform/troubleshoot/playbooks/api-calls-failing-tls-errors>
14. Apigee hybrid release notes — <https://docs.cloud.google.com/apigee/docs/hybrid/release-notes>
15. Apigee hybrid installation and administration guide (preview) — <https://docs.cloud.google.com/apigee/docs/hybrid/preview/new-install-user-guide>

Official Google Cloud Blog (Google-authored)

16. Understanding commonly used Apigee terms — <https://cloud.google.com/blog/products/api-management/understanding-commonly-used-apigee-terms>
17. Apigee X design for multiple distinct VPC connections — <https://cloud.google.com/blog/products/application-modernization/apigee-x-design-for-multiple-distinct-vpc-connections/>
18. Connecting to Cloud Run from Apigee X using PSC — <https://cloud.google.com/blog/products/serverless/connecting-to-cloud-run-from-apigee-x-using-psc>
19. Optimize managing APIs at scale across hybrid environments — <https://cloud.google.com/blog/products/api-management/optimize-managing-apis-at-scale-across-hybrid-environments>

Further reading — official URLs confirmed to exist (titles verified); consult directly for current, version-specific detail

20. Apigee architecture overview — <https://cloud.google.com/apigee/docs/api-platform/architecture/overview>
21. Understanding organizations — <https://cloud.google.com/apigee/docs/api-platform/fundamentals/organization-structure>
22. About environments and environment groups — <https://cloud.google.com/apigee/docs/api-platform/fundamentals/environments-overview>

23. Apigee networking options — <https://cloud.google.com/apigee/docs/api-platform/get-started/networking-options>
24. Northbound networking with Private Service Connect — <https://cloud.google.com/apigee/docs/api-platform/system-administration/northbound-networking-psc>
25. Southbound networking patterns — <https://cloud.google.com/apigee/docs/api-platform/architecture/southbound-networking-patterns-endpoints>
26. Peering ranges — <https://cloud.google.com/apigee/docs/api-platform/system-administration/peering-ranges>
27. Using shared VPC networks — <https://cloud.google.com/apigee/docs/api-platform/system-administration/shared-vpcs>
28. Apigee glossary — <https://cloud.google.com/apigee/docs/api-platform/reference/glossary>
29. What is Apigee hybrid? (v1.16) — <https://cloud.google.com/apigee/docs/hybrid/v1.16/what-is-hybrid>
30. Apigee hybrid — the big picture — <https://cloud.google.com/apigee/docs/hybrid/v1.16/big-picture>
31. Configure the Synchronizer — <https://cloud.google.com/apigee/docs/hybrid/v1.16/synchronizer-access>
32. Apigee Connect — <https://cloud.google.com/apigee/docs/hybrid/v1.16/apigee-connect>
33. Minimum cluster configurations — <https://cloud.google.com/apigee/docs/hybrid/v1.9/cluster-overview>
34. Scale and autoscale runtime services — <https://cloud.google.com/apigee/docs/hybrid/v1.14/scale-and-autoscale>
35. Configure Cassandra for production — <https://cloud.google.com/apigee/docs/hybrid/v1.13/cassandra-production>
36. Scaling Cassandra — <https://cloud.google.com/apigee/docs/hybrid/v1.14/cassandra-scaling>
37. Multi-region deployment (hybrid) — <https://cloud.google.com/apigee/docs/hybrid/v1.14/multi-region>
38. Managing Apigee ingress gateway — <https://cloud.google.com/apigee/docs/hybrid/v1.9/managing-ingress>
39. Apigee hybrid supported platforms — <https://cloud.google.com/apigee/docs/hybrid/supported-platforms>
40. apigeectl CLI reference — <https://cloud.google.com/apigee/docs/hybrid/v1.11/cli-reference>
41. Expanding Apigee to multiple regions (Apigee X) — <https://cloud.google.com/apigee/docs/api-platform/system-administration/multi-region>
42. Apigee API Analytics overview — <https://cloud.google.com/apigee/docs/api-platform/analytics/analytics-services-overview>
43. Analytics metrics, dimensions, and filters reference — <https://cloud.google.com/apigee/docs/api-platform/analytics/analytics-reference>
44. Debug, analytics, and deployment status data collection (hybrid) — <https://cloud.google.com/apigee/docs/hybrid/v1.16/other-data-collection>
45. Deploy a proxy (Apigee X) — <https://cloud.google.com/apigee/docs/api-platform/get-started/deploy-proxy>
46. Deploy a proxy (hybrid) — <https://cloud.google.com/apigee/docs/hybrid/v1.16/install-deploy-proxy>
47. Key Value Maps — <https://cloud.google.com/apigee/docs/api-platform/cache/key-value-maps>
48. Shared flows — <https://cloud.google.com/apigee/docs/api-platform/fundamentals/shared-flows>
49. Flow hooks — <https://cloud.google.com/apigee/docs/api-platform/fundamentals/flow-hooks>
50. Working with environment groups — <https://cloud.google.com/apigee/docs/api-platform/fundamentals/environmentgroups-working-with>
51. Apigee hybrid configuration property reference — <https://cloud.google.com/apigee/docs/hybrid/v1.14/config-prop-ref>

Kubernetes upstream documentation (generic concept definitions)

52. Deployments — <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
53. StatefulSets — <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>
54. DaemonSet — <https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/>
55. Jobs — <https://kubernetes.io/docs/concepts/workloads/controllers/job/>
56. Service — <https://kubernetes.io/docs/concepts/services-networking/service/>
57. Ingress — <https://kubernetes.io/docs/concepts/services-networking/ingress/>
58. Secrets — <https://kubernetes.io/docs/concepts/configuration/secret/>
59. ConfigMaps — <https://kubernetes.io/docs/concepts/configuration/configmap/>
60. Persistent Volumes — <https://kubernetes.io/docs/concepts/storage/persistent-volumes/>
61. Namespaces — <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/>
62. HorizontalPodAutoscaler — <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
63. kubectl — <https://kubernetes.io/docs/reference/kubectl/>

Secondary / non-official sources (used only for corroboration or historical context, always flagged inline where used)

64. Apigee Edge → Apigee X comparison (legacy Apigee documentation domain) — <https://docs.apigee.com/migration-to-x/compare-apigee-edge-to-apigee-x>
65. What is Apigee Edge? (legacy) — <https://docs.apigee.com/api-platform/get-started/what-apigee-edge>
66. Understanding deployment (legacy) — <https://docs.apigee.com/api-platform/deploy/understanding-deployment>
67. Shared flows (legacy) — <https://docs.apigee.com/api-platform/fundamentals/shared-flows>
68. Flow hooks (legacy) — <https://docs.apigee.com/api-platform/fundamentals/flow-hooks>
69. Key Value Maps (legacy) — <https://docs.apigee.com/api-platform/cache/key-value-maps>
70. Apigee hybrid 1.1.0 release notes (legacy) — <https://docs.apigee.com/release/notes/110-hybrid-runtime-release-notes>
71. TechCrunch — “Google Cloud launches Apigee X” (Feb 4, 2021) — <https://techcrunch.com/2021/02/04/google-cloud-launches-apigee-x-the-next-generation-of-its-api-management-platform/>
72. Google Cloud Community — “Apigee Hybrid off the Grid: What happens when the internet goes down?” — <https://discuss.google.dev/t/apigee-hybrid-off-the-grid-what-happens-when-the-internet-goes-down/158121>